

Entrez® Programming Utilities Help

Last Updated: October 8, 2026



National Center for Biotechnology Information (US)
Bethesda (MD)

National Center for Biotechnology Information (US), Bethesda (MD)

NLM Citation: Entrez® Programming Utilities Help [Internet]. Bethesda (MD): National Center for Biotechnology Information (US); 2010-.

- Please see the [E-utilities Introduction](#) video for a brief introduction.
- Please see the [Release Notes](#) for details and changes.

The Entrez® Programming Utilities (E-utilities) are a set of eight server-side programs that provide a stable interface into the Entrez query and database system at the National Center for Biotechnology Information (NCBI). The E-utilities use a fixed URL syntax that translates a standard set of input parameters into the values necessary for various NCBI software components to search for and retrieve the requested data. The E-utilities are therefore the structured interface to the Entrez system, which currently includes 38 databases covering a variety of biomedical data, including nucleotide and protein sequences, gene records, three-dimensional molecular structures, and the biomedical literature.

Table of Contents

E-utilities Quick Start	1
Release Notes	1
Announcement	1
Introduction	1
Searching a Database	1
Uploading UIDs to Entrez	3
Downloading Document Summaries	4
Downloading Full Records	7
Finding Related Data Through Entrez Links	8
Getting Database Statistics and Search Fields	9
Performing a Global Entrez Search	10
Retrieving Spelling Suggestions	11
Demonstration Programs	12
For More Information	16
A General Introduction to the E-utilities	17
Introduction	17
Usage Guidelines and Requirements	17
The Nine E-utilities in Brief	19
Understanding the E-utilities Within Entrez	20
Combining E-utility Calls to Create Entrez Applications	24
Demonstration Programs	25
For More Information	25
Sample Applications of the E-utilities	27
Introduction	27
Basic Pipelines	27
ESearch – ESummary/EFetch	27
EPost – ESummary/EFetch	28
ELink – ESummary/EFetch	29
ESearch – ELink – ESummary/EFetch	30
EPost – ELink – ESummary/EFetch	31
EPost – ESearch	32
ELink – ESearch	33

Application 1: Converting GI numbers to accession numbers.....	33
Application 2: Converting accession numbers to data.....	34
Application 3: Retrieving large datasets.....	35
Application 4: Finding unique sets of linked records for each member of a large dataset.....	35
Demonstration Programs.....	37
For More Information.....	37
The E-utilities In-Depth: Parameters, Syntax and More.....	39
Introduction.....	39
General Usage Guidelines.....	39
E-utilities DTDs.....	39
EInfo.....	40
ESearch.....	40
EPost.....	44
ESummary.....	45
EFetch.....	47
ELink.....	54
EGQuery.....	59
ESpell.....	60
ECitMatch.....	60
Release Notes.....	61
Demonstration Programs.....	62
For More Information.....	62
The E-utility Web Service (SOAP).....	63
Termination Announcement.....	63
For More Information.....	63
Entrez Direct: E-utilities on the Unix Command Line.....	65
Abstract.....	65
Acknowledgments.....	65
Installation.....	66
Introduction.....	66
Discovery by Navigation.....	69
Searching and Filtering.....	70
XML Data Extraction.....	76

Advanced Operations	83
Complex Objects	86
Expanding Horizons	89
Biological Data in Entrez	92
Sequence Analysis	98
Alignment Excavation	102
Local PubMed Archive	104
Adding a New Local Archive	108
Python Integration	114
Shell Scripting	115
Compiled Go Programs	116
Additional Examples	118
Summary	118
Appendices	119
References	121
Release Notes	121
For More Information	122

E-utilities Quick Start

Eric Sayers, PhD¹

Created: December 12, 2008; Updated: October 24, 2018.

Release Notes

Please see our Release Notes for details on recent changes and updates.

Announcement

On December 1, 2018, NCBI will begin enforcing the use of new API keys for E-utility calls. Please see Chapter 2 for more details about this important change.

Introduction

This chapter provides a brief overview of basic E-utility functions along with examples of URL calls. Please see Chapter 2 for a general introduction to these utilities and Chapter 4 for a detailed discussion of syntax and parameters.

Examples include live URLs that provide sample outputs.

All E-utility calls share the same base URL:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/>

Searching a Database

Basic Searching

`esearch.fcgi?db=<database>&term=<query>`

Input: Entrez database (&db); Any Entrez text query (&term)

Output: List of UIDs matching the Entrez query

Example: Get the PubMed IDs (PMIDs) for articles about breast cancer published in Science in 2008

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?db=pubmed&term=science\[journal\]+AND+breast+cancer+AND+2008\[pdat\]](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?db=pubmed&term=science[journal]+AND+breast+cancer+AND+2008[pdat])

Storing Search Results

`esearch.fcgi?db=<database>&term=<query>&usehistory=y`

Input: Any Entrez text query (&term); Entrez database (&db); &usehistory=y

Output: Web environment (&WebEnv) and query key (&query_key) parameters specifying the location on the Entrez history server of the list of UIDs matching the Entrez query

Example: Get the PubMed IDs (PMIDs) for articles about breast cancer published in Science in 2008, and store them on the Entrez history server for later use

Author Affiliation: 1 NCBI; Email: sayers@ncbi.nlm.nih.gov.

¹ Corresponding author.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=pubmed&term=science\[journal\]+AND+breast+cancer+AND+2008\[pdat\]&usehistory=y](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=pubmed&term=science[journal]+AND+breast+cancer+AND+2008[pdat]&usehistory=y)

Associating Search Results with Existing Search Results

`efetch.fcgi?db=<database>&term=<query1>&usehistory=y`

efetch produces WebEnv value (\$web1) and QueryKey value (\$key1)

`efetch.fcgi?db=<database>&term=<query2>&usehistory=y&WebEnv=$web1`

efetch produces WebEnv value (\$web2) that contains the results of both searches (\$key1 and \$key2)

Input: Any Entrez text query (&term); Entrez database (&db); &usehistory=y; Existing web environment (&WebEnv) from a prior E-utility call

Output: Web environment (&WebEnv) and query key (&query_key) parameters specifying the location on the Entrez history server of the list of UIDs matching the Entrez query

For More Information

Please see ESearch In-Depth for a full description of ESearch.

Sample ESearch Output

```
<?xml version="1.0" ?>
<!DOCTYPE eSearchResult PUBLIC "-//NLM//DTD eSearchResult, 11 May 2002//EN"
  "https://www.ncbi.nlm.nih.gov/entrez/query/DTD/eSearch_020511.dtd">
<eSearchResult>
  <Count>255147</Count>    # total number of records matching query
  <RetMax>20</RetMax># number of UIDs returned in this XML; default=20
  <RetStart>0</RetStart># index of first record returned; default=0
  <QueryKey>1</QueryKey># QueryKey, only present if &usehistory=y
  <WebEnv>0l93yIkBjmM60UBXuvBvPfBIq8-9nIsldXuMP0hhuMH-
  8GjCz7F_Dz1XL6z@397033B29A81FB01_0038SID</WebEnv>
    # WebEnv; only present if &usehistory=y
  <IdList>
    <Id>229486465</Id>    # list of UIDs returned
    <Id>229486321</Id>
    <Id>229485738</Id>
    <Id>229470359</Id>
    <Id>229463047</Id>
    <Id>229463037</Id>
    <Id>229463022</Id>
    <Id>229463019</Id>
    <Id>229463007</Id>
    <Id>229463002</Id>
    <Id>229463000</Id>
    <Id>229462974</Id>
    <Id>229462961</Id>
    <Id>229462956</Id>
    <Id>229462921</Id>
    <Id>229462905</Id>
    <Id>229462899</Id>
    <Id>229462873</Id>
    <Id>229462863</Id>
    <Id>229462862</Id>
```

```

</IdList>
<TranslationSet>          # details of how Entrez translated the query
  <Translation>
    <From>mouse[orgn]</From>
    <To>"Mus musculus"[Organism]</To>
  </Translation>
</TranslationSet>
<TranslationStack>
  <TermSet>
    <Term>"Mus musculus"[Organism]</Term>
    <Field>Organism</Field>
    <Count>255147</Count>
    <Explode>Y</Explode>
  </TermSet>
  <OP>GROUP</OP>
</TranslationStack>
<QueryTranslation>"Mus musculus"[Organism]</QueryTranslation>
</eSearchResult>

```

Searching PubMed with Citation Data

`ecitmatch.cgi?db=pubmed&rettype=xml&bdata=<citations>`

Input: List of citation strings separated by a carriage return (%0D), where each citation string has the following format:

`journal_title|year|volume|first_page|author_name|your_key|`

Output: A list of citation strings with the corresponding PubMed ID (PMID) appended.

Example: Search PubMed for the following citations:

Art1: Mann, BJ. (1991) *Proc. Natl. Acad. Sci. USA*. 88:3248

Art2: Palmenberg, AC. (1987) *Science* 235:182

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ecitmatch.cgi?](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ecitmatch.cgi?db=pubmed&retmode=xml&bdata=proc+natl+acad+sci+u+s+a|1991|88|3248|mann+bj|Art1|%0Dscience|1987|235|182|palmenberg+ac|Art2|)

[db=pubmed&retmode=xml&bdata=proc+natl+acad+sci+u+s+a|1991|88|3248|mann+bj|Art1|%0Dscience|1987|235|182|palmenberg+ac|Art2|](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ecitmatch.cgi?db=pubmed&retmode=xml&bdata=proc+natl+acad+sci+u+s+a|1991|88|3248|mann+bj|Art1|%0Dscience|1987|235|182|palmenberg+ac|Art2|)

Sample Output (the PMIDs appear in the rightmost field):

```

proc natl acad sci u s a|1991|88|3248|mann bj|Art1|2014248
science|1987|235|182|palmenberg ac|Art2|3026048

```

Please see ECitMatch In-Depth for a full description of ECitMatch.

Uploading UIDs to Entrez

Basic Uploading

`epost.fcgi?db=<database>&id=<uid_list>`

Input: List of UIDs (&id); Entrez database (&db)

Output: Web environment (&WebEnv) and query key (&query_key) parameters specifying the location on the Entrez history server of the list of uploaded UIDs

Example: Upload five Gene IDs (7173,22018,54314,403521,525013) for later processing.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/epost.fcgi?db=gene&id=7173,22018,54314,403521,525013>

Associating a Set of UIDs with Previously Posted Sets

```
epost.fcgi?db=<database1>&id=<uid_list1>
```

epost produces WebEnv value (\$web1) and QueryKey value (\$key1)

```
epost.fcgi?db=<database2>&id=<uid_list2>&WebEnv=$web1
```

epost produces WebEnv value (\$web2) that contains the results of both posts (\$key1 and \$key2)

Input: List of UIDs (&id); Entrez database (&db); Existing web environment (&WebEnv)

Output: Web environment (&WebEnv) and query key (&query_key) parameters specifying the location on the Entrez history server of the list of uploaded UIDs

For More Information

Please see EPost In-Depth for a full description of EPost.

Sample EPost Output

```
<?xml version="1.0"?>
<!DOCTYPE ePostResult PUBLIC "-//NLM//DTD ePostResult, 11 May 2002//EN"
  "https://www.ncbi.nlm.nih.gov/entrez/query/DTD/ePost_020511.dtd">
<ePostResult>
<QueryKey>1</QueryKey>
<WebEnv>NCID_01_268116914_130.14.18.47_9001_1241798628</WebEnv>
</ePostResult>
```

Downloading Document Summaries

Basic Downloading

```
esummary.fcgi?db=<database>&id=<uid_list>
```

Input: List of UIDs (&id); Entrez database (&db)

Output: XML DocSums

Example: Download DocSums for these protein GIs: 6678417,9507199,28558982,28558984,28558988,28558990

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=protein&id=6678417,9507199,28558982,28558984,28558988,28558990>

Downloading Data From a Previous Search

```
esearch.fcgi?db=<database>&term=<query>&usehistory=y
```

esearch produces WebEnv value (\$web1) and QueryKey value (\$key1)

```
esummary.fcgi?db=<database>&query_key=$key1&WebEnv=$web1
```

Input: Web environment (&WebEnv) and query key (&query_key) representing a set of Entrez UIDs on the Entrez history server

Output: XML DocSums

Sample ESummary Output

The output of ESummary is a series of XML “DocSums” (Document Summaries), the format of which depends on the database. Below is an example DocSum for Entrez Protein.

```
<?xml version="1.0"?>
<!DOCTYPE eSummaryResult PUBLIC "-//NLM//DTD eSummaryResult, 29 October
2004//EN" "https://www.ncbi.nlm.nih.gov/entrez/query/DTD/eSummary_
041029.dtd">
<eSummaryResult>
  <DocSum>
    <Id>15718680</Id>
    <Item Name="Caption" Type="String">NP_005537</Item>
    <Item Name="Title" Type="String">IL2-inducible T-cell kinase [Homo
sapiens]</Item>
    <Item Name="Extra"
Type="String">gi|15718680|ref|NP_005537.3|[15718680]</Item>
    <Item Name="Gi" Type="Integer">15718680</Item>
    <Item Name="CreateDate" Type="String">1999/06/09</Item>
    <Item Name="UpdateDate" Type="String">2009/04/05</Item>
    <Item Name="Flags" Type="Integer">512</Item>
    <Item Name="TaxId" Type="Integer">9606</Item>
    <Item Name="Length" Type="Integer">620</Item>
    <Item Name="Status" Type="String">live</Item>
    <Item Name="ReplacedBy" Type="String"></Item>
    <Item Name="Comment" Type="String"><![CDATA[  ]]></Item>
  </DocSum>
</eSummaryResult>
```

Sample ESummary version 2.0 Output

Version 2.0 of ESummary is an alternate XML presentation of Entrez DocSums. To retrieve version 2.0 DocSums, the URL should contain the &version parameter with an assigned value of ‘2.0’. Each Entrez database provides its own unique DTD for version 2.0 DocSums, and a link to the relevant DTD is provided in the header of the version 2.0 XML.

esummary.fcgi?db=<database>&id=<uid_list>&version=2.0

Below is an example version 2.0 DocSum from Entrez Protein (the same record as shown above in the default DocSum XML).

```
<?xml version="1.0"?>
<!DOCTYPE eSummaryResult PUBLIC "-//NLM//DTD eSummaryResult//EN" "https://
www.ncbi.nlm.nih.gov/entrez/query/DTD/eSummaryDTD/eSummary_protein.dtd">
<eSummaryResult>
  <DocumentSummarySet status="OK">
    <DocumentSummary uid="15718680">
      <Caption>NP_005537</Caption>
      <Title>tyrosine-protein kinase ITK/TSK [Homo sapiens]</Title>
      <Extra>gi|15718680|ref|NP_005537.3|</Extra>
      <Gi>15718680</Gi>

      <CreateDate>1999/06/09</CreateDate>
      <UpdateDate>2011/10/09</UpdateDate>
      <Flags>512</Flags>
      <TaxId>9606</TaxId>
```

```

<Slen>620</Slen>

<Biomol/>

<MolType>aa</MolType>
<Topology>linear</Topology>
<SourceDb>refseq</SourceDb>
<SegSetSize>0</SegSetSize>
<ProjectId>0</ProjectId>
<Genome>genomic</Genome>

<SubType>chromosome|map</SubType>
<SubName>5|5q31-q32</SubName>
<AssemblyGi>399658</AssemblyGi>
<AssemblyAcc>D13720.1</AssemblyAcc>
<Tech/>
<Completeness/>
<GeneticCode>1</GeneticCode>

<Strand/>
<Organism>Homo sapiens</Organism>
<Statistics>
  <Stat type="all" count="8"/>
  <Stat type="blob_size" count="16154"/>
  <Stat type="cdregion" count="1"/>
  <Stat type="cdregion" subtype="CDS" count="1"/>
  <Stat type="gene" count="1"/>
  <Stat type="gene" subtype="Gene" count="1"/>
  <Stat type="org" count="1"/>
  <Stat type="prot" count="1"/>
  <Stat type="prot" subtype="Prot" count="1"/>
  <Stat type="pub" count="14"/>
  <Stat type="pub" subtype="PubMed" count="10"/>
  <Stat type="pub" subtype="PubMed/Gene-rif" count="4"/>
  <Stat type="site" count="4"/>
  <Stat type="site" subtype="Site" count="4"/>
  <Stat source="CDD" type="all" count="15"/>
  <Stat source="CDD" type="region" count="6"/>
  <Stat source="CDD" type="region" subtype="Region" count="6"/>
  <Stat source="CDD" type="site" count="9"/>
  <Stat source="CDD" type="site" subtype="Site" count="9"/>
  <Stat source="HPRD" type="all" count="3"/>
  <Stat source="HPRD" type="site" count="3"/>
  <Stat source="HPRD" type="site" subtype="Site" count="3"/>
  <Stat source="SNP" type="all" count="31"/>
  <Stat source="SNP" type="imp" count="31"/>
  <Stat source="SNP" type="imp" subtype="variation" count="31"/>
  <Stat source="all" type="all" count="57"/>
  <Stat source="all" type="blob_size" count="16154"/>
  <Stat source="all" type="cdregion" count="1"/>
  <Stat source="all" type="gene" count="1"/>
  <Stat source="all" type="imp" count="31"/>
  <Stat source="all" type="org" count="1"/>
  <Stat source="all" type="prot" count="1"/>
  <Stat source="all" type="pub" count="14"/>
  <Stat source="all" type="region" count="6"/>
  <Stat source="all" type="site" count="16"/>
</Statistics>
<AccessionVersion>NP_005537.3</AccessionVersion>

```

```

        <Properties aa="2">2</Properties>
        <Comment/>
        <OSLT indexed="yes">NP_005537.3</OSLT>
        <IdGiClass mol="3" repr="2" gi_state="10" sat="4" sat_key="58760802"
owner="20"
        sat_name="NCBI" owner_name="NCBI-Genomes" defdiv="GNM" length="620"
extfeatmask="41"
        />
    </DocumentSummary>

</DocumentSummarySet>
</eSummaryResult>

```

Downloading Full Records

Basic Downloading

```
efetch.fcgi?db=<database>&id=<uid_list>&rettype=<retrieval_type>
&retmode=<retrieval_mode>
```

Input: List of UIDs (&id); Entrez database (&db); Retrieval type (&rettype); Retrieval mode (&retmode)

Output: Formatted data records as specified

Example: Download nuccore GIs 34577062 and 24475906 in FASTA format

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=nuccore&id=34577062,24475906&rettype=fasta&retmode=text
```

Downloading Data From a Previous Search

```
esearch.fcgi?db=<database>&term=<query>&usehistory=y
```

esearch produces WebEnv value (\$web1) and QueryKey value (\$key1)

```
efetch.fcgi?db=<database>&query_key=$key1&WebEnv=$web1&rettype=
<retrieval_type>&retmode=<retrieval_mode>
```

Input: Entrez database (&db); Web environment (&WebEnv) and query key (&query_key) representing a set of Entrez UIDs on the Entrez history server; Retrieval type (&rettype); Retrieval mode (&retmode)

Output: Formatted data records as specified

Downloading a Large Set of Records

Please see Application 3 **in Chapter 3**

Input: Entrez database (&db); Web environment (&WebEnv) and query key (&query_key) representing a set of Entrez UIDs on the Entrez history server; Retrieval start (&retstart), the first record of the set to retrieve; Retrieval maximum (&retmax), maximum number of records to retrieve

Output: Formatted data records as specified

For More Information

Please see EFetch In-Depth for a full description of EFetch.

Finding Related Data Through Entrez Links

Basic Linking

Batch mode – finds only one set of linked UIDs

`elink.fcgi?dbfrom=<source_db>&db=<destination_db>&id=<uid_list>`

Input: List of UIDs (&id); Source Entrez database (&dbfrom); Destination Entrez database (&db)

Output: XML containing linked UIDs from source and destination databases

Example: Find one set of Gene IDs linked to nucleotide GIs 34577062 and 24475906

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=nucleotide&db=gene&id=34577062,24475906>

‘By Id’ mode – finds one set of linked UIDs for each input UID

`elink.fcgi?dbfrom=<source_db>&db=<destination_db>&id=<uid1>&id=<uid2>&id=<uid3>...`

Example: Find separate sets of Gene IDs linked to nucleotide GIs 34577062 and 24475906

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=nucleotide&db=gene&id=34577062&id=24475906>

Note: &db may be a comma-delimited list of databases, so that elink returns multiple sets of linked UIDs in a single call

Finding Links to Data from a Previous Search

`esearch.fcgi?db=<source_db>&term=<query>&usehistory=y`

esearch produces WebEnv value (\$web1) and QueryKey value (\$key1)

`elink.fcgi?dbfrom=<source_db>&db=<destination_db>&query_key=$key1&WebEnv=$web1&cmd=neighbor_history`

Input: Source Entrez database (&dbfrom); Destination Entrez database (&db); Web environment (&WebEnv) and query key (&query_key) representing the set of source UIDs on the Entrez history server; Command mode (&cmd)

Output: XML containing Web environments and query keys for each set of linked UIDs

Note: To achieve ‘By Id’ mode, one must send each input UID as a separate &id parameter in the URL. Sending a WebEnv/query_key set always produces Batch mode behavior (one set of linked UIDs).

Finding Computational Neighbors Limited by an Entrez Search

`elink.fcgi?dbfrom=<source_db>&db=<source_db>&id=<uid_list>&term=<query>&cmd=neighbor_history`

Input: Source Entrez database (&dbfrom); Destination Entrez database (&db); List of UIDs (&id); Entrez query (&term); Command mode (&cmd)

Output: XML containing Web environments and query keys for each set of linked UIDs

Example: Find protein UIDs that are rat Reference Sequences and that are sequence similar to GI 15718680

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/eflink.fcgi?dbfrom=protein&db=protein&id=15718680&term=rat\[orgn\]+AND+srcdb+refseq\[prop\]&cmd=neighbor_history](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/eflink.fcgi?dbfrom=protein&db=protein&id=15718680&term=rat[orgn]+AND+srcdb+refseq[prop]&cmd=neighbor_history)

For More Information

Please see ELink In-Depth for a full description of ELink.

Getting Database Statistics and Search Fields

`einfo.fcgi?db=<database>`

Input: Entrez database (&db)

Output: XML containing database statistics

Note: If no database parameter is supplied, einfo will return a list of all valid Entrez databases.

Example: Find database statistics for Entrez Protein.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/einfo.fcgi?db=protein>

For More Information

Please see EInfo In-Depth for a full description of EInfo.

Sample EInfo Output

```
<?xml version="1.0"?>
<!DOCTYPE eInfoResult PUBLIC "-//NLM//DTD eInfoResult, 11 May 2002//EN"
"http://www.ncbi.nlm.nih.gov/entrez/query/DTD/eInfo_020511.dtd">
<eInfoResult>
  <DbInfo>
    <DbName>protein</DbName>
    <MenuName>Protein</MenuName>
    <Description>Protein sequence record</Description>
    <Count>26715092</Count>
    <LastUpdate>2009/05/12 04:39</LastUpdate>
    <FieldList>
      <Field>
        <Name>ALL</Name>
        <FullName>All Fields</FullName>
        <Description>All terms from all searchable fields</Description>
        <TermCount>133639432</TermCount>
        <IsDate>N</IsDate>
        <IsNumerical>N</IsNumerical>
        <SingleToken>N</SingleToken>
        <Hierarchy>N</Hierarchy>
        <IsHidden>N</IsHidden>
      </Field>
      ...
      <Field>
        <Name>PORG</Name>
        <FullName>Primary Organism</FullName>
        <Description>Scientific and common names
of primary organism, and all higher levels of taxonomy</Description>
        <TermCount>673555</TermCount>
        <IsDate>N</IsDate>
```

```

<IsNumerical>N</IsNumerical>
<SingleToken>Y</SingleToken>
<Hierarchy>Y</Hierarchy>
<IsHidden>N</IsHidden>
</Field>
</FieldList>
<LinkList>
<Link>
<Name>protein_biosystems</Name>
<Menu>BioSystem Links</Menu>
<Description>BioSystems</Description>
<DbTo>biosystems</DbTo>
</Link>
...
<Link>
<Name>protein_unigene</Name>
<Menu>UniGene Links</Menu>
<Description>Related UniGene records</Description>
<DbTo>unigene</DbTo>
</Link>
</LinkList>
</DbInfo>
</eInfoResult>

```

Performing a Global Entrez Search

egquery.fcgi?term=<query>

Input: Entrez text query (&term)

Output: XML containing the number of hits in each database.

Example: Determine the number of records for mouse in Entrez.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/egquery.fcgi?term=mouse\[orgn\]](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/egquery.fcgi?term=mouse[orgn])

For More Information

Please see EGQuery In-Depth for a full description of EGQuery.

Sample EGQuery Output

```

<?xml version="1.0"?>
<!DOCTYPE Result PUBLIC "-//NLM//DTD eSearchResult, January 2004//EN"
  "https://www.ncbi.nlm.nih.gov/entrez/query/DTD/egquery.dtd">
<!--
      $Id: egquery_template.xml 106311 2007-06-26 14:46:31Z osipov $
-->
<!-- ===== -->
<Result>
  <Term>mouse[orgn]</Term>
  <eQueryResult>
    <ResultItem>
      <DbName>pubmed</DbName>
      <MenuName>PubMed</MenuName>
      <Count>0</Count>
      <Status>Term or Database is not found</Status>
    </ResultItem>
    <ResultItem>

```

```

        <DbName>pmc</DbName>
        <MenuName>PMC</MenuName>
        <Count>3823</Count>
        <Status>Ok</Status>
    </ResultItem>
...
    <ResultItem>
        <DbName>nuccore</DbName>
        <MenuName>Nucleotide</MenuName>
        <Count>1739903</Count>
        <Status>Ok</Status>
    </ResultItem>
    <ResultItem>
        <DbName>nucgss</DbName>
        <MenuName>GSS</MenuName>
        <Count>2264567</Count>
        <Status>Ok</Status>
    </ResultItem>
    <ResultItem>
        <DbName>nucest</DbName>
        <MenuName>EST</MenuName>
        <Count>4852140</Count>
        <Status>Ok</Status>
    </ResultItem>
    <ResultItem>
        <DbName>protein</DbName>
        <MenuName>Protein</MenuName>
        <Count>255212</Count>
        <Status>Ok</Status>
    </ResultItem>
...
    <ResultItem>
        <DbName>proteinclusters</DbName>
        <MenuName>Protein Clusters</MenuName>
        <Count>13</Count>
        <Status>Ok</Status>
    </ResultItem>
</eGQueryResult>
</Result>

```

Retrieving Spelling Suggestions

`espell.fcgi?term=<query>&db=<database>`

Input: Entrez text query (&term); Entrez database (&db)

Output: XML containing the original query and spelling suggestions.

Example: Find spelling suggestions for the PubMed Central query 'fiberblast cell growth'.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/espell.fcgi?term=fiberblast+cell+growth&db=pmc>

For More Information

Please see ESpell In-Depth for a full description of EGQuery.

Sample ESpell Output

```
<?xml version="1.0"?>
<!DOCTYPE eSpellResult PUBLIC "-//NLM//DTD eSpellResult, 23 November
2004//EN" "https://www.ncbi.nlm.nih.gov/entrez/query/DTD/eSpell.dtd">
<eSpellResult>
<Database>pmc</Database>
<Query>fiberblast cell grwth</Query>
<CorrectedQuery>fibroblast cell growth</CorrectedQuery>
<SpelledQuery>
  <Replaced>fibroblast</Replaced>
  <Original> cell </Original>
  <Replaced>growth</Replaced>
</SpelledQuery>
<ERROR/>
</eSpellResult>
```

Demonstration Programs

EBot

[EBot](#) is an interactive web tool that first allows users to construct an arbitrary E-utility analysis pipeline and then generates a Perl script to execute the pipeline. The Perl script can be downloaded and executed on any computer with a Perl installation. For more details, see the [EBot](#) page linked above.

Sample Perl Scripts

The two sample Perl scripts below demonstrate basic E-utility functions. Both scripts should be copied and saved as plain text files and can be executed on any computer with a Perl installation.

ESearch-EFetch demonstrates basic search and retrieval functions.

```
#!/usr/local/bin/perl -w
# =====
#
#                                     PUBLIC DOMAIN NOTICE
#                               National Center for Biotechnology Information
#
# This software/database is a "United States Government Work" under the
# terms of the United States Copyright Act. It was written as part of
# the author's official duties as a United States Government employee and
# thus cannot be copyrighted. This software/database is freely available
# to the public for use. The National Library of Medicine and the U.S.
# Government have not placed any restriction on its use or reproduction.
#
# Although all reasonable efforts have been taken to ensure the accuracy
# and reliability of the software and data, the NLM and the U.S.
# Government do not and cannot warrant the performance or results that
# may be obtained by using this software or data. The NLM and the U.S.
# Government disclaim all warranties, express or implied, including
# warranties of performance, merchantability or fitness for any particular
# purpose.
#
# Please cite the author in any work or product based on this material.
# =====
#
```

```

# Author:  Oleg Khovayko
#
# File Description: eSearch/eFetch calling example
#
# -----
# Subroutine to prompt user for variables in the next section

sub ask_user {
    print "$_[0] [$_[1]]: ";
    my $rc = <>;
    chomp $rc;
    if($rc eq "") { $rc = $_[1]; }
    return $rc;
}

# -----
# Define library for the 'get' function used in the next section.
# $utils contains route for the utilities.
# $db, $query, and $report may be supplied by the user when prompted;
# if not answered, default values, will be assigned as shown below.

use LWP::Simple;

my $utils = "https://www.ncbi.nlm.nih.gov/entrez/eutils";

my $db      = ask_user("Database", "Pubmed");
my $query   = ask_user("Query",    "zanzibar");
my $report  = ask_user("Report",   "abstract");

# -----
# $esearch contains the PATH & parameters for the ESearch call
# $esearch_result contains the result of the ESearch call
# the results are displayed and parsed into variables
# $Count, $QueryKey, and $WebEnv for later use and then displayed.

my $esearch = "$utils/esearch.fcgi?" .
    "db=$db&retmax=1&usehistory=y&term=";

my $esearch_result = get($esearch . $query);

print "\nESEARCH RESULT: $esearch_result\n";

$esearch_result =~
    m|<Count>(\d+)</Count>.*<QueryKey>(\d+)</QueryKey>.*<WebEnv>(\S+)</WebEnv>|s;

my $Count      = $1;
my $QueryKey   = $2;
my $WebEnv     = $3;

print "Count = $Count; QueryKey = $QueryKey; WebEnv = $WebEnv\n";

# -----
# this area defines a loop which will display $retmax citation results from
# efetch each time the the Enter Key is pressed, after a prompt.

my $retstart;
my $retmax=3;

for($retstart = 0; $retstart < $Count; $retstart += $retmax) {

```

```

my $efetch = "$utils/efetch.fcgi?" .
    "rettype=$report&retmode=text&retstart=$retstart&retmax=$retmax&" .
    "db=$db&query_key=$QueryKey&WebEnv=$WebEnv";

print "\nEF_QUERY=$efetch\n";

my $efetch_result = get($efetch);

print "-----\nEFETCH RESULT(" .
    ($retstart + 1) . ".." . ($retstart + $retmax) . "): " .
    "[$efetch_result]\n-----PRESS ENTER!!!-----\n";
<>;
}

```

EPost-ESummary demonstrates basic uploading and document summary retrieval.

```

#!/usr/local/bin/perl -w
# =====
#
#                               PUBLIC DOMAIN NOTICE
#                               National Center for Biotechnology Information
#
# This software/database is a "United States Government Work" under the
# terms of the United States Copyright Act. It was written as part of
# the author's official duties as a United States Government employee and
# thus cannot be copyrighted. This software/database is freely available
# to the public for use. The National Library of Medicine and the U.S.
# Government have not placed any restriction on its use or reproduction.
#
# Although all reasonable efforts have been taken to ensure the accuracy
# and reliability of the software and data, the NLM and the U.S.
# Government do not and cannot warrant the performance or results that
# may be obtained by using this software or data. The NLM and the U.S.
# Government disclaim all warranties, express or implied, including
# warranties of performance, merchantability or fitness for any particular
# purpose.
#
# Please cite the author in any work or product based on this material.
#
# =====
#
# Author:  Oleg Khovayko
#
# File Description: ePost/eSummary calling example
#
# -----
my $utils_root = "https://www.ncbi.nlm.nih.gov/entrez/efetch";
my $ePost_url  = "$utils_root/epost.fcgi";
my $eSummary_url = "$utils_root/esummary.fcgi";

my $db_name = "PubMed";

# -----
use strict;

use LWP::UserAgent;
use LWP::Simple;
use HTTP::Request;

```

```

use HTTP::Headers;
use CGI;

# -----
# Read input file into variable $file
# File name - first argument $ARGV[0]

undef $/; #for load whole file

open IF, $ARGV[0] || die "Can't open for read: $!\n";
my $file = <IF>;
close IF;
print "Loaded file: [$file]\n";

# Prepare file - substitute all separators to comma

$file =~ s/\s+/,/gs;
print "Prepared file: [$file]\n";

#Create CGI param line

my $form_data = "db=$db_name&id=$file";

# -----
# Create HTTP request

my $headers = new HTTP::Headers(
    Accept      => "text/html, text/plain",
    Content_Type => "application/x-www-form-urlencoded"
);

my $request = new HTTP::Request("POST", $ePost_url, $headers );

$request->content($form_data);

# Create the user agent object

my $ua = new LWP::UserAgent;
$ua->agent("ePost/example");

# -----
# send file to ePost by HTTP

my $response = $ua->request($request);

# -----

print "Response status message: [" . $response->message . "]\n";
print "Response content: [" . $response->content . "]\n";

# -----
# Parse response->content and extract QueryKey & WebEnv
$response->content =~
    m|<QueryKey>(\d+)</QueryKey>.*<WebEnv>(\S+)</WebEnv>|s;

my $QueryKey = $1;
my $WebEnv   = $2;

print "\nEXTRACTED:\nQueryKey = $QueryKey;\nWebEnv = $WebEnv\n\n";

```

```
# -----  
# Retrieve DocSum from eSummary by simple::get method and print it  
#  
print "eSummary result: [" .  
    get("$eSummary_url?db=$db_name&query_key=$QueryKey&WebEnv=$WebEnv") .  
    "]\n";
```

For More Information

Announcement Mailing List

NCBI posts general announcements regarding the E-utilities to the [utilities-announce announcement mailing list](#). This mailing list is an announcement list only; individual subscribers may **not** send mail to the list. Also, the list of subscribers is private and is not shared or used in any other way except for providing announcements to list members. The list receives about one posting per month. Please subscribe at the above link.

Getting Help

Please refer to the [PubMed](#) and [Entrez](#) help documents for more information about search queries, database indexing, field limitations and database content.

Suggestions, comments, and questions specifically relating to the EUtility programs may be sent to utilities@ncbi.nlm.nih.gov.

A General Introduction to the E-utilities

Eric Sayers, PhD¹

Created: May 26, 2009; Updated: November 17, 2022.

Introduction

The Entrez Programming Utilities (E-utilities) are a set of nine server-side programs that provide a stable interface into the Entrez query and database system at the National Center for Biotechnology Information (NCBI). The E-utilities use a fixed URL syntax that translates a standard set of input parameters into the values necessary for various NCBI software components to search for and retrieve the requested data. The E-utilities are therefore the structured interface to the Entrez system, which currently includes 38 databases covering a variety of biomedical data, including nucleotide and protein sequences, gene records, three-dimensional molecular structures, and the biomedical literature.

To access these data, a piece of software first posts an E-utility URL to NCBI, then retrieves the results of this posting, after which it processes the data as required. The software can thus use any computer language that can send a URL to the E-utilities server and interpret the XML response; examples of such languages are Perl, Python, Java, and C++. Combining E-utilities components to form customized data pipelines within these applications is a powerful approach to data manipulation.

This chapter first describes the general function and use of the eight E-utilities, followed by basic usage guidelines and requirements, and concludes with a discussion of how the E-utilities function within the Entrez system.

Usage Guidelines and Requirements

Use the E-utility URL

All E-utility requests should be made to URLs beginning with the following string:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/>

These URLs direct requests to servers that are used only by the E-utilities and that are optimized to give users the best performance.

Frequency, Timing and Registration of E-utility URL Requests

In order not to overload the E-utility servers, NCBI recommends that users post no more than three URL requests per second and limit large jobs to either weekends or between 9:00 PM and 5:00 AM Eastern time during weekdays. Failure to comply with this policy may result in an IP address being blocked from accessing NCBI. If NCBI blocks an IP address, service will not be restored unless the developers of the software accessing the E-utilities register values of the **tool** and **email** parameters with NCBI. The value of **tool** should be a string with no internal spaces that uniquely identifies the software producing the request. The value of **email** should be a complete and valid e-mail address of the software developer and not that of a third-party end user. The value of **email** will be used only to contact developers if NCBI observes requests that violate our policies, and we will attempt such contact prior to blocking access. In addition, developers may request that the value of **email** be added to the E-utility mailing list that provides announcements of software updates, known bugs and other

Author Affiliation: 1 NCBI; Email: sayers@ncbi.nlm.nih.gov.

¹ Corresponding author.

policy changes affecting the E-utilities. To register **tool** and **email** values, simply send an e-mail to entrez@ncbi.nlm.nih.gov including the desired values along with the name of either a developer or the organization creating the software. Once NCBI establishes communication with a developer, receives values for **tool** and **email** and validates the e-mail address in **email**, the block will be lifted. Once **tool** and **email** values are registered, all subsequent E-utility requests from that software package should contain both values. Please be aware that merely providing values for **tool** and **email** in requests is not sufficient to comply with this policy; these values must be registered with NCBI. Requests from any IP that lack registered values for **tool** and **email** and that violate the above usage policies may be blocked. Software developers may register values of **tool** and **email** at any time, and are encouraged to do so.

API Keys

Since December 1, 2018, NCBI has provided API keys that offer enhanced levels of supported access to the E-utilities. Without an API key, any site (IP address) posting more than 3 requests per second to the E-utilities will receive an error message. By including an API key, a site can post up to 10 requests per second by default. Higher rates are available by request (entrez@ncbi.nlm.nih.gov). Users can obtain an API key now from the Settings page of their NCBI account (to create an account, visit <http://www.ncbi.nlm.nih.gov/account/>). After creating the key, users should include it in each E-utility request by assigning it to the *api_key* parameter.

Example request including an API key:

```
esummary.fcgi?db=pubmed&id=123456&api_key=ABCDE12345
```

Example error message if rates are exceeded:

```
{"error":"API rate limit exceeded","count":"11"}
```

Only one API key is allowed per NCBI account; however, a user may request a new key at any time. Such a request will invalidate any existing API key associated with that NCBI account.

Minimizing the Number of Requests

If a task requires searching for and/or downloading a large number of records, it is much more efficient to use the Entrez History to upload and/or retrieve these records in batches rather than using separate requests for each record. Please refer to Application 3 in Chapter 3 for an example. Many thousands of IDs can be uploaded using a single EPost request, and several hundred records can be downloaded using one EFetch request.

Disclaimer and Copyright Issues

If you use the E-utilities within software, NCBI's Disclaimer and Copyright notice (<https://www.ncbi.nlm.nih.gov/About/disclaimer.html>) must be evident to users of your product. Please note that abstracts in PubMed may incorporate material that may be protected by U.S. and foreign copyright laws. All persons reproducing, redistributing, or making commercial use of this information are expected to adhere to the terms and conditions asserted by the copyright holder. Transmission or reproduction of protected items beyond that allowed by fair use (PDF) as defined in the copyright laws requires the written permission of the copyright owners. NLM provides no legal advice concerning distribution of copyrighted materials. Please consult your legal counsel. If you wish to do a large data mining project on PubMed data, you can download a local copy of the database at https://www.nlm.nih.gov/databases/download/pubmed_medline.html.

Handling Special Characters Within URLs

When constructing URLs for the E-utilities, please use lowercase characters for all parameters except &WebEnv. There is no required order for the URL parameters in an E-utility URL, and null values or inappropriate parameters are generally ignored. Avoid placing spaces in the URLs, particularly in queries. If a space is required, use a plus sign (+) instead of a space:

Incorrect: &id=352, 25125, 234

Correct: &id=352,25125,234

Incorrect: &term=biomol mrna[properties] AND mouse[organism]

Correct: &term=biomol+mrna[properties]+AND+mouse[organism]

Other special characters, such as quotation marks (") or the # symbol used in referring to a query key on the History server, should be represented by their URL encodings (%22 for "; %23 for #).

Incorrect: &term=#2+AND+"gene in genomic"[properties]

Correct: &term=%232+AND+%22gene+in+genomic%22[properties]

The Nine E-utilities in Brief

EInfo (database statistics)

eutils.ncbi.nlm.nih.gov/entrez/eutils/einfo.fcgi

Provides the number of records indexed in each field of a given database, the date of the last update of the database, and the available links from the database to other Entrez databases.

ESearch (text searches)

eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi

Responds to a text query with the list of matching UIDs in a given database (for later use in ESummary, EFetch or ELink), along with the term translations of the query.

EPost (UID uploads)

eutils.ncbi.nlm.nih.gov/entrez/eutils/epost.fcgi

Accepts a list of UIDs from a given database, stores the set on the History Server, and responds with a query key and web environment for the uploaded dataset.

ESummary (document summary downloads)

eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi

Responds to a list of UIDs from a given database with the corresponding document summaries.

EFetch (data record downloads)

eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi

Responds to a list of UIDs in a given database with the corresponding data records in a specified format.

ELink (Entrez links)

eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi

Responds to a list of UIDs in a given database with either a list of related UIDs (and relevancy scores) in the same database or a list of linked UIDs in another Entrez database; checks for the existence of a specified link from a list of one or more UIDs; creates a hyperlink to the primary LinkOut provider for a specific UID and database, or lists LinkOut URLs and attributes for multiple UIDs.

EGQuery (global query)

eutils.ncbi.nlm.nih.gov/entrez/eutils/egquery.fcgi

Responds to a text query with the number of records matching the query in each Entrez database.

ESpell (spelling suggestions)

eutils.ncbi.nlm.nih.gov/entrez/eutils/espell.fcgi

Retrieves spelling suggestions for a text query in a given database.

ECitMatch (batch citation searching in PubMed)

eutils.ncbi.nlm.nih.gov/entrez/eutils/ecitmatch.cgi

Retrieves PubMed IDs (PMIDs) corresponding to a set of input citation strings.

Understanding the E-utilities Within Entrez

The E-utilities Access Entrez Databases

The E-utilities access the core search and retrieval engine of the Entrez system and, therefore, are only capable of retrieving data that are already in Entrez. Although the majority of data at NCBI are in Entrez, there are several datasets that exist outside of the Entrez system. Before beginning a project with the E-utilities, check that the desired data can be found within an Entrez database.

The Entrez System Identifies Database Records Using UUIDs

Each Entrez database refers to the data records within it by an integer ID called a UUID (unique identifier). Examples of UUIDs are GI numbers for Nucleotide and Protein, PMIDs for PubMed, or MMDB-IDs for Structure. The E-utilities use UUIDs for both data input and output, and thus it is often critical, especially for advanced data pipelines, to know how to find the UUIDs associated with the desired data before beginning a project with the E-utilities.

See Table 1 for a complete list of UUIDs in Entrez.

Table 1 – Entrez Unique Identifiers (UUIDs) for selected databases

Entrez Database	UUID common name	E-utility Database Name
BioProject	BioProject ID	bioproject
BioSample	BioSample ID	biosample
Books	Book ID	books
Conserved Domains	PSSM-ID	cdd
dbGaP	dbGaP ID	gap
dbVar	dbVar ID	dbvar
Gene	Gene ID	gene
Genome	Genome ID	genome
GEO Datasets	GDS ID	gds
GEO Profiles	GEO ID	geoprofiles
HomoloGene	HomoloGene ID	homologene

Table 1 continued from previous page.

Entrez Database	UID common name	E-utility Database Name
MeSH	MeSH ID	mesh
NCBI C++ Toolkit	Toolkit ID	toolkit
NLM Catalog	NLM Catalog ID	nlmcatalog
Nucleotide	GI number	nucore
PopSet	PopSet ID	popset
Probe	Probe ID	probe
Protein	GI number	protein
Protein Clusters	Protein Cluster ID	proteinclusters
PubChem BioAssay	AID	pcassay
PubChem Compound	CID	pccompound
PubChem Substance	SID	pcsubstance
PubMed	PMID	pubmed
PubMed Central	PMCID	pmc
SNP	rs number	snp
SRA	SRA ID	sra
Structure	MMDB-ID	structure
Taxonomy	TaxID	taxonomy

Accessing Sequence Records Using Accession.Version Identifiers

NCBI now uses the accession.version identifier rather than the GI number (UID) as the primary identifier for nucleotide and protein sequence records (records in the nucore, nucest, nucgss, popset, and protein databases). Even so, the E-utilities continue to provide access to these records using either GI numbers or accession.version identifiers. Those E-utilities that accept UIDs as input will also accept accession.version identifiers (for the sequence databases listed above). Those E-utilities that output UIDs can output accession.version identifiers instead by setting the &idtype parameter to “acc”. Finally, EFetch can retrieve *any* sequence record by its accession.version identifier, including sequences that do not have GI numbers. Please see Chapter 4 for more details about how each E-utility handles accession.version identifiers.

The Entrez Core Engine: EQuery, ESearch, and ESummary

The core of Entrez is an engine that performs two basic tasks for any Entrez database: 1) assemble a list of UIDs that match a text query, and 2) retrieve a brief summary record called a Document Summary (DocSum) for each UID. These two basic tasks of the Entrez engine are performed by ESearch and ESummary. ESearch returns a list of UIDs that match a text query in a given Entrez database, and ESummary returns DocSums that match a list of input UIDs. A text search in web Entrez is equivalent to ESearch-ESummary. EQuery is a global version of ESearch that searches all Entrez databases simultaneously. Because these three E-utilities perform the two core Entrez functions, they function for all Entrez databases.

```
egquery.fcgi?term=query
esearch.fcgi?db=database&term=query
esummary.fcgi?db=database&id=uid1,uid2,uid3,...
```

Syntax and Initial Parsing of Entrez Queries

Text search strings entered into the Entrez system are converted into Entrez queries with the following format:

`term1[field1] Op term2[field2] Op term3[field3] Op ...`

where the terms are search terms, each limited to a particular Entrez field in square brackets, combined using one of three Boolean operators: Op = AND, OR, or NOT. These Boolean operators must be typed in all capital letters.

Example: `human[organism] AND topoisomerase[protein name]`

Entrez initially splits the query into a series of items that were originally separated by spaces in the query; therefore it is critical that spaces separate each term and Boolean operator. If the query consists *only* of a list of UID numbers (unique identifiers) or accession numbers, the Entrez system simply returns the corresponding records and no further parsing is performed. If the query contains any Boolean operators (AND, OR, or NOT), the query is split into the terms separated by these operators, and then each term is parsed independently. The results of these searches are then combined according to the Boolean operators.

A full account of how to search Entrez can be found in the [Entrez Help Document](#). Additional information is available from [Entrez Help](#).

Entrez Databases: EInfo, EFetch, and ELink

The NCBI Entrez system currently contains 38 databases. EInfo provides detailed information about each database, including lists of the indexing fields in the database and the available links to other Entrez databases.

`einfo.fcgi?db=database`

Each Entrez database includes two primary enhancements to the raw data records: 1) software for producing a variety of display formats appropriate to the given database, and 2) links to records in other Entrez databases manifested as lists of associated UIDs. The display format function is performed by EFetch, which generates formatted output for a list of input UIDs. For example, EFetch can produce abstracts from Entrez PubMed or FASTA format from Entrez Protein. EFetch does not yet support all Entrez databases; please see the EFetch documentation for details.

`efetch.fcgi?db=database&id=uid1,uid2,uid3&rettype=report_type&retmode=data_mode`

The linking function is performed by ELink, which generates a list of UIDs in a specified Entrez database that are linked to a set of input UIDs in either the same or another database. For example, ELink can find Entrez SNP records linked to records in Entrez Nucleotide, or Entrez Domain records linked to records in Entrez Protein.

`elink.fcgi?dbfrom=initial_databasedb=target_database&id=uid1,uid2,uid3`

Using the Entrez History Server

A powerful feature of the Entrez system is that it can store retrieved sets of UIDs temporarily on the servers so that they can be subsequently combined or provided as input for other E-utility calls. The Entrez History server provides this service and is accessed on the Web using either the Preview/Index or History tabs on Entrez search pages. Each of the E-utilities can also use the History server, which assigns each set of UIDs an integer label called a query key (&query_key) and an encoded cookie string called a Web environment (&WebEnv). EPost allows any list of UIDs to be uploaded to the History Server and returns the query key and Web environment. ESearch can also post its output set of UIDs to the History Server, but only if the &usehistory parameter is set to "y". ELink also can post its output to the History server if &cmd is set to "neighbor_history". The resulting query

key and Web environment from either EPost or ESearch can then be used in place of a UID list in ESummary, EFetch, and ELink.

In Entrez, a set of UIDs is represented on the History by three parameters:

`&db = database; &query_key = query key; &WebEnv = web environment`

Upload steps that generate a web environment and query key

`esearch.fcgi?db=database&term=query&usehistory=y`

`epost.fcgi?db=database&id=uid1,uid2,uid3,...`

`elink.fcgi?dbfrom=source_db&db=destination_db&cmd=neighbor_history&id=uid1,uid2,...`

Download steps that use a web environment and query key

`esummary.fcgi?db=database&WebEnv=webenv&query_key=key`

`efetch.fcgi?db=database&WebEnv=webenv&query_key=key&rettype=report_type&retmode=data_mode`

Link step that uses a web environment and query key

`elink.fcgi?dbfrom=initial_databasedb=target_database&WebEnv=webenv&query_key=key`

Search step that uses a web environment and a query key in the `&term` parameter (preceded by #, encoded as %23)

`esearch.fcgi?db=database&term=%23key+AND+query&WebEnv=webenv&usehistory=y`

Generating Multiple Data Sets on the History Server

Each web environment on the History Server can be associated with any number of query keys. This allows different data sets to be combined with the Boolean operators AND, OR, and NOT, or with another Entrez query. It is important to remember that for two data sets (query keys) to be combined, they must be associated with the same web environment. By default, successive E-utility calls produce query keys that are *not* associated with the same web environment, and so to overcome this, each E-utility call after the initial call must set the `&WebEnv` parameter to the value of the pre-existing web environment.

Default behavior: These two URLs...

URL 1: `epost.fcgi?db=database&id=uid1,uid2,uid3`

URL 2: `esearch.fcgi?db=database&term=query&usehistory=y`

will produce two History sets associated with different web environments:

URL	WebEnv	query_key	UIDs
1	web1	1	uid1,uid2,uid3
2	web2	1	uids matching query

Desired behavior: These two URLs...

URL 1: `epost.fcgi?db=database&id=uid1,uid2,uid3`

(extract *web1* from the output of URL 1)

URL 2: `esearch.fcgi?db=database&term=query&usehistory=y&WebEnv=web1`

will produce two sets associated with the same (new) web environment:

URL	WebEnv	query_key	UIDs
1	web2	1	uid1,uid2,uid3
2	web2	2	uids matching query

Combining E-utility Calls to Create Entrez Applications

The E-utilities are useful when used by themselves in single URLs; however, their full potential is realized when successive E-utility URLs are combined to create a data pipeline. When used within such pipelines, the Entrez History server simplifies complex retrieval tasks by allowing easy data transfer between successive E-utility calls. Listed below are several examples of pipelines produced by combining E-utilities, with the arrows representing the passing of db, WebEnv and query_key values from one E-utility to another. These and related pipelines are discussed in detail in Chapter 3.

Basic Pipelines

Retrieving data records matching an Entrez query

ESearch → ESummary

ESearch → EFetch

Retrieving data records matching a list of UIDs

EPost → ESummary

EPost → EFetch

Finding UIDs linked to a set of records

ESearch → ELink

EPost → ELink

Limiting a set of records with an Entrez query

EPost → ESearch

ELink → ESearch

Advanced Pipelines

Retrieving data records in database B linked to records in database A matching an Entrez query

ESearch → ELink → ESummary

ESearch → ELink → EFetch

Retrieving data records from a subset of an ID list defined by an Entrez query

EPost → ESearch → ESummary

EPost → ESearch → EFetch

Retrieving a set of data records, defined by an Entrez query, in database B from a larger set of records linked to a list of UIDs in database A

EPost → ELink → ESearch → ESummary

EPost → ELink → ESearch → EFetch

Demonstration Programs

Please see Chapter 1 for sample Perl scripts.

For More Information

Please see [Chapter 1](#) for getting additional information about the E-utilities.

Sample Applications of the E-utilities

Eric Sayers, PhD^{✉1}

Created: April 24, 2009; Updated: November 1, 2017.

Introduction

This chapter presents several examples of how the E-utilities can be used to build useful applications. These examples use Perl to create the E-utility pipelines, and assume that the LWP::Simple module is installed. This module includes the *get* function that supports HTTP GET requests. One example (Application 4) uses an HTTP POST request, and requires the LWP::UserAgent module. In Perl, scalar variable names are preceded by a "\$" symbol, and array names are preceded by a "@". In several instances, results will be stored in such variables for use in subsequent E-utility calls. The code examples here are working programs that can be copied to a text editor and executed directly. Equivalent HTTP requests can be constructed in many modern programming languages; all that is required is the ability to create and post an HTTP request.

Basic Pipelines

All E-utility applications consist of a series of calls that we will refer to as a pipeline. The simplest E-utility pipelines consist of two calls, and any arbitrary pipeline can be assembled from these basic building blocks. Many of these pipelines conclude with either ESummary (to retrieve DocSums) or EFetch (to retrieve full records). The comments indicate those portions of the code that are required for either call.

ESearch – ESummary/EFetch

Input: Entrez text query

ESummary Output: XML Document Summaries

EFetch Output: Formatted data records (e.g. abstracts, FASTA)

```
use LWP::Simple;

# Download PubMed records that are indexed in MeSH for both asthma and
# leukotrienes and were also published in 2009.

$db = 'pubmed';
$query = 'asthma[mesh]+AND+leukotrienes[mesh]+AND+2009[pdat]';

#assemble the esearch URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "esearch.fcgi?db=$db&term=$query&usehistory=y";

#post the esearch URL
$output = get($url);

#parse WebEnv and QueryKey
$web = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);
```

Author Affiliation: 1 NCBI; Email: sayers@ncbi.nlm.nih.gov.

[✉] Corresponding author.

```

#### include this code for ESearch-ESummary
#assemble the esummary URL
$url = $base . "esummary.fcgi?db=$db&query_key=$key&WebEnv=$web";

#post the esummary URL
$docsums = get($url);
print "$docsums";

#### include this code for ESearch-EFetch
#assemble the efetch URL
$url = $base . "efetch.fcgi?db=$db&query_key=$key&WebEnv=$web";
$url .= "&rettype=abstract&retmode=text";

#post the efetch URL
$data = get($url);
print "$data";

```

EPost – ESummary/EFetch

Input: List of Entrez UUIDs (integer identifiers, e.g. PMID, GI, Gene ID)

ESummary Output: XML Document Summaries

EFetch Output: Formatted data records (e.g. abstracts, FASTA)

```

use LWP::Simple;

# Download protein records corresponding to a list of GI numbers.

$db = 'protein';
$id_list = '194680922,50978626,28558982,9507199,6678417';

#assemble the epost URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "epost.fcgi?db=$db&id=$id_list";

#post the epost URL
$output = get($url);

#parse WebEnv and QueryKey
$web = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);

#### include this code for EPost-ESummary
#assemble the esummary URL
$url = $base . "esummary.fcgi?db=$db&query_key=$key&WebEnv=$web";

#post the esummary URL
$docsums = get($url);
print "$docsums";

#### include this code for EPost-EFetch
#assemble the efetch URL
$url = $base . "efetch.fcgi?db=$db&query_key=$key&WebEnv=$web";
$url .= "&rettype=fasta&retmode=text";

#post the efetch URL
$data = get($url);
print "$data";

```

Note: To post a large number (more than a few hundred) UIDs in a single URL, please use the HTTP POST method for the EPost call (see Application 4).

ELink – ESummary/Efetch

Input: List of Entrez UIDs in database A (integer identifiers, e.g. PMID, GI, Gene ID)

ESummary Output: Linked XML Document Summaries from database B

EFetch Output: Formatted data records (e.g. abstracts, FASTA) from database B

use LWP::Simple;

Download gene records linked to a set of proteins corresponding to a list
of GI numbers.

```
$db1 = 'protein'; # &dbfrom
$db2 = 'gene';    # &db
$linkname = 'protein_gene'; # desired link &linkname
#input UIDs in $db1 (protein GIs)
$id_list = '194680922,50978626,28558982,9507199,6678417';

#assemble the elink URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "elink.fcgi?dbfrom=$db1&db=$db2&id=$id_list";
$url .= "&linkname=$linkname&cmd=neighbor_history";

#post the elink URL
$output = get($url);

#parse WebEnv and QueryKey
$web = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);

### include this code for ELink-ESummary
#assemble the esummary URL
$url = $base . "esummary.fcgi?db=$db&query_key=$key&WebEnv=$web";

#post the esummary URL
$docsums = get($url);
print "$docsums";

### include this code for ELink-EFetch
#assemble the efetch URL
$url = $base . "efetch.fcgi?db=$db2&query_key=$key&WebEnv=$web";
$url .= "&rettype=xml&retmode=xml";

#post the efetch URL
$data = get($url);
print "$data";
```

Notes: To submit a large number (more than a few hundred) UIDs to ELink in one URL, please use the HTTP POST method for the Elink call (see Application 4). The `&linkname` parameter is used to force ELink to return only one set of links (one `&query_key`) to simplify parsing. If more than one link is desired, the above code must be altered to parse the multiple `&query_key` values from the ELink XML output. This code uses ELink in "batch" mode, in that only one set of gene IDs is returned and the one-to-one correspondence between protein GIs and gene IDs is lost. To preserve this one-to-one correspondence, please see Application 4 below.

ESearch – ELink – ESummary/EFetch

Input: Entrez text query in database A

ESummary Output: Linked XML Document Summaries from database B

EFetch Output: Formatted data records (e.g. abstracts, FASTA) from database B

```
use LWP::Simple;
# Download protein FASTA records linked to abstracts published
# in 2009 that are indexed in MeSH for both asthma and
# leukotrienes.

$db1 = 'pubmed';
$db2 = 'protein';
$linkname = 'pubmed_protein';
$query = 'asthma[mesh]+AND+leukotrienes[mesh]+AND+2009[pdat]';

#assemble the esearch URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "esearch.fcgi?db=$db1&term=$query&usehistory=y";
#post the esearch URL
$output = get($url);

#parse WebEnv and QueryKey
$web1 = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key1 = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);

#assemble the elink URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "elink.fcgi?dbfrom=$db1&db=$db2";
$url .= "&query_key=$key1&WebEnv=$web1";
$url .= "&linkname=$linkname&cmd=neighbor_history";
print "$url\n";

#post the elink URL
$output = get($url);
print "$output\n";

#parse WebEnv and QueryKey
$web2 = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key2 = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);

### include this code for ESearch-ELink-ESummary
#assemble the esummary URL
$url = $base . "esummary.fcgi?db=$db2&query_key=$key2&WebEnv=$web2";
#post the esummary URL
$docsums = get($url);
print "$docsums";

### include this code for ESearch-ELink-EFetch
#assemble the efetch URL
$url = $base . "efetch.fcgi?db=$db2&query_key=$key2&WebEnv=$web2";
$url .= "&rettype=fasta&retmode=text";
#post the efetch URL
$data = get($url);
print "$data";
```

Notes: The `&linkname` parameter is used to force ELink to return only one set of links (one `&query_key`) to simplify parsing. If more than one link is desired, the above code must be altered to parse the multiple `&query_key` values from the ELink XML output. This code uses ELink in "batch" mode, in that only one set of PubMed IDs is returned and the one-to-one correspondence between PubMed IDs and their related PubMed IDs is lost. To preserve this one-to-one correspondence, please see Application 4 below.

EPost – ELink – ESummary/EFetch

Input: List of Entrez UIDs (integer identifiers, e.g. PMID, GI, Gene ID) in database A

ESummary Output: Linked XML Document Summaries from database B

EFetch Output: Formatted data records (e.g. abstracts, FASTA) from database B

```
use LWP::Simple;

# Downloads gene records linked to a set of proteins corresponding
# to a list of protein GI numbers.

$db1 = 'protein'; # &dbfrom
$db2 = 'gene';    # &db
$linkname = 'protein_gene';
#input UIDs in $db1 (protein GIs)
$id_list = '194680922,50978626,28558982,9507199,6678417';

#assemble the epost URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "epost.fcgi?db=$db1&id=$id_list";

#post the epost URL
$output = get($url);

#parse WebEnv and QueryKey
$web1 = $1 if ($output =~ /<WebEnv>(\S+)<\WebEnv>/);
$key1 = $1 if ($output =~ /<QueryKey>(\d+)<\QueryKey>/);

#assemble the elink URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "elink.fcgi?dbfrom=$db1&db=$db2&query_key=$key1";
$url .= "&WebEnv=$web1&linkname=$linkname&cmd=neighbor_history";

#post the elink URL
$output = get($url);

#parse WebEnv and QueryKey
$web2 = $1 if ($output =~ /<WebEnv>(\S+)<\WebEnv>/);
$key2 = $1 if ($output =~ /<QueryKey>(\d+)<\QueryKey>/);

### include this code for ESearch-ELink-ESummary
#assemble the esummary URL
$url = $base . "esummary.fcgi?db=$db2&query_key=$key2&WebEnv=$web2";

#post the esummary URL
$docsums = get($url);
print "$docsums";

### include this code for ESearch-ELink-EFetch
#assemble the efetch URL
```

```
$url = $base . "efetch.fcgi?db=$db2&query_key=$key2&WebEnv=$web2";
$url .= "&rettype=xml&retmode=xml";
```

```
#post the efetch URL
$data = get($url);
print "$data";
```

Notes: To post a large number (more than a few hundred) UIDs in a single URL, please use the HTTP POST method for the EPost call (see Application 4 below). The `&linkname` parameter is used to force ELink to return only one set of links (one `&query_key`) to simplify parsing. If more than one link is desired, the above code must be altered to parse the multiple `&query_key` values from the ELink XML output. This code uses ELink in "batch" mode, in that only one set of gene IDs is returned and the one-to-one correspondence between protein GIs and Gene IDs is lost. To preserve this one-to-one correspondence, please see Application 4 below.

EPost – ESearch

Input: List of Entrez UIDs (integer identifiers, e.g. PMID, GI, Gene ID)

Output: History set consisting of the subset of posted UIDs that match an Entrez text query

```
use LWP::Simple;

# Given an input set of protein GI numbers, this script creates
# a history set containing the members of the input set that
# correspond to human proteins.
#(Which of these proteins are from human?)

$db = 'protein';
$query = 'human[orgn]';
$id_list = '194680922,50978626,28558982,9507199,6678417';

#assemble the epost URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "epost.fcgi?db=$db&id=$id_list";

#post the epost URL
$output = get($url);

#parse WebEnv and QueryKey
$web = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);

#assemble the esearch URL
$term = "%23$key+AND+$query";
# %23 places a '#' before the query key
$url = $base . "esearch.fcgi?db=$db&term=$term";
$url .= "&WebEnv=$web&usehistory=y";

#post esearch URL
$limited = get($url);

print "$limited\n";

# Output remains on the history server (&query_key, &WebEnv)
# Use ESummary or EFetch as above to retrieve them
```

Note: To post a large number (more than a few hundred) UIDs in a single URL, please use the HTTP POST method for the EPost call (see Application 4).

ELink – ESearch

Input: List of Entrez UIDs (integer identifiers, e.g. PMID, GI, Gene ID) in database A

Output: History set consisting of the subset of linked UIDs in database B that match an Entrez text query

use LWP::Simple;

```
# Given an input set of protein GI numbers, this script creates a
# history set containing the gene IDs linked to members of the input
# set that also are on human chromosome X.
#(Which of the input proteins are encoded by a gene on human
# chromosome X?)

$db1 = 'protein'; # &dbfrom
$db2 = 'gene';    # &db
$linkname = 'protein_gene'; # desired link &linkname
$query = 'human[orgn]+AND+x[chr]';
#input UIDs in $db1 (protein GIs)
$id_list = '148596974,42544182,187937179,4557377,6678417';

#assemble the elink URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "elink.fcgi?dbfrom=$db1&db=$db2&id=$id_list";
$url .= "&linkname=$linkname&cmd=neighbor_history";

#post the elink URL
$output = get($url);

#parse WebEnv and QueryKey
$web = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);

#assemble the esearch URL
$term = "%23$key+AND+$query"; # %23 places a '#' before the query key
$url = $base . "esearch.fcgi?db=$db2&term=$term&WebEnv=$web&usehistory=y";

#post esearch URL
$limited = get($url);

print "$limited\n";

# Output remains on the history server (&query_key, &WebEnv)
# Use ESummary or EFetch as in previous examples to retrieve them
```

Note: To submit a large number (more than a few hundred) UIDs to ELink in one URL, please use the HTTP POST method for the Elink call (see Application 4). The `&linkname` parameter is used to force ELink to return only one set of links (one `&query_key`) to simplify parsing. If more than one link is desired, the above code must be altered to parse the multiple `&query_key` values from the ELink XML output. This code uses ELink in "batch" mode, in that only one set of gene IDs is returned and the one-to-one correspondence between protein GIs and Gene IDs is lost. To preserve this one-to-one correspondence, please see Application 4 below.

Application 1: Converting GI numbers to accession numbers

Goal: Starting with a list of nucleotide GI numbers, prepare a set of corresponding accession numbers.

Solution: Use EFetch with `&rettype=acc`

Input: \$gi_list – comma-delimited list of GI numbers

Output: List of accession numbers.

```
use LWP::Simple;
$gi_list = '24475906,224465210,50978625,9507198';

#assemble the URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "efetch.fcgi?db=nucleotide&id=$gi_list&rettype=acc";

#post the URL
$output = get($url);
print "$output";
```

Notes: The order of the accessions in the output will be the same order as the GI numbers in \$gi_list.

Application 2: Converting accession numbers to data

Goal: Starting with a list of protein accession numbers, return the sequences in FASTA format.

Solution: Create a string consisting of items separated by 'OR', where each item is an accession number followed by '[accn]'.

Example: accn1[accn]+OR+accn2[accn]+OR+accn3[accn]+OR+...

Submit this string as a &term in ESearch, then use EFetch to retrieve the FASTA data.

Input: \$acc_list – comma-delimited list of accessions

Output: FASTA data

```
use LWP::Simple;
$acc_list = 'NM_009417,NM_000547,NM_001003009,NM_019353';
@acc_array = split(/,/,$acc_list);

#append [accn] field to each accession
for ($i=0; $i < @acc_array; $i++) {
    $acc_array[$i] .= "[accn]";
}

#join the accessions with OR
$query = join('+OR+',@acc_array);

#assemble the esearch URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "esearch.fcgi?db=nuccore&term=$query&usehistory=y";

#post the esearch URL
$output = get($url);

#parse WebEnv and QueryKey
$web = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);

#assemble the efetch URL
$url = $base . "efetch.fcgi?db=nuccore&query_key=$key&WebEnv=$web";
$url .= "&rettype=fasta&retmode=text";
```

```
#post the efetch URL
$fasta = get($url);
print "$fasta";
```

Notes: For large numbers of accessions, use HTTP POST to submit the esearch request (see Application 4), and see Application 3 below for downloading the large set in batches.

Application 3: Retrieving large datasets

Goal: Download all chimpanzee mRNA sequences in FASTA format (>50,000 sequences).

Solution: First use ESearch to retrieve the GI numbers for these sequences and post them on the History server, then use multiple EFetch calls to retrieve the data in batches of 500.

Input: \$query – chimpanzee[orgn]+AND+biomol+mRNA[prop]

Output: A file named "chimp.fna" containing FASTA data.

```
use LWP::Simple;
$query = 'chimpanzee[orgn]+AND+biomol+mRNA[prop]';

#assemble the esearch URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "esearch.fcgi?db=nucleotide&term=$query&usehistory=y";

#post the esearch URL
$output = get($url);

#parse WebEnv, QueryKey and Count (# records retrieved)
$web = $1 if ($output =~ /<WebEnv>(\S+)<\/WebEnv>/);
$key = $1 if ($output =~ /<QueryKey>(\d+)<\/QueryKey>/);
$count = $1 if ($output =~ /<Count>(\d+)<\/Count>/);

#open output file for writing
open(OUT, ">chimp.fna") || die "Can't open file!\n";

#retrieve data in batches of 500
$retmax = 500;
for ($retstart = 0; $retstart < $count; $retstart += $retmax) {
    $efetch_url = $base . "efetch.fcgi?db=nucleotide&WebEnv=$web";
    $efetch_url .= "&query_key=$key&retstart=$retstart";
    $efetch_url .= "&retmax=$retmax&rettype=fasta&retmode=text";
    $efetch_out = get($efetch_url);
    print OUT "$efetch_out";
}
close OUT;
```

Application 4: Finding unique sets of linked records for each member of a large dataset

Goal: Download separately the SNP rs numbers (identifiers) for each current gene on human chromosome 20.

Solution: First use ESearch to retrieve the Gene IDs for the genes, and then assemble an ELink URL where each Gene ID is submitted as a separate &id parameter.

Input: \$query – human[orgn]+AND+20[chr]+AND+alive[prop]

Output: A file named "snp_table" containing on each line the gene id followed by a colon (":") followed by a comma-delimited list of the linked SNP rs numbers.

```

use LWP::Simple;
use LWP::UserAgent;
$query = 'human[orgn]+AND+20[chr]+AND+alive[prop]';
$db1 = 'gene';
$db2 = 'snp';
$linkname = 'gene_snp';

#assemble the esearch URL
$base = 'https://eutils.ncbi.nlm.nih.gov/entrez/eutils/';
$url = $base . "esearch.fcgi?db=$db1&term=$query&usehistory=y&retmax=5000";

#post the esearch URL
$output = get($url);

#parse IDs retrieved
while ($output =~ /<Id>(\d+?)<\Id>/sg) {
    push(@ids, $1);
}

#assemble the elink URL as an HTTP POST call
$url = $base . "elink.fcgi";

$url_params = "dbfrom=$db1&db=$db2&linkname=$linkname";
foreach $id (@ids) {
    $url_params .= "&id=$id";
}

#create HTTP user agent
$ua = new LWP::UserAgent;
$ua->agent("elink/1.0 " . $ua->agent);

#create HTTP request object
$req = new HTTP::Request POST => "$url";
$req->content_type('application/x-www-form-urlencoded');
$req->content("$url_params");

#post the HTTP request
$response = $ua->request($req);
$output = $response->content;

open (OUT, ">snp_table") || die "Can't open file!\n";

while ($output =~ /<LinkSet>(.*?)<\LinkSet>/sg) {

    $linkset = $1;
    if ($linkset =~ /<IdList>(.*?)<\IdList>/sg) {
        $input = $1;
        $input_id = $1 if ($input =~ /<Id>(\d+)<\Id>/sg);
    }

    while ($linkset =~ /<Link>(.*?)<\Link>/sg) {
        $link = $1;
        push (@output, $1) if ($link =~ /<Id>(\d+)<\Id>/);
    }
}

```

```
print OUT "$input_id:" . join(',', @output) . "\n";  
}  
close OUT;
```

Notes: This example uses an HTTP POST request for the elink call, as the number of Gene IDs is over 500. The `&retmax` parameter in the ESearch call is set to 5000, as this is a reasonable limit to the number of IDs to send to ELink in one request (if you send 5000 IDs, you are effectively performing 5000 ELink operations). If you need to link more than 5000 records, add `&retstart` to the ESearch call and repeat the entire procedure for each batch of 5000 IDs, incrementing `&retstart` for each batch.

Demonstration Programs

Please see Chapter 1 for sample Perl scripts.

For More Information

Please see [Chapter 1](#) for getting additional information about the E-utilities.

The E-utilities In-Depth: Parameters, Syntax and More

Eric Sayers, PhD^{✉1}

Created: May 29, 2009; Updated: March 4, 2026.

Introduction

This chapter serves as a reference for all supported parameters for the E-utilities, along with accepted values and usage guidelines. This information is provided for each E-utility in sections below, and parameters and/or values specific to particular databases are discussed within each section. Most E-utilities have a set of parameters that are required for any call, in addition to several additional optional parameters that extend the tool's functionality. These two sets of parameters are discussed separately in each section.

General Usage Guidelines

Please see Chapter 2 for a detailed discussion of E-utility usage policy. The following two parameters should be included in all E-utility requests.

tool

Name of application making the E-utility call. Value must be a string with no internal spaces.

email

E-mail address of the E-utility user. Value must be a string with no internal spaces, and should be a valid e-mail address.

If you expect to post more than 3 E-utility requests per second from a single IP address, consider including the following parameter:

api_key

Value of the API key for sites that post more than 3 requests per second. Please see Chapter 2 for a full discussion of this policy.

E-utilities DTDs

With the exception of EFetch, the E-utilities each generate a single XML output format that conforms to a DTD specific for that utility. Links to these DTDs are provided in the XML headers of the E-utility returns.

ESummary version 2.0 produces unique XML DocSums for each Entrez database, and as such each Entrez database has a unique DTD for version 2.0 DocSums. Links to these DTDs are provided in the version 2.0 XML.

EFetch produces output in a variety of formats, some of which are XML. Most of these XML formats also conform to DTDs or schema specific to the relevant Entrez database. Please follow the appropriate link below for the PubMed DTD:

- [PubMed DTD June 2018 – current PubMed DTD](#)
- [PubMed DTD January 2019 – forthcoming DTD](#)

Author Affiliation: 1 NCBI; Email: sayers@ncbi.nlm.nih.gov.

[✉] Corresponding author.

EInfo

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/einfo.fcgi>

Functions

- Provides a list of the names of all valid Entrez databases
- Provides statistics for a single database, including lists of indexing fields and available link names

Required Parameters

None. If no **db** parameter is provided, einfo will return a list of the names of all valid Entrez databases.

Optional Parameters

db

Target database about which to gather statistics. Value must be a valid [Entrez database name](#).

version

Used to specify version 2.0 EInfo XML. The only supported value is '2.0'. When present, EInfo will return XML that includes two new fields: <IsTruncatable> and <IsRangeable>. Fields that are truncatable allow the wildcard character '*' in terms. The wildcard character will expand to match any set of characters up to a limit of 600 unique expansions. Fields that are rangeable allow the range operator ':' to be placed between a lower and upper limit for the desired range (e.g. 2008:2010[pdat]).

retmode

Retrieval type. Determines the format of the returned output. The default value is 'xml' for EInfo XML, but 'json' is also supported to return output in JSON format.

Examples

Return a list of all Entrez database names:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/einfo.fcgi>

Return version 2.0 statistics for Entrez Protein:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/einfo.fcgi?db=protein&version=2.0>

ESearch

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi>

Functions

- Provides a list of UIDs matching a text query
- Posts the results of a search on the History server
- Downloads all UIDs from a dataset stored on the History server

- Combines or limits UID datasets stored on the History server
- Sorts sets of UIDs

API users should be aware that some NCBI products contain search tools that generate content from searches on the web interface that are not available to ESearch. For example, the PubMed web interface (pubmed.ncbi.nlm.nih.gov) contains citation matching and spelling correction tools that are only available through that interface. Please see ECitMatch and ESpell below for API equivalents.

Required Parameters

db

Database to search. Value must be a valid [Entrez database name](#) (default = pubmed).

term

Entrez text query. All special characters must be URL encoded. Spaces may be replaced by '+' signs. For very long queries (more than several hundred characters long), consider using an HTTP POST call. See the [PubMed](#) or [Entrez](#) help for information about search field descriptions and tags. Search fields and tags are database specific.

```
esearch.fcgi?db=pubmed&term=asthma
```

PubMed also offers “[proximity searching](#)” for multiple terms appearing in any order within a specified number of words from one another in the [Title] or [Title/Abstract] fields.

```
esearch.fcgi?db=pubmed&term="asthma treatment"[Title:~3]
```

Optional Parameters – History Server

usehistory

When **usehistory** is set to 'y', ESearch will post the UIDs resulting from the search operation onto the History server so that they can be used directly in a subsequent E-utility call. Also, **usehistory** must be set to 'y' for ESearch to interpret query key values included in **term** or to accept a **WebEnv** as input.

WebEnv

Web environment string returned from a previous ESearch, EPost or ELink call. When provided, ESearch will post the results of the search operation to this pre-existing WebEnv, thereby appending the results to the existing environment. In addition, providing **WebEnv** allows query keys to be used in **term** so that previous search sets can be combined or limited. As described above, if **WebEnv** is used, **usehistory** must be set to 'y'.

```
esearch.fcgi?db=pubmed&term=asthma&WebEnv=<webenv string>&usehistory=y
```

query_key

Integer query key returned by a previous ESearch, EPost or ELink call. When provided, ESearch will find the intersection of the set specified by **query_key** and the set retrieved by the query in **term** (i.e. joins the two with AND). For **query_key** to function, **WebEnv** must be assigned an existing WebEnv string and **usehistory** must be set to 'y'.

Values for query keys may also be provided in **term** if they are preceded by a '#' (%23 in the URL). While only one **query_key** parameter can be provided to ESearch, any number of query keys can be combined in **term**. Also, if query keys are provided in **term**, they can be combined with OR or NOT in addition to AND.

The following two URLs are functionally equivalent:

```
esearch.fcgi?db=pubmed&term=asthma&query_key=1&WebEnv=
<webenv string>&usehistory=y
```

```
esearch.fcgi?db=pubmed&term=%231+AND+asthma&WebEnv=
<webenv string>&usehistory=y
```

Optional Parameters – Retrieval

retstart

Sequential index of the first UID in the retrieved set to be shown in the XML output (default=0, corresponding to the first record of the entire set). This parameter can be used in conjunction with **retmax** to download an arbitrary subset of UIDs retrieved from a search.

retmax

Total number of UIDs from the retrieved set to be shown in the XML output (default=20). By default, ESearch only includes the first 20 UIDs retrieved in the XML output. If **usehistory** is set to 'y', the remainder of the retrieved set will be stored on the History server; otherwise these UIDs are lost. Increasing **retmax** allows more of the retrieved UIDs to be included in the XML output, up to a maximum of 10,000 records.

To retrieve more than 10,000 UIDs from databases other than PubMed or PMC, submit multiple Esearch requests while incrementing the value of **retstart** (see Application 3). For PubMed and PMC, ESearch can only retrieve the first 10,000 records matching the query. To obtain more than 10,000 PubMed records, consider using EDirect (https://www.ncbi.nlm.nih.gov/books/NBK179288/#_chapter6_Local_PubMed_Cache_) that contains additional logic to batch PubMed search results automatically so that an arbitrary number can be retrieved. Alternatively, consider adding terms to your query (such as date limitations) to segment the desired set into batches of less than 10,000 records each.

rettype

Retrieval type. There are two allowed values for ESearch: 'uilst' (default), which displays the standard XML output, and 'count', which displays only the <Count> tag.

retmode

Retrieval type. Determines the format of the returned output. The default value is 'xml' for ESearch XML, but 'json' is also supported to return output in JSON format.

sort

Specifies the method used to sort UIDs in the ESearch output. The available values vary by database (**db**) and may be found in the Display Settings menu on an Entrez search results page. If **usehistory** is set to 'y', the UIDs are loaded onto the History Server in the specified sort order and will be retrieved in that order by ESummary or EFetch. Example values are 'relevance' and 'name' for Gene. Users should be aware that the default value of **sort** varies from one database to another, and that the default value used by ESearch for a given database may differ from that used on NCBI web search pages.

Values of **sort** for PubMed are as follows:

- *pub_date* – descending sort by publication date
- *Author* – ascending sort by first author
- *JournalName* – ascending sort by journal name

- *relevance* – default sort order, (“Best Match”) on web PubMed

field

Search field. If used, the entire search term will be limited to the specified Entrez field. The following two URLs are equivalent:

```
esearch.fcgi?db=pubmed&term=asthma&field=title
```

```
esearch.fcgi?db=pubmed&term=asthma[title]
```

idtype

Specifies the type of identifier to return for sequence databases (nuccore, popset, protein). By default, ESearch returns GI numbers in its output. If **idtype** is set to 'acc', ESearch will return accession.version identifiers rather than GI numbers.

Optional Parameters – Dates

datatype

Type of date used to limit a search. The allowed values vary between Entrez databases, but common values are 'mdat' (modification date), 'pdat' (publication date) and 'edat' (Entrez date). Generally an Entrez database will have only two allowed values for **datatype**.

reldate

When **reldate** is set to an integer *n*, the search returns only those items that have a date specified by **datatype** within the last *n* days.

mindate, maxdate

Date range used to limit a search result by the date specified by **datatype**. These two parameters (**mindate**, **maxdate**) must be used together to specify an arbitrary date range. The general date format is YYYY/MM/DD, and these variants are also allowed: YYYY, YYYY/MM.

Examples

Search in PubMed with the term *cancer* for abstracts that have an Entrez date within the last 60 days; retrieve the first 100 PMIDs and translations; post the results on the History server and return a **WebEnv** and **query_key**:

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?  
db=pubmed&term=cancer&reldate=60&datatype=edat&retmax=100&usehistory=y
```

Search in PubMed for the journal PNAS, Volume 97, and retrieve six PMIDs starting with the seventh PMID in the list:

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?db=pubmed&term=PNAS\[ta\]  
+AND+97\[vi\]&retstart=6&retmax=6&tool=biomed3
```

Search in the NLM Catalog for journals matching the term *obstetrics*:

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?  
db.nlmcatalog&term=obstetrics+AND+ncbijournals\[filter\]
```

Search PubMed Central for free full text articles containing the query *stem cells*:

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=pmc&term=stem+cells+AND+free+fulltext\[filter\]](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=pmc&term=stem+cells+AND+free+fulltext[filter])

Search in Nucleotide for all tRNAs:

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&term=biomol+trna\[prop\]](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&term=biomol+trna[prop])

Search in Protein for a molecular weight range:

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=protein&term=70000:90000\[molecular+weight\]](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=protein&term=70000:90000[molecular+weight])

EPost

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/epost.fcgi>

Functions

- Uploads a list of UIDs to the Entrez History server
- Appends a list of UIDs to an existing set of UID lists attached to a Web Environment

Required Parameters

db

Database containing the UIDs in the input list. The value must be a valid [Entrez database name](#) (default = pubmed).

id

UID list. Either a single UID or a comma-delimited list of UIDs may be provided. All of the UIDs must be from the database specified by **db**. For PubMed and PMC, no more than 10,000 UIDs can be included in a single URL request. For other databases there is no set maximum for the number of UIDs that can be passed to epost, but if more than about 200 UIDs are to be posted, the request should be made using the HTTP POST method.

For sequence databases (nucleotide, protein), the UID list may be a mixed list of GI numbers and accession.version identifiers. **Note:** When using accession.version identifiers, there is a conversion step that takes place that causes large lists of identifiers to time out, even when using POST. Therefore, we recommend batching these types of requests in sizes of about 500 UIDs or less, to avoid retrieving only a partial amount of records from your original POST input list.

```
epost.fcgi?db=pubmed&id=19393038,30242208,29453458  
epost.fcgi?db=protein&id=15718680,NP_001098858.1,119703751
```

Optional Parameter

WebEnv

Web Environment. If provided, this parameter specifies the Web Environment that will receive the UID list sent by post. EPost will create a new query key associated with that Web Environment. Usually this WebEnv value is obtained from the output of a previous ESearch, EPost or ELink call. If no **WebEnv** parameter is provided, EPost will create a new Web Environment and post the UID list to **query_key 1**.

```
epost.fcgi?db=protein&id=15718680,157427902,119703751&WebEnv=
<webenv string>
```

Example

Post records to PubMed:

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/epost.fcgi?db=pubmed&id=11237011,12466850
```

ESummary

Base URL

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi
```

Functions

- Returns document summaries (DocSums) for a list of input UIDs
- Returns DocSums for a set of UIDs stored on the Entrez History server

Required Parameter

db

Database from which to retrieve DocSums. The value must be a valid [Entrez database name](#) (default = pubmed).

Required Parameter – Used only when input is from a UID list

id

UID list. Either a single UID or a comma-delimited list of UIDs may be provided. All of the UIDs must be from the database specified by **db**. There is no set maximum for the number of UIDs that can be passed to ESummary, but if more than about 200 UIDs are to be provided, the request should be made using the HTTP POST method.

For sequence databases (nuccore, popset, protein), the UID list may be a mixed list of GI numbers and accession.version identifiers.

```
esummary.fcgi?db=pubmed&id=19393038,30242208,29453458
esummary.fcgi?db=protein&id=15718680,NP_001098858.1,119703751
```

Required Parameters – Used only when input is from the Entrez History server

query_key

Query key. This integer specifies which of the UID lists attached to the given Web Environment will be used as input to ESummary. Query keys are obtained from the output of previous ESearch, EPost or ELink calls. The **query_key** parameter must be used in conjunction with **WebEnv**.

WebEnv

Web Environment. This parameter specifies the Web Environment that contains the UID list to be provided as input to ESummary. Usually this WebEnv value is obtained from the output of a previous ESearch, EPost or ELink call. The **WebEnv** parameter must be used in conjunction with **query_key**.

```
esummary.fcgi?db=protein&query_key=<key>&WebEnv=<webenv string>
```

Optional Parameters – Retrieval

retstart

Sequential index of the first DocSum to be retrieved (default=1, corresponding to the first record of the entire set). This parameter can be used in conjunction with **retmax** to download an arbitrary subset of DocSums from the input set.

retmax

Total number of DocSums from the input set to be retrieved, up to a maximum of 10,000. If the total set is larger than this maximum, the value of **retstart** can be iterated while holding **retmax** constant, thereby downloading the entire set in batches of size **retmax**.

retmode

Retrieval type. Determines the format of the returned output. The default value is 'xml' for ESummary XML, but 'json' is also supported to return output in JSON format.

version

Used to specify version 2.0 ESummary XML. The only supported value is '2.0'. When present, ESummary will return version 2.0 DocSum XML that is unique to each Entrez database and that often contains more data than the default DocSum XML.

Examples

PubMed:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=pubmed&id=11850928,11482001>

PubMed, version 2.0 XML:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=pubmed&id=11850928,11482001&version=2.0>

Protein:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=protein&id=28800982,28628843>

Nucleotide:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=nucleotide&id=28864546,28800981>

Structure:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=structure&id=19923,12120>

Taxonomy:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=taxonomy&id=9913,30521>

UniSTS:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi?db=unists&id=254085,254086>

EFetch

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi>

Functions

- Returns formatted data records for a list of input UIDs
- Returns formatted data records for a set of UIDs stored on the Entrez History server

Required Parameters

db

Database from which to retrieve records. The value must be a valid [Entrez database name](#) (default = pubmed). Currently EFetch does not support all Entrez databases. Please see Table 1 in Chapter 2 for a list of available databases.

Required Parameter – Used only when input is from a UID list

id

UID list. Either a single UID or a comma-delimited list of UIDs may be provided. All of the UIDs must be from the database specified by **db**. There is no set maximum for the number of UIDs that can be passed to EFetch, but if more than about 200 UIDs are to be provided, the request should be made using the HTTP POST method.

For sequence databases (nucore, popset, protein), the UID list may be a mixed list of GI numbers and accession.version identifiers.

```
efetch.fcgi?db=pubmed&id=19393038,30242208,29453458  
efetch.fcgi?db=protein&id=15718680,NP_001098858.1,119703751
```

Special note for sequence databases.

NCBI is no longer assigning GI numbers to a growing number of new sequence records. As such, these records are not indexed in Entrez, and so cannot be retrieved using ESearch or ESummary, and have no Entrez links accessible by ELink. EFetch *can* retrieve these records by including their accession.version identifier in the **id** parameter.

Required Parameters – Used only when input is from the Entrez History server

query_key

Query key. This integer specifies which of the UID lists attached to the given Web Environment will be used as input to EFetch. Query keys are obtained from the output of previous ESearch, EPost or ELink calls. The **query_key** parameter must be used in conjunction with **WebEnv**.

WebEnv

Web Environment. This parameter specifies the Web Environment that contains the UID list to be provided as input to EFetch. Usually this WebEnv value is obtained from the output of a previous ESearch, EPost or ELink call. The **WebEnv** parameter must be used in conjunction with **query_key**.

efetch.fcgi?db=protein&query_key=<key>&WebEnv=<webenv string>

Optional Parameters – Retrieval

retmode

Retrieval mode. This parameter specifies the data format of the records returned, such as plain text, HTML or XML. See Table 1 for a full list of allowed values for each database.

Table 1 – Valid values of &retmode and &rettype for EFetch (null = empty string)

Record Type	&rettype	&retmode
All Databases		
Document summary	docsum	xml, <i>default</i>
List of UIDs in XML	uilest	xml
List of UIDs in plain text	uilest	text
db = bioproject		
Full record XML	xml, <i>default</i>	xml, <i>default</i>
db = biosample		
Full record XML	full, <i>default</i>	xml, <i>default</i>
Full record text	full, <i>default</i>	text
db = gds		
Summary	summary, <i>default</i>	text, <i>default</i>
db = gene		
text ASN.1	<i>null</i>	asn.1, <i>default</i>
XML	<i>null</i>	xml
Gene table	gene_table	text
db = homologene		

Table 1 continued from previous page.

text ASN.1	<i>null</i>	asn.1, <i>default</i>
XML	<i>null</i>	xml
Alignment scores	alignmentscores	text
FASTA	fasta	text
HomoloGene	homologene	text
db = mesh		
Full record	full, <i>default</i>	text, <i>default</i>
db = nlmcatalog		
Full record	<i>null</i>	text, <i>default</i>
XML	<i>null</i>	xml
db = nuccore, protein or popset		
text ASN.1	<i>null</i>	text, <i>default</i>
binary ASN.1	<i>null</i>	asn.1
Full record in XML	native	xml
Accession number(s)	acc	text
FASTA	fasta	text
TinySeq XML	fasta	xml
SeqID string	seqid	text
Additional options for db = nuccore or popset		
GenBank flat file	gb	text

Table 1 continued from previous page.

GBSeq XML	gb	xml
INSDSeq XML	gbc	xml
Additional option for db = nuccore and protein		
Feature table	ft	text
Additional option for db = nuccore		
GenBank flat file with full sequence (contigs)	gbwithparts	text
CDS nucleotide FASTA	fasta_cds_na	text
CDS protein FASTA	fasta_cds_aa	text
Additional options for db = protein		
GenPept flat file	gp	text
GBSeq XML	gp	xml
INSDSeq XML	gpc	xml
Identical Protein XML	ipg	xml
db = pmc		
XML	<i>null</i>	xml, <i>default</i>
MEDLINE	medline	text
db = pubmed		
XML	<i>null</i>	xml, <i>default</i>
MEDLINE	medline	text
PMID list	uolist	text

Table 1 continued from previous page.

Abstract	abstract	text
db = sequences		
text ASN.1	<i>null</i>	text, <i>default</i>
Accession number(s)	acc	text
FASTA	fasta	text
SeqID string	seqid	text
db = snp		
text ASN.1	<i>null</i>	asn.1, <i>default</i>
XML	<i>null</i>	xml
Flat file	flt	text
FASTA	fasta	text
RS Cluster report	rsr	text
SS Exemplar list	ssexemplar	text
Chromosome report	chr	text
Summary	docset	text
UID list	uilst	text or xml
db = sra		
XML	full, <i>default</i>	xml, <i>default</i>
db = taxonomy		
XML	<i>null</i>	xml, <i>default</i>

Table 1 continued from previous page.

TaxID list	uilib	text or xml
db = clinvar		
ClinVar Set	clinvarset	xml, <i>default</i>
UID list	uilib	text or xml
db = gtr		
GTR Test Report	gtracc	xml, <i>default</i>

rettype

Retrieval type. This parameter specifies the record view returned, such as Abstract or MEDLINE from PubMed, or GenPept or FASTA from protein. Please see Table 1 for a full list of allowed values for each database.

retstart

Sequential index of the first record to be retrieved (default=0, corresponding to the first record of the entire set). This parameter can be used in conjunction with **retmax** to download an arbitrary subset of records from the input set.

retmax

Total number of records from the input set to be retrieved, up to a maximum of 10,000. Optionally, for a large set the value of **retstart** can be iterated while holding **retmax** constant, thereby downloading the entire set in batches of size **retmax**.

Optional Parameters – Sequence Databases**strand**

Strand of DNA to retrieve. Available values are "1" for the plus strand and "2" for the minus strand.

seq_start

First sequence base to retrieve. The value should be the integer coordinate of the first desired base, with "1" representing the first base of the sequence.

seq_stop

Last sequence base to retrieve. The value should be the integer coordinate of the last desired base, with "1" representing the first base of the sequence.

complexity

Data content to return. Many sequence records are part of a larger data structure or "blob", and the **complexity** parameter determines how much of that blob to return. For example, an mRNA may be stored together with its protein product. The available values are as follows:

Value of complexity	Data returned for each requested GI
0	entire blob
1	bioseq
2	minimal bioseq-set
3	minimal nuc-prot
4	minimal pub-set

Examples

PubMed

Fetch PMIDs 17284678 and 9997 as text abstracts:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=pubmed&id=17284678,9997&retmode=text&rettype=abstract>

Fetch PMIDs in XML:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=pubmed&id=11748933,11700088&retmode=xml>

PubMed Central

Fetch XML for PubMed Central ID 212403:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=pmc&id=212403>

Nucleotide/Nuccore

Fetch the first 100 bases of the plus strand of GI 21614549 in FASTA format:

https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=21614549&strand=1&seq_start=1&seq_stop=100&rettype=fasta&retmode=text

Fetch the first 100 bases of the minus strand of GI 21614549 in FASTA format:

https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=21614549&strand=2&seq_start=1&seq_stop=100&rettype=fasta&retmode=text

Fetch the nuc-prot object for GI 21614549:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=21614549&complexity=3>

Fetch the full ASN.1 record for GI 5:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&id=5>

Fetch FASTA for GI 5:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&id=5&rettype=fasta>

Fetch the GenBank flat file for GI 5:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&id=5&rettype=gb>

Fetch GBSeqXML for GI 5:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&id=5&rettype=gb&retmode=xml>

Fetch TinySeqXML for GI 5:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nucleotide&id=5&rettype=fasta&retmode=xml>

Popset

Fetch the GenPept flat file for Popset ID 12829836:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=popset&id=12829836&rettype=gp>

Protein

Fetch the GenPept flat file for GI 8:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=protein&id=8&rettype=gp>

Fetch GBSeqXML for GI 8:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=protein&id=8&rettype=gp&retmode=xml>

Sequences

Fetch FASTA for a transcript and its protein product (GIs 312836839 and 34577063)

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=sequences&id=312836839,34577063&rettype=fasta&retmode=text>

Gene

Fetch full XML record for Gene ID 2:

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=gene&id=2&retmode=xml>

ELink

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi>

Functions

- Returns UUIDs linked to an input set of UUIDs in either the same or a different Entrez database
- Returns UUIDs linked to other UUIDs in the same Entrez database that match an Entrez query
- Checks for the existence of Entrez links for a set of UUIDs within the same database
- Lists the available links for a UUID
- Lists LinkOut URLs and attributes for a set of UUIDs
- Lists hyperlinks to primary LinkOut providers for a set of UUIDs
- Creates hyperlinks to the primary LinkOut provider for a single UUID

Required Parameters

Specifying the desired link (linkname, db, dbfrom)

Option 1 (preferred): linkname

Using **linkname** is the most efficient option as ELink will retrieve data for only that single link type.

linkname: Name of the Entrez link to retrieve. Every link in Entrez is given a name of the form

dbfrom_db_subset.

The values of *subset* vary depending on the values of *dbfrom* and *db*. Many *dbfrom/db* combinations have no *subset* values. See [the list of Entrez links](#) for a listing of all available linknames. When **linkname** is used, only the links with that name will be retrieved.

The **linkname** parameter only functions when **cmd** is set to *neighbor* or *neighbor_history*.

Find RefSeq proteins linked to a gene.

https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?id=93986&linkname=gene_protein_refseq

Option 2: db and dbfrom

Using **db** and/or **dbfrom** will retrieve all available link types between **dbfrom** and **db**. ELink will assign the value *pubmed* to either parameter if that parameter is omitted. If **linkname** is included, ELink will ignore both **db** and **dbfrom**.

db: Database from which to retrieve UIDs. The value must be a valid [Entrez database name](#) (default = pubmed). This is the destination database for the link operation.

dbfrom: Database containing the input UIDs. The value must be a valid [Entrez database name](#) (default = pubmed). This is the origin database of the link operation. If **db** and **dbfrom** are set to the same database value, then ELink will return computational neighbors within that database. Please see the [full list of Entrez links](#) for available computational neighbors. Computational neighbors have linknames that begin with *dbname_dbname* (examples: *protein_protein*, *pcassay_pcassay_activityneighbor*).

Find all proteins linked to a gene.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=gene&db=protein&id=93986>

cmd

ELink command mode. The command mode specified which function ELink will perform. Some optional parameters only function for certain values of **cmd** (see below).

cmd=neighbor (default)

ELink returns a set of UIDs in **db** linked to the input UIDs in **dbfrom**.

Example: Link from protein to gene

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=protein&db=gene&id=15718680,157427902>

cmd=neighbor_score

ELink returns a set of UIDs within the same database as the input UIDs along with computed similarity scores.

Example: Find related articles to PMID 20210808

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?  
dbfrom=pubmed&db=pubmed&id=20210808&cmd=neighbor_score  
cmd=neighbor_history
```

ELink posts the output UIDs to the Entrez History server and returns a **query_key** and **WebEnv** corresponding to the location of the output set.

Example: Link from protein to gene and post the results on the Entrez History

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?  
dbfrom=protein&db=gene&id=15718680,157427902&cmd=neighbor_history  
cmd=acheck
```

ELink lists all links available for a set of UIDs.

Example: List all possible links from two protein GIs

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?  
dbfrom=protein&id=15718680,157427902&cmd=acheck
```

Example: List all possible links from two protein GIs to PubMed

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?  
dbfrom=protein&db=pubmed&id=15718680,157427902&cmd=acheck  
cmd=ncheck
```

ELink checks for the existence of links *within the same database* for a set of UIDs. These links are equivalent to setting **db** and **dbfrom** to the same value.

Example: Check whether two nucleotide sequences have "related sequences" links.

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?  
dbfrom=nucleotide&id=21614549,219152114&cmd=ncheck  
cmd=lcheck
```

Elink checks for the existence of external links (LinkOuts) for a set of UIDs.

Example: Check whether two protein sequences have any LinkOut providers.

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?  
dbfrom=protein&id=15718680,157427902&cmd=lcheck  
cmd=llinks
```

For each input UID, ELink lists the URLs and attributes for the LinkOut providers that are not libraries.

Example: List the LinkOut URLs for non-library providers for two pubmed abstracts.

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?  
dbfrom=pubmed&id=19880848,19822630&cmd=llinks  
cmd=llinkslib
```

For each input UID, ELink lists the URLs and attributes for *all* LinkOut providers including libraries.

Example: List all LinkOut URLs for two PubMed abstracts.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
dbfrom=pubmed&id=19880848,19822630&cmd=llinkslib](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?dbfrom=pubmed&id=19880848,19822630&cmd=llinkslib)
cmd=prlinks

ELink lists the primary LinkOut provider for each input UID, or links directly to the LinkOut provider's web site for a single UID if **retmode** is set to *ref*.

Example: Find links to full text providers for two PubMed abstracts.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
dbfrom=pubmed&id=19880848,19822630&cmd=prlinks](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?dbfrom=pubmed&id=19880848,19822630&cmd=prlinks)

Example: Link directly to the full text for a PubMed abstract at the provider's web site.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
dbfrom=pubmed&id=19880848&cmd=prlinks&retmode=ref](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?dbfrom=pubmed&id=19880848&cmd=prlinks&retmode=ref)

Required Parameter – Used only when input is from a UID list

id

UID list. Either a single UID or a comma-delimited list of UIDs may be provided. All of the UIDs must be from the database specified by **dbfrom**. There is no set maximum for the number of UIDs that can be passed to ELink, but if more than about 200 UIDs are to be provided, the request should be made using the HTTP POST method.

Link from protein to gene.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
dbfrom=protein&db=gene&id=15718680,157427902,119703751](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?dbfrom=protein&db=gene&id=15718680,157427902,119703751)

Find related sequences (link from nuccore to nuccore).

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?dbfrom=nuccore&db=nuccore&id=34577062>

If more than one **id** parameter is provided, ELink will perform a separate link operation for the set of UIDs specified by each **id** parameter. This effectively accomplishes "one-to-one" links and preserves the connection between the input and output UIDs.

Find one-to-one links from protein to gene.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
dbfrom=protein&db=gene&id=15718680&id=157427902&id=119703751](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?dbfrom=protein&db=gene&id=15718680&id=157427902&id=119703751)

For sequence databases (nuccore, popset, protein), the UID list may be a mixed list of GI numbers and accession.version identifiers.

Find one-to-one links from protein to gene.

[https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
dbfrom=protein&db=gene&id=15718680&id=NP_001098858.1&id=119703751](https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?dbfrom=protein&db=gene&id=15718680&id=NP_001098858.1&id=119703751)

Required Parameters – Used only when input is from the Entrez History server

query_key

Query key. This integer specifies which of the UID lists attached to the given Web Environment will be used as input to ELink. Query keys are obtained from the output of previous ESearch, EPost or ELink calls. The **query_key** parameter must be used in conjunction with **WebEnv**.

WebEnv

Web Environment. This parameter specifies the Web Environment that contains the UID list to be provided as input to ELink. Usually this WebEnv value is obtained from the output of a previous ESearch, EPost or ELink call. The **WebEnv** parameter must be used in conjunction with **query_key**.

Link from protein to gene:

```
elink.fcgi?dbfrom=protein&db=gene&query_key=<key>&WebEnv=<webenv string>
```

Find related sequences (link from protein to protein):

```
elink.fcgi?dbfrom=protein&db=protein&query_key=<key>&WebEnv=
<webenv string>
```

Optional Parameter – Retrieval

retmode

Retrieval type. Determines the format of the returned output. The default value is 'xml' for ELink XML, but 'json' is also supported to return output in JSON format.

idtype

Specifies the type of identifier to return for sequence databases (nuccore, popset, protein). By default, ELink returns GI numbers in its output. If **idtype** is set to 'acc', ELink will return accession.version identifiers rather than GI numbers.

Optional Parameters – Limiting the Output Set of Links

term

Entrez query used to limit the output set of linked UIDs. The query in the **term** parameter will be applied after the link operation, and only those UIDs matching the query will be returned by ELink. The **term** parameter only functions when **db** and **dbfrom** are set to the same database value.

Find all related articles for a PMID.

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=pubmed&db=pubmed&id=19879512
```

Find all related review articles published in 2008 for a PMID.

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=pubmed&db=pubmed&id=19879512&term=review%5Bfilter%5D+AND+2008%5Bpdat%5Dh
```

holding

Name of LinkOut provider. Only URLs for the LinkOut provider specified by **holding** will be returned. The value provided to **holding** should be the abbreviation of the LinkOut provider's name found in the <NameAbbr>

tag of the ELink XML output when **cmd** is set to *llinks* or *llinkslib*. The **holding** parameter only functions when **cmd** is set to *llinks* or *llinkslib*.

Find information for all LinkOut providers for a PMID.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=pubmed&cmd=llinkslib&id=16210666>

Find information from clinicaltrials.gov for a PMID.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi?dbfrom=pubmed&cmd=llinkslib&id=16210666&holding=CTgov>

Optional Parameters – Dates

These parameters only function when **cmd** is set to *neighbor* or *neighbor_history* and **dbfrom** is *pubmed*.

datatype

Type of date used to limit a link operation. The allowed values vary between Entrez databases, but common values are 'mdat' (modification date), 'pdat' (publication date) and 'edat' (Entrez date). Generally an Entrez database will have only two allowed values for **datatype**.

reldate

When **reldate** is set to an integer *n*, ELink returns only those items that have a date specified by **datatype** within the last *n* days.

mindate, maxdate

Date range used to limit a link operation by the date specified by **datatype**. These two parameters (**mindate**, **maxdate**) must be used together to specify an arbitrary date range. The general date format is YYYY/MM/DD, and these variants are also allowed: YYYY, YYYY/MM.

EGQuery

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/egquery.fcgi>

Function

Provides the number of records retrieved in all Entrez databases by a single text query.

Required Parameter

term

Entrez text query. All special characters must be URL encoded. Spaces may be replaced by '+' signs. For very long queries (more than several hundred characters long), consider using an HTTP POST call. See the [PubMed](#) or [Entrez](#) help for information about search field descriptions and tags. Search fields and tags are database specific.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/egquery.fcgi?term=asthma>

ESpell

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/espell.fcgi>

Function

Provides spelling suggestions for terms within a single text query in a given database.

Required Parameters

db

Database to search. Value must be a valid [Entrez database name](#) (default = pubmed).

term

Entrez text query. All special characters must be URL encoded. Spaces may be replaced by '+' signs. For very long queries (more than several hundred characters long), consider using an HTTP POST call. See the [PubMed](#) or [Entrez](#) help for information about search field descriptions and tags. Search fields and tags are database specific.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/espell.fcgi?db=pubmed&term=asthmaa+OR+allergies>

ECitMatch

Base URL

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ecitmatch.cgi>

Function

Retrieves PubMed IDs (PMIDs) that correspond to a set of input citation strings.

Required Parameters

db

Database to search. The only supported value is 'pubmed'.

rettype

Retrieval type. The only supported value is 'xml'.

bdata

Citation strings. Each input citation must be represented by a citation string in the following format:

journal_title|year|volume|first_page|author_name|your_key|

Multiple citation strings may be provided by separating the strings with a carriage return character (%0D). The *your_key* value is an arbitrary label provided by the user that may serve as a local identifier for the citation, and it will be included in the output. Be aware that all spaces must be replaced by '+' symbols and that citation strings should end with a final vertical bar '|'.

<https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ecitmatch.cgi?db=pubmed&retmode=xml&bdata=proc+natl+acad+sci+u+s+a|1991|88|3248|mann+bj|Art1|%0Dscience|1987|235|182|palmenberg+ac|Art2>

Release Notes

EFetch; ELink JSON output: June 24, 2015

- EFetch now supports ClinVar and GTR
- ELink now provides output in JSON format

ESearch &sort; JSON output format: February 14, 2014

- ESearch now provides a supported **sort** parameter
- EInfo, ESearch and ESummary now provide output data in JSON format

ECitMatch, EInfo Version 2.0, EFetch: August 9, 2013

- ECitMatch is a new E-utility that serves as an API to the PubMed [batch citation matcher](#)
- EInfo has an updated XML output that includes two new fields: <IsTruncatable> and <IsRangeable>
- EFetch now supports the BioProject database.

EFetch Version 2.0. Target release date: February 15, 2012

- EFetch now supports the following databases: biosample, biosystems and sra
- EFetch now has defined default values for &retmode and &rettype for all supported databases (please see Table 1 for all supported values of these parameters)
- EFetch no longer supports &retmode=html; requests containing &retmode=html will return data using the default &retmode value for the specified database (&db)
- EFetch requests including &rettype=docsum return XML data equivalent to ESummary output

Release of new Genome database: November 9, 2011

- Entrez Genome has been completely redesigned, and database records now correspond to a species rather than an individual chromosome sequence. Please see full details of the change at <https://www.ncbi.nlm.nih.gov/About/news/17Nov2011.html>
- Old Genome IDs are no longer valid. A file is available on the NCBI FTP site that maps old Genome IDs to Nucleotide GIs: ftp.ncbi.nih.gov/genomes/old_genomeID2nucGI
- EFetch no longer supports retrievals from Genome (db=genome).
- The ESummary XML for Genome has been recast to reflect the new data model.
- To view the new search fields and links supported for the new Genome database, please see <https://eutils.ncbi.nlm.nih.gov/entrez/eutils/einfo.fcgi?db=genome>

ESummary Version 2.0. November 4, 2011

- ESummary now supports a new, alternative XML presentation for Entrez document summaries (DocSums). The new XML is unique to each Entrez database and generally contains more extensive data about the record than the original DocSum XML.
- There are no plans at present to discontinue the original DocSum XML, so developers can continue to use this presentation, which will remain the default.
- Version 2.0 XML is returned when &version=2.0 is included in the ESummary URL.

Demonstration Programs

Please see Chapter 1 for sample Perl scripts.

For More Information

Please see [Chapter 1](#) for getting additional information about the E-utilities.

The E-utility Web Service (SOAP)

Eric Sayers, PhD¹ and Vadim Miller²

Created: January 21, 2010; Updated: January 23, 2015.

Termination Announcement

The SOAP web service for the E-utilities will be **TERMINATED** permanently on July 1, 2015. All requests made to this service after that date will fail.

If you have software that is currently using the E-utility SOAP web service, please plan to transition to using the standard URL interface described in Chapters 1-4 of this book.

Please contact info@ncbi.nlm.nih.gov if you have questions about this change.

For More Information

[E-utility DTDs](#)

Please see [Chapter 1](#) for getting additional information about the E-utilities.

Entrez Direct: E-utilities on the Unix Command Line

Jonathan Kans, PhD^{✉1}

Created: April 23, 2013; Updated: October 8, 2026.

Abstract

Entrez Direct was designed to make it easy for biologists to automate Entrez queries and analyze structured data results without requiring extensive computer programming experience.

EDirect supports both online network queries and local record archives by providing parallel search, link, filter, and fetch scripts. The same structured message format is used for communication between steps, allowing query pipelines to seamlessly operate on a mixture of local and remote data sources.

Central to EDirect's success is a utility that uses command-line arguments to control data extraction from XML, JSON, and ASN.1 text records. It deliberately avoids explicit object paths and complicated path formulas. Instead, the input stream is first partitioned into separate records by parent object name. Then, within each record, data items are found by element name, using a depth-first recursive search.

Exploration commands allow data context to be taken into account by redirecting the search order to separately present each instance of a named container. They are equivalent to nested "for loops", and can keep related fields together in the output. For example, exploration by Author restricts the subsequent searches for Initials and LastName elements to child nodes of the current author object. Conditional processing tests that examine data content are also applied at this level.

Element extraction variants perform simple text modifications or integer calculations on the raw data values. Formatting commands control record, field, and instance separators. A wrapping shortcut places the extracted information inside a new XML object. Combining these features can reduce a complex structured record analysis to an elegant sequence of simpler transformations. No custom reading or writing code is needed, since the data in each step remains in parsable XML format.

EDirect's data extraction and network access tools simplify the construction of local archives. Users can add new public datasets by writing custom helper scripts for the download, generate, populate, and index steps of the build process. Similar processing of tables that associate biological concepts with existing record identifiers can incorporate new indexed search fields. Contributing the helper scripts to a public repository would allow other users to install local copies of any desired datasets.

Researchers or coding agents could then create experimental discovery pipelines that use EDirect's simple query interface to access the new local data domains. The same indexed search mechanism can apply biological reality checks to filter the results of intermediate steps.

Acknowledgments

This work was supported by the National Center for Biotechnology Information of the National Library of Medicine (NLM), National Institutes of Health (NIH). The contributions of the NIH author(s) are considered Works of the United States Government. The findings and conclusions presented in this paper are those of the author(s) and do not necessarily reflect the views of the NIH or the U.S. Department of Health and Human Services.

Author Affiliation: 1 NCBI; Email: kans@ncbi.nlm.nih.gov.

[✉] Corresponding author.

Installation

EDirect consists of a set of scripts and programs that are downloaded to the user's computer. It will run on Unix and Macintosh computers, and under the Cygwin Unix-emulation environment on Windows PCs. To install the EDirect software, open a terminal window and execute one of the following two commands:

```
sh -c "$(curl -fsSL https://ftp.ncbi.nlm.nih.gov/entrez/entrezdirect/install-edirect.sh)"
```

```
sh -c "$(wget -q https://ftp.ncbi.nlm.nih.gov/entrez/entrezdirect/install-edirect.sh -O -)"
```

This will download a number of scripts and several precompiled programs into an "edirect" folder in the user's home directory. It may then print an additional command for updating the PATH environment variable in the user's configuration file. The editing instructions will look something like:

```
echo "export PATH=\$HOME/edirect:\$PATH" >> \$HOME/.bash_profile
```

As a convenience, the installation process ends by offering to run the PATH update command for you. Answer "y" and press the Return key if you want it run. If the PATH is already set correctly, or if you prefer to make any editing changes manually, just press Return.

Finally, quit the running terminal application. Windows in subsequent terminal sessions will be able to locate the edirect folder and execute EDirect commands.

For best performance of Entrez network service requests, obtain an API Key from NCBI, and place the following line in your .bash_profile and .zshrc configuration files:

```
export NCBI_API_KEY=unique_api_key
```

For faster downloading of NCBI release files, install the free Aspera Connect file transfer utility on your computer. It is available from the IBM Aspera Connect link at:

```
https://www.ibm.com/products/aspera/downloads#cds
```

Introduction

Entrez Direct (EDirect) provides access to Entrez (1), the NCBI's suite of interconnected databases, from a Unix terminal window. Search terms are entered as command-line arguments. Individual operations are connected with Unix pipes to construct multi-step queries. Selected records can then be retrieved in a variety of formats. Each record in a database is assigned its own unique identifier (UID).

The **EDIRECT.pdf** reference document included in the edirect folder contains an abridged version of this document. It is intended to convey the most important points in the least amount of time for the new user, while still presenting the minimal essential details.

The full documentation here gives a more in-depth exploration of the underlying topics, especially in the Additional Examples web page, which is organized by Entrez database.

Programmatic Access

EDirect connects to Entrez through the Entrez Programming Utilities (EUtils) interface. It supports searching by indexed terms, looking up precomputed neighbors or links, filtering results by date or category, and downloading record summaries or reports.

Navigation programs (**esearch**, **elink**, **efilter**, and **efetch**) communicate by means of a small structured message, which can be passed invisibly between operations with a Unix pipe. The message includes the current database, so it does not need to be given as an argument after the first step.

Accessory programs (**nquire**, **transmute**, and **xtract**) can help eliminate the need for writing custom software to answer ad hoc questions. Queries can move effortlessly between EDirect programs and Unix utilities or scripts to perform actions that cannot be accomplished entirely within Entrez.

EDirect programs are designed to work on large sets of data. They handle many technical details behind the scenes, thus avoiding the learning curve normally required for EUtils programming. There is no need to use a script to loop over records in small groups, or write code to retry a query after a transient network or server failure, or add a time delay between requests. All of those features are already built into the system.

Unix programs are run by typing the name of the program and then supplying any required or optional arguments on the command line. Argument names are letters or words that start with a dash ("-") character.

Each program has a **-help** command that prints detailed information about available arguments.

Running **xtract -unix** will print a reference sheet of useful Unix commands and constructs. Additional documentation can be obtained by typing "**man**" followed by a Unix command name.

Navigation Functions

Esearch performs a new Entrez search using terms in indexed fields. It requires a **-db** argument for the database name and uses **-query** for the search terms. For PubMed, without field qualifiers, the server uses automatic term mapping to compose a search strategy by translating the supplied query:

```
esearch -db pubmed -query "selective serotonin reuptake inhibitor"
```

Search terms can also be qualified with a bracketed field name to match within the specified index:

```
esearch -db nuccore -query "insulin [PROT] AND rodents [ORGN]"
```

Elink looks up precomputed neighbors within a database:

```
elink -related
```

or follows annotated links to records in other databases:

```
elink -target gene
```

Elink also connects to the NIH Open Citation Collection service (2) to find publications that cite the selected PubMed articles:

```
elink -cited
```

or to retrieve the reference lists of PubMed records:

```
elink -cites
```

Efilter limits the results of a previous query, with shortcuts that can also be used in esearch:

```
efilter -molecule genomic -location chloroplast -country sweden -mindate 1985
```

Efetch downloads selected records or reports in a style designated by **-format**:

```
efetch -format abstract
```

Einfo can report the fields and links that are indexed for each database:

```
einfo -db protein -fields
```

This will return a table of field abbreviations and names indexed for proteins:

ACCN	Accession
ALL	All Fields
ASSM	Assembly
AUTH	Author
BRD	Breed
CULT	Cultivar
DIV	Division
ECNO	EC/RN Number
FILT	Filter
FKEY	Feature key
...	

Constructing Multi-Line or Multi-Step Queries

A single command can be continued on the next line by typing the Unix backslash ("\") escape character immediately before pressing the Return key. Pay careful attention to the exact spacing if the line break is within a quoted argument:

```
esearch -db biosample -query "package mims metagenome/environmental, \
water version 6 0 [PROP] AND ncbi [FILT] AND biosample sra [FILT]"
```

EDirect allows individual operations to be described separately, combining them into a multi-step query by using the Unix vertical bar ("| ") pipe symbol:

```
esearch -db pubmed -query "tn3 transposition immunity" | efetch -format apa
```

The vertical bar also allows query steps to be placed on separate lines:

```
esearch -db pubmed -query "raynaud disease AND fish oil" |
efetch -format medline
```

In most modern versions of Unix the vertical bar pipe symbol also allows the query to continue on the next line, without the need for an additional backslash. That convention will be followed for the remainder of this document.

Accessory Programs

Nquire retrieves data from remote servers with URLs constructed from command line arguments:

```
nquire -get https://icite.od.nih.gov api/pubs -pmids 2539356 |
```

Transmute converts a concatenated stream of JSON objects or other structured formats into XML:

```
transmute -j2x -nest depth |
```

Xtract uses waypoints to navigate complex XML hierarchies, and selects data values by field name:

```
xtract -pattern data -element cited_by |
```

The resulting output can be post-processed by Unix utilities or scripts:

```
fmt -w 1 | sort -V | uniq
```

Mixed-Content Records

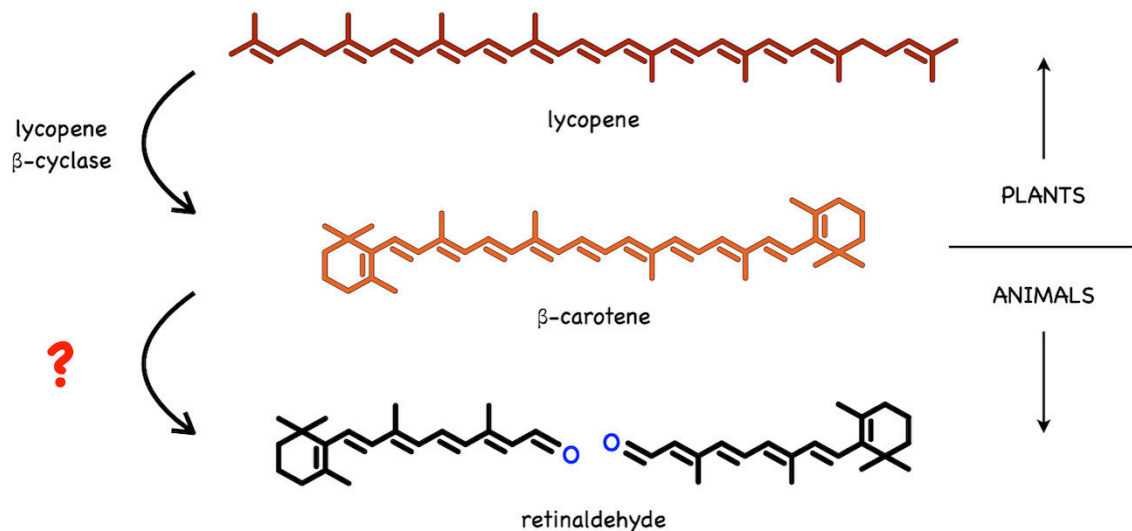
The **-mixed** flag is needed when processing XML or HTML records that contain embedded markup instructions. Using this argument, transmute **-format** can reformat the records for easier reading, and xtract **-verify** can confirm that all opening and closing tags are properly balanced:

```
efetch -db pmc -id 4305238 -format xml |
transmute -mixed -format | xtract -mixed -verify
```

Discovery by Navigation

PubMed related articles are identified by a statistical text retrieval algorithm (3) that examines the title, abstract, and medical subject headings (MeSH terms). The connections between papers can be used to help make discoveries. An example of this is finding the last enzymatic step in the vitamin A biosynthetic pathway.

Lycopene cyclase in plants converts lycopene into β -carotene, the immediate biochemical precursor of vitamin A. β -Carotene is an essential nutrient, required in the diet of herbivores. This indicates that lycopene cyclase is not present in animals (with a few exceptions caused by horizontal gene transfer from fungi to insects), and that the enzyme responsible for converting β -carotene into vitamin A is not present in plants:



An initial literature search on the lycopene cyclase enzyme:

```
esearch -db pubmed -query "lycopene cyclase" |
```

finds 334 articles. Looking up their precomputed neighbors:

```
elink -related |
```

returns 20,407 papers, some of which might be expected to discuss other enzymes in the pathway.

We cannot reliably limit the results to animals within PubMed, but we can for sequence records, which are indexed by the NCBI taxonomy. Linking the publication neighbors to their associated protein records:

```
elink -target protein -complete -log |
```

finds 2,319,795 sequences. (Using elink **-complete** will bypass the default input sampling. Otherwise it returns a fraction of that number.) Restricting those results to mice, and limiting to curated Reference Sequence records:

```
efilter -organism mouse -source refseq |
```

excludes plants, fungi, and bacteria, thus eliminating enzymes catalyzing the earlier intermediate reactions.

This matches only 15 sequences, which is small enough to examine by retrieving the individual records:

```
efetch -format fasta
```

As expected, the results include the enzyme that splits β -carotene into two molecules of retinal:

```
...
>NP_067461.2 beta,beta-carotene 15,15'-dioxygenase isoform 1 [Mus musculus]
MEIIFGQNKKEQLEPVQAKVTGSIPAWLQGTLLRNGPGMHTVGESKYNHWFDGLALLHSFSIRDGEVFYR
SKYLQSDTYIANIEANRIVVSEFGTMAYDPCKNIFSKAFSYLSHTIPDFTDNCLINIMKCGEDFYATTE
...
```

A better example used Entrez protein neighbors to instantly rediscover the similarity between a human colon cancer gene and microbial DNA repair genes. Unfortunately, precomputed BLAST links were discontinued due to the exponential growth of the sequence databases.

Searching and Filtering

Retrieving PubMed Reports

Piping PubMed query results to `efetch` and specifying the "abstract" format:

```
esearch -db pubmed -query "lycopene cyclase" |  
efetch -format abstract
```

returns a set of reports that can be read by a person:

```
...
85. PLoS One. 2013;8(3):e58144. doi: 10.1371/journal.pone.0058144. Epub ...

Levels of lycopene  $\beta$ -cyclase 1 modulate carotenoid gene expression and
accumulation in Daucus carota.

Moreno JC(1), Pizarro L, Fuentes P, Handford M, Cifuentes V, Stange C.

Author information:
(1)Departamento de Biología, Facultad de Ciencias, Universidad de Chile,
Santiago, Chile.

Plant carotenoids are synthesized and accumulated in plastids through a
highly regulated pathway. Lycopene  $\beta$ -cyclase (LCYB) is a key enzyme
involved directly in the synthesis of  $\alpha$ -carotene and  $\beta$ -carotene through
...
```

If "medline" format is used instead:

```
esearch -db pubmed -query "lycopene cyclase" |  
efetch -format medline
```

the output can be entered into common bibliographic management software packages:

```
...
PMID- 23555569
OWN - NLM
STAT- MEDLINE
DA - 20130404
DCOM- 20130930
LR - 20131121
IS - 1932-6203 (Electronic)
IS - 1932-6203 (Linking)
VI - 8
IP - 3
DP - 2013
TI - Levels of lycopene beta-cyclase 1 modulate carotenoid gene expression
and accumulation in Daucus carota.
```

```
PG - e58144
LID - 10.1371/journal.pone.0058144 [doi]
AB - Plant carotenoids are synthesized and accumulated in plastids
    through a highly regulated pathway. Lycopene beta-cyclase (LCYB) is a
    key enzyme involved directly in the synthesis of alpha-carotene and
    ...
```

Retrieving Sequence Reports

Nucleotide and protein records can be downloaded in FASTA format:

```
esearch -db protein -query "lycopene cyclase" |
efetch -format fasta
```

which consists of a definition line followed by the sequence:

```
...
>gi|735882|gb|AAA81880.1| lycopene cyclase [Arabidopsis thaliana]
MDTLLKTPNKLDFFIPQFHGFERLCSNNPYPSRVRLGVKKRAIKIVSSVSGSAALLDLPETKKENLDF
ELPLYDTSKSQVVDLAIVGGGPAGLAVAQQVSEAGLSVCSIDPSPKLIWPNNYGVWVDEFEAMDLLDCLD
TTWSGAVVYVDEGVKKDLSPYGRVNRKQLKSKMLQKCITNGVKFHQSKVTNVVHEEANSTVVCSDGVKI
QASVVLDTGFSRCLVQYDKPYNPGYQVAYGIIAEVDGHPFDVDKMFMDWRDKHLDSEPELKERNSKIP
TFLYAMPFSSNRIFLEETSLVARPGLRMEDIQERMAARLKHLLGINVKRIEEDERCVIPMGGPLPVLPQRV
VGIGGTAGMVHPSTGYMVARTLAAPIVANAIIVRYLGSPSSNSLRGDQLSAEVWRDLWPIERRRQREFFC
FGMDILLKLDLDATRRFFDAFFDLQPHYWHGFLSSRLFLPELLVFGLSLFSHASNTSRLEIMTKGTVPLA
KMINNLVQDRD
...
```

Sequence records can also be obtained as GenBank or GenPept flatfiles:

```
esearch -db protein -query "lycopene cyclase" |
efetch -format gp
```

which have features annotating particular regions of the sequence:

```
...
LOCUS      AAA81880                      501 aa          linear   PLN ...
DEFINITION lycopene cyclase [Arabidopsis thaliana].
ACCESSION  AAA81880
VERSION    AAA81880.1  GI:735882
DBSOURCE   locus ATHLYC accession L40176.1
KEYWORDS   .
SOURCE     Arabidopsis thaliana (thale cress)
  ORGANISM Arabidopsis thaliana
            Eukaryota; Viridiplantae; Streptophyta; Embryophyta;
            Tracheophyta; Spermatophyta; Magnoliophyta; eudicotyledons;
            Brassicales; Brassicaceae; Camelineae; Arabidopsis.
REFERENCE  1  (residues 1 to 501)
  AUTHORS  Scolnik,P.A. and Bartley,G.E.
  TITLE    Nucleotide sequence of lycopene cyclase (GenBank L40176) from
            Arabidopsis (PGR95-019)
  JOURNAL  Plant Physiol. 108 (3), 1343 (1995)
...
FEATURES             Location/Qualifiers
     source            1..501
                       /organism="Arabidopsis thaliana"
                       /db_xref="taxon:3702"
     Protein           1..501
                       /product="lycopene cyclase"
     transit_peptide   1..80
```

```

mat_peptide      81..501
                  /product="lycopene cyclase"
CDS              1..501
                  /gene="LYC"
                  /coded_by="L40176.1:2..1507"
ORIGIN
      1 mdtllktpnk ldffipqfhg ferlcsnnp py srvr lgvkk raikivssv sg saalldlv
     61 petkkenldf elplydtsks qvvdlaivgg gpaglavaqq vseaglsvcs idpskliwp
    121 nnygvwvdef eamdllldcl dtwsgavvyv degvkkdlr pygrvnrql kskmlqkcit
    181 ngvkfhqskv tnvvheeans tvvcsgdgvki qasvldatg fsrclvqydk pynpgyqvay
    241 giiaevdghp fdvdkmvfmd wrdkhldsyp elkernskip tflyampfss nrifleetsl
    301 varpglrmed iqermaarkl hlginvkrie edercvipmg gplpvlpqr vvgiggtagmv
    361 hpstgymvar tlaapivan aivrylgsp ssnlrgdqls aevwrdlwp errrqrfffc
    421 fgmdillkld ldatrrffda ffdlqphywh gflssrlflp ellvfglslf shasntsrl
    481 imtkgtvpla kminnlvqdr d
//
...

```

Restricting Query Results

The current results can be refined by further term searching in Entrez:

```

esearch -db pubmed -query "opsin gene conversion" |
elink -related |
efilter -query "tetrachromacy"

```

Limiting by Date

Results can also be filtered by date. For example, the following statements:

```

efilter -days 60 -datatype PDAT

efilter -mindate 2000

efilter -maxdate 1985

efilter -mindate 1990 -maxdate 1999

```

restrict results to articles published in the previous two months, since the beginning of 2000, through the end of 1985, or in the 1990s, respectively. YYYY/MM and YYYY/MM/DD date formats are also accepted.

Fetch by Identifier

Efetch and elink can take a list of numeric identifiers or accessions in an **-id** argument:

```

efetch -db pubmed -id 7252148,1937004 -format xml

efetch -db nuccore -id 1121073309 -format acc

efetch -db protein -id 3QZ_a -format fasta

efetch -db bioproject -id PRJNA257197 -format docsum

efetch -db pmc -id PMC209839 -format medline

elink -db pubmed -id 2539356 -cites

```

without the need for a preceding esearch command.

Non-integer accessions will be looked up with an internal search, using the appropriate field for the database:

```
esearch -db bioproject -query "PRJNA257197 [PRJA]" |
efetch -format uid |
...
```

Most databases use the [ACCN] field for identifier lookup, but there are a few exceptions:

annotinfo	[ASAC]
assembly	[ASAC]
bioproject	[PRJA]
books	[AID]
clinvar	[VACC]
gds	[ALL]
genome	[PRJA]
geoprofiles	[NAME]
gtr	[GTRACC]
mesh	[MHUI]
nuccore	[ACCN] or [PACC]
pcsubstance	[SRID]
snp	[RS] or [SS]

(For `-db pmc` it merely removes any "PMC" prefix from the integer identifier.)

For backward compatibility, **esummary** is a shortcut for `esearch -format docsum`:

```
esummary -db bioproject -id PRJNA257197

esummary -db sra -id SRR5437876
```

Reading Large Lists of Identifiers

Efetch and elink can also read a large list of identifiers or accessions piped in through stdin:

```
cat "file_of_identifiers.txt" |
efetch -db pubmed -format docsum
```

or from a file indicated by an `-input` argument:

```
efetch -input "file_of_identifiers.txt" -db pubmed -format docsum
```

As mentioned above, there is no need to use a script to split the identifiers into smaller groups or add a time delay between individual requests, since that functionality is already built into EDirect.

Processing Groups of Terms

The **join-into-groups-of** script combines text containing words, numbers, or identifiers into lines of comma-separated items:

```
join-into-groups-of 20 |
```

Splitting text into one element per line is done by piping through **word-at-a-time** for prose or **accn-at-a-time** for accessions. These functions also lower-case the output:

```
echo "$abstract_in_variable" |
word-at-a-time |
while read term
do
  echo "$term"
done |
```

The **sort-uniq-count-rank** function then sorts the individual terms, counts the number of times each one appears in the list, and finally resorts the terms based on their frequency of occurrence:

sort-uniq-count-rank

A faster version of **accn-at-a-time** runs without lower-case transformation:

```
tr -cs a-zA-Z0-9_ . '\n' |
```

Qualifying Queries by Indexed Field

Query terms in **esearch** or **efilter** can be qualified by entering an indexed field abbreviation in brackets. Boolean operators and parentheses can also be used in the query expression for more complex searches.

Commonly used fields for PubMed queries include:

[AFFL]	Affiliation	[LANG]	Language
[ALL]	All Fields	[MAJR]	MeSH Major Topic
[AUTH]	Author	[SUBH]	MeSH Subheading
[FAUT]	Author - First	[MESH]	MeSH Terms
[LAUT]	Author - Last	[PTYP]	Publication Type
[CRDT]	Date - Create	[WORD]	Text Word
[PDAT]	Date - Publication	[TITL]	Title
[FILT]	Filter	[TIAB]	Title/Abstract
[JOUR]	Journal	[UID]	UID

and a qualified query looks like:

```
"Tager HS [AUTH] AND glucagon [TIAB]"
```

Filters that limit search results to subsets of PubMed include:

```
humans [MESH]
pharmacokinetics [MESH]
chemically induced [SUBH]
all child [FILT]
english [FILT]
freetext [FILT]
has abstract [FILT]
historical article [FILT]
randomized controlled trial [FILT]
clinical trial, phase ii [PTYP]
review [PTYP]
```

Sequence databases are indexed with a different set of search fields, including:

[ACCN]	Accession	[MLWT]	Molecular Weight
[ALL]	All Fields	[ORGN]	Organism
[AUTH]	Author	[PACC]	Primary Accession
[GPRJ]	BioProject	[PROP]	Properties
[BIOS]	BioSample	[PROT]	Protein Name
[ECNO]	EC/RN Number	[SQID]	SeqID String
[FKEY]	Feature key	[SLEN]	Sequence Length
[FILT]	Filter	[SUBS]	Substance Name
[GENE]	Gene Name	[WORD]	Text Word
[JOUR]	Journal	[TITL]	Title
[KYWD]	Keyword	[UID]	UID

and a sample query in the protein database is:

```
"alcohol dehydrogenase [PROT] NOT (bacteria [ORGN] OR fungi [ORGN])"
```

Additional examples of subset filters in sequence databases are:

```
mammalia [ORGN]
mammalia [ORGN:noexp]
txid40674 [ORGN]
cds [FKEY]
lacZ [GENE]
beta galactosidase [PROT]
protein snp [FILT]
reviewed [FILT]
country united kingdom glasgow [TEXT]
country united states [TEXT]
biomol genomic [PROP]
dbxref flybase [TEXT]
gbdiv phg [PROP]
phylogenetic study [PROP]
sequence from mitochondrion [PROP]
src cultivar [PROP]
srcdb refseq validated [PROP]
150:200 [SLEN]
```

(The calculated molecular weight (MLWT) field is only indexed for proteins and structures, not nucleotides.)

See **efilter -help** for a list of filter shortcuts available for several Entrez databases.

Examining Intermediate Results

EDirect navigation functions produce a custom XML message with the relevant fields (database, web environment, query key, and record count) that can be read by the next command in the pipeline. EDirect may store intermediate results on the Entrez history server or instantiate them in the XML message.

The results of each step in a query can be examined to confirm expected behavior before adding the next step. The Count field in the ENTREZ_DIRECT object contains the number of records returned by the previous step. A good measure of query success is a reasonable (non-zero) count value. For example:

```
esearch -db protein -query "tryptophan synthase alpha chain [PROT]" |
efilter -query "28000:30000 [MLWT]" | tee /dev/tty |
elink -target structure -log |
efilter -query "0:2 [RESO]"
```

produces:

```
<ENTREZ_DIRECT>
  <Db>protein</Db>
  <WebEnv>MCID_6a56bdcecaf7c3511b0de8e1</WebEnv>
  <QueryKey>2</QueryKey>
  <Count>17617</Count>
  <Step>2</Step>
  <Elapsed>1</Elapsed>
</ENTREZ_DIRECT>
<ENTREZ_DIRECT>
  <Db>structure</Db>
  <Count>57</Count>
  <Step>4</Step>
  <Id>5822</Id>
  <Id>11701</Id>
  ...
  <Id>211309</Id>
  <Id>255950</Id>
```

```
<Elapsed>203</Elapsed>
</ENTREZ_DIRECT>
```

The first message shows the molecular weight range filter returning 17,617 proteins. (Piping through **tee /dev/tty** prints the intermediate result on the terminal and also pipes it to the next step.) The second message shows the final result of 57 protein structures having the desired (X-ray crystallographic) atomic position resolution.

(When saving to history, the QueryKey value may differ from Step because the elink command splits its query into smaller chunks to avoid server truncation limits and timeout errors.)

Combining Independent Queries

Independent esearch/elink/efilter/efetch pipelines can be combined by passing the separate UID results to the **intersect-uid-lists** script. (The output of both pipelines must be in the same target database.)

(The history server's "#" convention is no longer used for this example. Intermediate results are now saved in Unix shell script variables, which will be discussed in a later section of this document.)

For example, the query:

```
amyl=$(
  esearch -db protein -query "amyloid* [PROT]" -log | tee /dev/tty |
  elink -target pubmed |
  efetch -format uid
)

apoe=$(
  esearch -db gene -query "apo* [GENE]" -log | tee /dev/tty |
  elink -target pubmed |
  efetch -format uid
)

both=$( intersect-uid-lists <( echo "$amyl" ) <( echo "$apoe" ) )

echo "$both" |
efetch -db pubmed -format docsum |
xtract -mixed -pattern DocumentSummary -element Id Title |
cat -v
```

uses truncation searching (entering the first few letters of a word followed by an asterisk) for the two main concepts, and return titles of papers with links to both amyloid protein sequences and apolipoprotein gene records:

```
23962925    Genome analysis reveals insights into physiology and ...
23959870    Low levels of copper disrupt brain amyloid-β homeostasis ...
23371554    Genomic diversity and evolution of the head crest in the ...
23251661    Novel genetic loci identified for the pathophysiology of ...
...
```

XML Data Extraction

The ability to obtain Entrez records in structured format, and to easily extract the underlying data, allows the user to ask novel questions that are not addressed by existing analysis software.

The xtract program uses command-line arguments to direct the conversion of data in eXtensible Markup Language (XML) format. It allows record detection, path exploration, conditional processing, element selection, substring isolation, and report formatting to be controlled independently.

The advantage of XML is that information is in specific locations in a well-defined data hierarchy. Accessing individual units of data that are fielded by name, such as:

```
<PubDate>2013</PubDate>
<Source>PLoS One</Source>
<Volume>8</Volume>
<Issue>3</Issue>
<Pages>e58144</Pages>
```

requires matching the same general pattern, differing only by the element name. This is much simpler than parsing the units from a long, complex string:

```
1. PLoS One. 2013;8(3):e58144 ...
```

The disadvantage of XML is that data extraction usually requires custom programming. But EDirect relies on the common pattern of XML value representation to provide a simplified approach to interpreting XML data.

The **-pattern** (or **-record**) command partitions an XML stream by object name into individual records that are processed separately. Within each record, the **-element** command does an exhaustive, depth-first search to find data content by field name. For example:

```
xtract -pattern ENTREZ_DIRECT -element Count
```

Neither explicit object paths nor complicated path formulas are needed for element identification.

Format Customization

By default, the **-pattern** argument divides the results into rows, while placement of data into columns is controlled by **-element**, to create a tab-delimited table.

Formatting commands allow extensive customization of the output. The line break between **-pattern** rows is changed with **-ret**, while the tab character between **-element** columns is modified by **-tab**.

Multiple instances of the same element are distinguished using **-sep**, which controls their separation independently of the **-tab** command. The following query:

```
efetch -db pubmed -id 6271474,24178092 -format docsum |
xtract -pattern DocumentSummary -sep "|" -element Id PubDate Name
```

returns a tab-delimited table with individual author names separated by vertical bars:

```
6271474      1981      Casadaban MJ|Chou J|Lemaux P|Tu CP|Cohen SN
24178092     1994 Dec   Garber ED|Ruddat M
```

The **-sep** value also applies to distinct **-element** arguments that are grouped with **commas**. This can be used to keep data from multiple related fields in the same column:

```
-sep " " -element Initials,LastName
```

Groups of fields are preceded by the **-pfx** value and followed by the **-sfx** value, both of which are initially empty.

The **-def** command sets a default placeholder to be printed when none of the comma-separated fields in an **-element** clause are present:

```
-def "-" -sep " " -element Year,Month,MedlineDate
```

Limit by Parent

An `-element` argument can use the **parent** / **child** construct to limit selection when items can only be disambiguated by position, not by name. In this case, it prevents the display of additional PMIDs that might be present inside `CommentsCorrections` objects deeper within the `MedlineCitation` container:

```
xtract -pattern PubmedArticle -element MedlineCitation/PMID
```

Element Variants

Derivatives of `-element` were initially created to avoid having to write post-processing scripts just to perform trivial modifications or integer calculations on extracted data. Other variants were added for content normalization, report formatting, or index generation. The commands are in several categories:

Positional:	-first, -last, -even, -odd, -backward
Numeric:	-num, -len, -inc, -dec, -mod, -bin, -hex, -bit
Statistics:	-sum, -acc, -min, -max, -dev, -med
Averages:	-avg, -geo, -hrm, -rms
Logarithms:	-sqt, -lge, -lg2, -log
Character:	-encode, -upper, -title, -mirror, -alpha, -alnum
String:	-basic, -plain, -simple, -prose
Text:	-words, -pairs, -letters, -split, -order, -reverse, -pentamers
Citation:	-year, -month, -date, -auth, -initials, -page, -author, -journal
Sequence:	-revcomp, -fasta, -ncbi2na, -molwt, -pept, -nucleic
Translation:	-cds2prot, -gcode, -frame
Coordinate:	-0-based, -1-based, -ucsc-based
Variation:	-hgvs
Frequency:	-histogram
Expression:	-reg, -exp, -replace
Substitution:	-transform, -translate
Indexing:	-indexer, -aliases, -classify
Miscellaneous:	-doi, -wct, -trim, -pad, -mask, -accession, -numeric

The original `-element` prefix shortcuts, `"#"` and `"%"`, are redirected to `-num` and `-len`, respectively.

See `xtract -help` for a brief description of each command.

Substring Extraction

Following an element name with square brackets allows subset selection by numeric range (with a colon between the character start and stop positions), or permits removal of leading and trailing text (with a vertical bar [or caret] separating the required [or optional] interior prefix and suffix endpoints):

```
-author Initials[1:1] -numeric "ArticleId[PMC|]" -upper "Accession[^.]"
```

XML Repackaging

Repackaging commands (**-wrp**, **-enc**, and **-pkg**) wrap extracted data values with bracketed XML tags given only the object name. For example, "**-wrp Word**" issues the following formatting instructions:

```
-pfx "<Word>" -sep "</Word><Word>" -sfx "</Word>"
```

It also sets an internal flag to ensure that data values containing encoded symbols (ampersands, angle brackets, apostrophes, and quotation marks) remain properly encoded inside the new XML.

Combining **-wrp** commands and **-element** variants can break up complex tasks into a series of simpler operations. No custom reading or writing code is needed, since the data remains in XML format.

Generating Attributes

Additional commands (**-tag**, **-att**, **-atr**, **-cls**, **-slf**, and **-end**) allow generation of XML tags with attributes. The following will produce regular and self-closing XML objects, respectively:

```
-tag Item -att type journal -cls -element Source -end Item
```

```
<Item type="journal">J Bacteriol</Item>
```

```
-tag Item -att type journal -atr name Source -slf
```

```
<Item type="journal" name="J Bacteriol" />
```

Exploration Control

Exploration commands control the order in which XML record contents are examined, by separately presenting each instance of the chosen subregion. This limits what subsequent commands "see" at any one time, and can allow related fields in an object to be kept together in the output.

Unlike the simpler DocumentSummary format, records retrieved as PubmedArticle XML:

```
efetch -db pubmed -id 1413997 -format xml |
```

have authors with separate fields for last name and initials:

```
<Author>
  <LastName>Mortimer</LastName>
  <Initials>RK</Initials>
</Author>
```

Without being given any guidance about context, an **-element** command on initials and last names:

```
xtract -pattern PubmedArticle -element Initials LastName
```

will explore the current record for each argument in turn, printing all initials followed by all last names:

```
RK    CR    JS    Mortimer    Contopoulou    King
```

Inserting a **-block** command adds a new exploration layer between **-pattern** and **-element**, which redirects data exploration to present the authors one at a time:

```
xtract -pattern PubmedArticle -block Author -element Initials LastName
```

Each time through the loop, the **-element** command only sees the current author's values. This restores the correct association of initials and last names in the output:

```
RK    Mortimer    CR    Contopoulou    JS    King
```

Grouping the two author subfields with a comma, and adjusting the **-sep** and **-tab** values:

```
xtract -pattern PubmedArticle -block Author \  
-sep " " -tab ", " -element Initials,LastName
```

produces a more traditional formatting of author names:

```
RK Mortimer, CR Contopoulou, JS King
```

Sequential Exploration

Multiple **-block** statements can be used in a single **xtract** to explore different areas of the XML. This limits element extraction to the desired subregions, and allows disambiguation of fields with identical names but with different parents. For example:

```
efetch -db pubmed -id 6092233,4640931,4296474 -format xml |  
xtract -pattern PubmedArticle -element MedlineCitation/PMID \  
-block PubDate -sep " " -element Year,Month,MedlineDate \  
-block AuthorList -num Author -sep "/" -element LastName |  
sort-table -k 3,3n -k 4,4f
```

generates a table that allows easy parsing of author last names, and sorts the results by author count:

4296474	1968 Apr	1	Friedmann
4640931	1972 Dec	2	Tager/Steiner
6092233	1984 Jul-Aug	3	Calderon/Contopoulou/Mortimer

Like **-element** arguments, the individual **-block** statements are executed sequentially, in order of appearance.

Note also that the **PubDate** object can exist either in a structured form:

```
<PubDate>  
  <Year>1968</Year>  
  <Month>Apr</Month>  
  <Day>25</Day>  
</PubDate>
```

(with the **Day** field frequently absent), or in a string form:

```
<PubDate>  
  <MedlineDate>1984 Jul-Aug</MedlineDate>  
</PubDate>
```

but would not contain a mixture of both types, so the directive:

```
-sep " " -element Year,Month,MedlineDate
```

will only contribute a single column to the output.

Nested Exploration

Exploration command names (**-group**, **-block**, and **-subset**) are assigned to a precedence hierarchy:

```
-pattern > -group > -block > -subset > -element
```

and are combined in ranked order to control object iteration at progressively deeper levels in the XML data structure. Each command argument acts as a nested "for loop" control variable, retaining information about the context, or state of exploration, at its level.

(Hypothetical) census data would need several nested loops to visit each unique address in context:

```
-pattern State -group City -block Street -subset Number -element Resident
```

A nucleotide or protein sequence record can have multiple features. Each feature can have multiple qualifiers. And every qualifier has separate name and value nodes. Exploring this natural data hierarchy, with **-pattern** for the sequence record, **-group** for the feature, and **-block** for the qualifier:

```
efetch -db nuccore -id NM_021486.4 -format gbc |
xtract -pattern INSDSeq -element INSDSeq_accession-version \
-group INSDFeature -deq "\n\t" -element INSDFeature_key \
-block INSDQualifier -deq "\n\t\t" \
-element INSDQualifier_name INSDQualifier_value
```

keeps qualifiers, such as gene and product, associated with their parent features, and keeps qualifier names and values together on the same line:

```
NM_021486.4
source
    organism      Mus musculus
    mol_type      mRNA
gene
    gene          Bco1
CDS
    gene          Bco1
    product       beta,beta-carotene 15,15'-dioxygenase isoform 1
    protein_id    NP_067461.2
    translation   MEIIFGQNKKEQLEPVQAKVTGSIPAWLQGTLLRNGPGM ...
...
```

Saving Data in Variables

A value can be recorded in a variable and used wherever needed. Variables are created by a hyphen followed by a name consisting of a string of capital letters or digits (e.g., **-KEY**). Variable values are retrieved by placing an ampersand before the variable name (e.g., **"&KEY"**) in an **-element** statement:

```
efetch -db nuccore -id NM_021486.4 -format gbc |
xtract -pattern INSDSeq -element INSDSeq_accession-version \
-group INSDFeature -KEY INSDFeature_key \
-block INSDQualifier -deq "\n\t" \
-element "&KEY" INSDQualifier_name INSDQualifier_value
```

This prints the feature key on each line before the qualifier name and value, even though the feature key is now outside of the visibility scope (which is the current qualifier):

```
NM_021486.4
source      organism      Mus musculus
source      mol_type      mRNA
gene        gene          Bco1
```

```

CDS      gene      Bco1
CDS      product   beta,beta-carotene 15,15'-dioxygenase isoform 1
CDS      protein_id NP_067461.2
CDS      translation MEIIFGQNKKEQLEPVQAKVTGSIPAWLQGTLLRNGPGM ...
...
```

Variables can be (re)initialized with an explicit literal value inside parentheses:

```
-block Author -sep " " -tab "" -element "&COM" Initials,LastName -COM "(, )"
```

They can also be used as the first argument in a conditional statement:

```
-CHR Chromosome -block GenomicInfoType -if "&CHR" -differs-from ChrLoc
```

A double-hyphen (e.g., **--KYWDS**) appends a value to the variable.

A variable can also save the modified data resulting from an **-element** variant operation. This can allow multiple sequential transitions within a single **xtract** command:

```
-END -sum "Start,Length" -MID -avg "Start,&END"
```

All variables are reset when the next record is processed.

Conditional Execution

Conditional processing commands (**-if** and **-unless**) restrict object exploration by data content. They check to see if the named field is within the scope, and may be used in conjunction with string, numeric, or object constraints to require an additional match by value. Use **-and** and **-or** to build compound tests. Streaming input through **-select** removes records that do not satisfy the condition:

```

esearch -db pubmed -query "Havran W [AUTH]" |
efetch -format xml |
xtract -pattern PubmedArticle -select Language -equals eng |
xtract -pattern PubmedArticle -block Author -if LastName -is-not Havran \
  -sep ", " -tab "\n" -author LastName,Initials[1:1] |
sort-uniq-count-rank
```

This limits the results to papers written in English and prints a table of the most frequent collaborators, using a range to keep only the first initial so that variants like "Berg, C" and "Berg, CM" are combined:

```

35   Witherden, D
15   Boismenu, R
12   Jameson, J
10   Allison, J
10   Fitch, F
...
```

Numeric constraints can compare the integer values of two fields. This can be used to find genes that are encoded on the minus strand of a particular chromosome:

```
-if ChrLoc -equals X -and ChrStart -gt ChrStop
```

Object constraints will compare the string values of two named fields, and can look for internal inconsistencies between fields whose contents should (in most cases) be identical:

```
-if Chromosome -differs-from ChrLoc
```

The **-position** command restricts presentation of objects by relative location or index number:

```
-block Author -position last -sep ", " -element LastName,Initials
```

The **-else** command can run alternative **-element** or **-lbl** instructions if the condition is not satisfied:

```
-if Strand -contains "-" -lbl "minus strand" -else -lbl "plus strand"
```

Parallel **-if** and **-unless** statements can be used to provide a more complex response to alternative conditions that include nested explorations.

Copying XML Objects

The **-element "*" construct prints the entirety of the current XML container, including all XML tags. (A period or a percent sign writes ASN.1 or JSON, respectively. Tag names with leading, trailing, or internal underscores control item-specific formats, such as unquoted values or unnamed brackets.)**

Automatic Format Conversion

Xtract can now detect and convert input data in JSON, text ASN.1, and GenBank/GenPept flatfile formats, into XML. Explicit transmute or shortcut commands are only needed to view the intermediate XML's field names or override the default conversion settings.

Advanced Operations

Text Modification

For custom editing, target and replacement strings are set with **-reg** and **-exp**, respectively. Regular expression text matching and substitution is then performed on an element with a **-replace** command:

```
xtract ... -reg "-" -exp "." -replace Phone
```

Value Substitution

External values can be loaded by reading a two-column precomputed file or ad hoc conversion table with **-transform**. Lookup is then requested by applying **-translate** to an element:

```
xtract -transform accn-to-uid.txt ... -translate Accession
```

```
xtract -transform <( echo -e "Genomic\t1\nCoding\t2\nProtein\t3\n" ) ...
```

Multi-Step Transformations

Although xtract provides **-element** derivatives to do simple data manipulation, more complex tasks may be broken up into a series of simpler transformations, or "processing chains".

BioSample document summaries:

```
efetch -db biosample -id SAMN38051082 -format docsum |
```

store preferred qualifier names in a "harmonized_name" XML attribute:

```
<Attribute harmonized_name="strain">BALB/c</Attribute>
<Attribute harmonized_name="isolate">Mtb infected Spleen MZB-2</Attribute>
<Attribute harmonized_name="geo_loc_name">Singapore</Attribute>
```

Piping the data to a first xtract command, and using the "@" sign to select the attribute:

```
xtract -rec BioSampleInfo -pattern DocumentSummary \
-wrp Accession -element Accession \
-group Attribute -if @harmonized_name \
-TAG -lower @harmonized_name -wrp "&TAG" -element Attribute |
```

generates an intermediate form, with XML tag names taken from the original XML attributes:

```
<BioSampleInfo>
  <Accession>SAMN38051082</Accession>
  <strain>BALB/c</strain>
  <isolate>Mtb infected Spleen MZB-2</isolate>
  <geo_loc_name>Singapore</geo_loc_name>
  ...
```

Desired fields can then be selected by name in a second xtract command:

```
xtract -pattern BioSampleInfo -def "-" -first Accession \
      geo_loc_name strain isolate
```

Parsing XML Elements

GFF3 can be normalized with **gff-sort**, and then converted to XML with **tbl2xml**, which gets field names from the remaining arguments (or, with **-header**, from the first row of the input data):

```
tbl2xml -rec Rec SeqID Source Type Start End Score Strand Phase Attributes |
```

The Attributes element consists of multiple clauses that are delimited by semicolons:

```
<Attributes>ID=gene-XXXX_016554;Name=XXXX_016554; ... </Attributes>
```

Piping to xtract creates a container, and uses **-with** and **-split** to parse the Attribute field:

```
xtract -pattern Rec -pkg Context -wrp Item -with ";" -split Attributes |
```

into individual "**tag = value**" elements within the new package:

```
<Context>
  <Item>ID=gene-XXXX_016554</Item>
  <Item>Name=XXXX_016554</Item>
  ...
```

A second xtract can now use element suffix and prefix removal, respectively, to isolate the tag and value. The tag is saved in a variable to become the name of a new XML object in which the value is stored:

```
xtract -pattern Context -pkg Fields -block Item \
      -TAG "Item[=]" -wrp "&TAG" -element "Item[=]" |
```

The final XML can be queried by field names that were extracted from the original Attributes clauses:

```
<Fields>
  <ID>gene-XXXX_016554</ID>
  <Name>XXXX_016554</Name>
  ...
```

XML Namespaces

Namespace prefixes are followed by a colon, while a leading colon matches any prefix:

```
nquire -url https://webservice.wikipathways.org getPathway -pwId WP455 |
xtract -pattern "ns1:getPathwayResponse" -decode ":gpml" |
```

The embedded Graphical Pathway Markup Language object can then be processed:

```
xtract -pattern Pathway -block Xref \
      -if @Database -equals "Entrez Gene" -tab "\n" -element @ID
```

Record Filtering

The **filter-records** script can extract an arbitrary range of text records using **-min** and **-max**:

```
filter-records -pattern "<PubmedArticle>" -min 13 -max 24
```

It also has arguments to **-require** or **-exclude** specific text content.

Sorting or Partitioning Sets of Records

To sort XML records by a particular field, pass the element name to **xtract -sort-fwd** or **-sort-rev**. Stable sorting, which preserves the relative order of records with identical sort key values, is provided by the **-stable-sort-fwd** and **-stable-sort-rev** commands:

```
xtract -pattern PubmedArticle -sort-rev MedlineCitation/PMID
```

To partition XML records, give the number of records per output file to **xtract -split-by-num**:

```
xtract -set PubmedArticleSet -pattern PubmedArticle \  
-split-by-num 100 -prefix subset -suffix xml
```

or use the maximum byte size for each output file as the argument to **xtract -split-by-len**:

```
xtract -set PubmedArticleSet -pattern PubmedArticle \  
-split-by-len 1000000 -prefix subset -suffix xml
```

or make individual files by passing the record identifier's element name to **xtract -split-by-id**:

```
xtract -mixed -set pmc-articleset -pattern article \  
-split-by-id article-id -prefix PMC -suffix xml
```

Numbered output file names will be of the form "subset001.xml", "subset002.xml", etc.

To distribute text records into separate files, use **split-records** with **-by-num** or **-by-len**.

Viewing an XML Hierarchy

Piping a PubmedArticle XML object to **xtract -outline**:

```
esearch -db pubmed -query "Cozzarelli NR [AUTH] AND Casadaban MJ [AUTH]" |  
efetch -format xml | xtract -outline 3
```

will give an indented overview of the XML hierarchy, optionally limited to a specified object depth:

```
PubmedArticle
  MedlineCitation
    PMID
    DateCompleted
    DateRevised
    Article
    MedlineJournalInfo
    ChemicalList
    CitationSubset
    MeshHeadingList
  PubmedData
  ...
```

Using **xtract -synopsis** or **-contour** will show the full paths to all nodes or just the terminal (leaf) nodes, respectively. Piping those results to **sort-uniq-count** will produce a table of unique paths.

Use a **caret** ("^") as an element argument prefix to print its absolute depth in the XML hierarchy:

```
xtract -pattern PubmedArticle -element "^PMID"
```

Code Nesting Comparison

Sketching with indented pseudo code can clarify relative nesting levels. The extraction command:

```
xtract -pattern PubmedArticle \
  -block Author -element Initials,LastName \
  -block MeshHeading \
    -if QualifierName \
      -element DescriptorName \
      -subset QualifierName -element QualifierName
```

where the rank of the argument name controls the nesting depth, could be represented as a computer program in pseudo code by:

```
for pat = each PubmedArticle {
  for blk = each pat.Author {
    print blk.Initials blk.LastName
  }
  for blk = each pat.MeSHTerm {
    if blk.Qual is present {
      print blk.MeshName
      for sbs = each blk.Qual {
        print sbs.QualName
      }
    }
  }
}
```

where the brace indentation count controls the nesting depth.

Extra arguments are held in reserve to provide additional levels of organization, should the need arise in the future for processing complex, deeply-nested XML data. The full set of exploration commands below `-pattern`, in order of rank, are:

```
-path
  -division
    -group
      -branch
        -block
          -section
            -subset
              -unit
```

Starting `xtract` exploration with `-block`, and expanding with `-group` and `-subset`, leaves additional level names that can be used wherever needed without having to redesign the entire command.

Complex Objects

Author Exploration

What's in a name? That which we call an author, by any other name, may be a consortium, investigator, or editor:

```
<PubmedArticle>
  <MedlineCitation>
    <PMID>99999999</PMID>
    <Article>
      <AuthorList>
```

```

    <Author>
      <LastName>Tinker</LastName>
    </Author>
    <Author>
      <LastName>Evers</LastName>
    </Author>
    <Author>
      <LastName>Chance</LastName>
    </Author>
    <Author>
      <CollectiveName>FlyBase Consortium</CollectiveName>
    </Author>
  </AuthorList>
</Article>
<InvestigatorList>
  <Investigator>
    <LastName>Alpher</LastName>
  </Investigator>
  <Investigator>
    <LastName>Bethe</LastName>
  </Investigator>
  <Investigator>
    <LastName>Gamow</LastName>
  </Investigator>
</InvestigatorList>
</MedlineCitation>
</PubmedArticle>

```

Within the record, -element exploration on last name:

```
xtract -pattern PubmedArticle -element LastName
```

prints each last name, but does not match the consortium:

```
Tinker    Evers    Chance    Alpher    Bethe    Gamow
```

Limiting to the author list:

```
xtract -pattern PubmedArticle -block AuthorList -element LastName
```

excludes the investigators:

```
Tinker    Evers    Chance
```

Using -num on each type of object:

```
xtract -pattern PubmedArticle -num Author Investigator LastName CollectiveName
```

displays the various object counts:

```
4      3      6      1
```

Date Selection

Dates come in all shapes and sizes:

```

<PubmedArticle>
  <MedlineCitation>
    <PMID>99999999</PMID>
    <DateCompleted>
      <Year>2011</Year>
    </DateCompleted>
  </MedlineCitation>
</PubmedArticle>

```

```

<DateRevised>
  <Year>2012</Year>
</DateRevised>
<Article>
  <Journal>
    <JournalIssue>
      <PubDate>
        <Year>2013</Year>
      </PubDate>
    </JournalIssue>
  </Journal>
  <ArticleDate>
    <Year>2014</Year>
  </ArticleDate>
</Article>
</MedlineCitation>
<PubmedData>
  <History>
    <PubMedPubDate PubStatus="received">
      <Year>2015</Year>
    </PubMedPubDate>
    <PubMedPubDate PubStatus="accepted">
      <Year>2016</Year>
    </PubMedPubDate>
    <PubMedPubDate PubStatus="entrez">
      <Year>2017</Year>
    </PubMedPubDate>
    <PubMedPubDate PubStatus="pubmed">
      <Year>2018</Year>
    </PubMedPubDate>
    <PubMedPubDate PubStatus="medline">
      <Year>2019</Year>
    </PubMedPubDate>
  </History>
</PubmedData>
</PubmedArticle>

```

Within the record, -element exploration on the year:

```
xtract -pattern PubmedArticle -element Year
```

finds and prints all nine instances:

```
2011    2012    2013    2014    2015    2016    2017    2018    2019
```

Using -block to limit the scope:

```
xtract -pattern PubmedArticle -block History -element Year
```

prints only the five years within the History object:

```
2015    2016    2017    2018    2019
```

Surprisingly, inserting a conditional statement to limit element selection to a date with a specific attribute:

```
xtract -pattern PubmedArticle -block History \
-if @PubStatus -equals "pubmed" -element Year
```

still prints all five years within History:

```
2015    2016    2017    2018    2019
```

This is because the `-if` command uses the same exploration logic as `-element`, but is designed to declare success if it finds a match **anywhere** within the current scope. There is indeed a "pubmed" attribute within History, in one of its five PubMedPubDate child objects, so the test succeeds. Thus, `-element` is given free rein to do its own exploration within History, and prints all five years.

The solution is to explore the individual PubMedPubDate objects within History:

```
xtract -pattern PubmedArticle -block History \
  -subset PubMedPubDate \
    -if @PubStatus -equals "pubmed" -element Year
```

This visits each PubMedPubDate separately, with the `-if` test matching only the indicated date type, thus returning only the desired year:

```
2018
```

PMID Extraction

Because of the presence of a CommentsCorrections object:

```
<PubmedArticle>
  <MedlineCitation>
    <PMID>99999999</PMID>
    <CommentsCorrectionsList>
      <CommentsCorrections RefType="ErratumFor">
        <PMID>88888888</PMID>
      </CommentsCorrections>
    </CommentsCorrectionsList>
  </MedlineCitation>
</PubmedArticle>
```

attempting to print the record's PubMed Identifier:

```
xtract -pattern PubmedArticle -element PMID
```

also returns the PMID of the comment:

```
99999999      88888888
```

Using an exploration command cannot exclude the second instance, because it would need a parent node unique to the first element, and the chain of parents to the first PMID:

```
PubmedArticle/MedlineCitation
```

is a subset of the chain of parents to the second PMID:

```
PubmedArticle/MedlineCitation/CommentsCorrectionList/CommentsCorrections
```

Although **-first** PMID will work in this particular case, the more general solution is to limit by subpath with the parent / child construct:

```
xtract -pattern PubmedArticle -element MedlineCitation/PMID
```

That would work even if the order of objects were reversed.

Expanding Horizons

The nquire program uses command-line arguments to obtain data from external RESTful, CGI, or FTP servers. (Xtract can read JSON, ASN.1, and GenBank formats directly, but previously-required conversion commands - now for inspecting XML or overriding defaults - are shown below in light text.)

JSON Arrays

Human β -globin information from a Scripps Research data integration project (4):

```
nquire -get https://mygene.info/v3 gene 3043 | json2xml |
```

contains a multi-dimensional JavaScript Object Notation array of exon coordinates:

```
"position": [
  [ 5225463, 5225726 ],
  [ 5226576, 5226799 ],
  [ 5226929, 5227071 ]
],
"strand": -1,
```

Conversion to XML assigns distinct tag names to each level with the json2xml **-nest element** default:

```
<position>
  <position_E>5225463</position_E>
  <position_E>5225726</position_E>
</position>
...
```

Heterogeneous Data

A query for the human green-sensitive opsin gene:

```
nquire -get https://mygene.info/v3/gene/2652 | json2xml |
```

returns data containing a heterogeneous mixture of objects in the pathway section:

```
<pathway>
  <reactome>
    <id>R-HSA-162582</id>
    <name>Signal Transduction</name>
  </reactome>
  ...
  <wikipathways>
    <id>WP455</id>
    <name>GPCRs, Class A Rhodopsin-like</name>
  </wikipathways>
</pathway>
```

The **parent / star** construct is used to visit the individual components of a parent object without needing to explicitly specify their names. For printing, the name of a child object is indicated by a **question mark** ("?"), while a **tilde** ("~") would return the anonymous object's value:

```
xtract -pattern opt -group "pathway/*" \
  -pfc "\n" -element "? ,name,id"
```

This displays a table of pathway database references:

reactome	Signal Transduction	R-HSA-162582
reactome	Disease	R-HSA-1643685
...		
reactome	Diseases of the neuronal system	R-HSA-9675143
wikipathways	GPCRs, Class A Rhodopsin-like	WP455

Exhaustive Exploration

PubMed Central full-text records consist of recursive "sec" objects that contain section title and paragraph text elements. A record obtained in XML format:

```
esearch -db pmc -query "Kitts PA [AUTH] AND type strains [TITL]" |
efetch -format xml |
```

is first processed by `xtract -mask`, to remove complex formula and table mark-up instructions:

```
xtract -mixed -pattern article -mask "table-wrap,alternatives,inline-formula,\
disp-formula,pub-history,related-article,list-item,fig" -element "*" |
```

Extracting the title and abstract fields near the front of the document is straightforward:

```
xtract -mixed -rec PMCData -pattern article \
-division front \
-wrp Title -prose title-group/article-title \
-wrp Abstract -prose abstract/p \
```

The full text is obtained by exhaustive exploration using the **parent / double star** construct:

```
-division "body/**" \
```

Multiple `-if` clauses use a **question mark** to test the child object's name and select only paragraph and title elements. A **caret** returns the nesting depth of the element:

```
-if "?" -equals "p" -pkg Paragraph \
-wrp Level -element "^" -wrp Text -prose p \
-if "?" -equals "title" -pkg Section \
-wrp Level -element "^" -wrp Title -prose sec/title
```

Like `-else`, the multiple `-if` convention can only be used for element extraction operations. Multiple explorations below the double star would require the use of individual `-group "*" commands`.

```
<PMCData>
<Title>Collection and curation of prokaryotic genome assemblies ... </Title>
<Abstract>The public sequence databases are entrusted with the ... </Abstract>
...
<Section>
<Level>5</Level>
<Title>Assemblies not used as types</Title>
</Section>
<Paragraph>
<Level>5</Level>
<Text>NCBI evaluates all assemblies, including type assemblies ... </Text>
</Paragraph>
...
```

GenBank Filtering

The most recent GenBank virus release file can be downloaded from NCBI servers:

```
nquire -lst ftp.ncbi.nlm.nih.gov genbank |
grep "^gbvrl" | grep ".seq.gz" | sort -V |
tail -n 1 | skip-if-file-exists |
nquire -dwn ftp.ncbi.nlm.nih.gov genbank
```

GenBank flatfile records can be selected by organism name or taxon identifier, or by presence or absence of an arbitrary text string, with **filter-genbank**:

```
gunzip -c *.seq.gz | filter-genbank -taxid 11292 |
```

The full set of filter-genbank arguments are **-accession**, **-accessions**, **-taxid**, **-taxids**, **-organism**, **-truncate**, **-exclude**, **-require**, **-min**, and **-max**.

While GenBank format can be read directly by xtract, explicit conversion to INSDSeq XML with **gbf2xml** may be up to three times faster for large sets of records:

```
gbf2xml |
```

Feature location intervals and underlying sequences of individual coding regions are then obtained by:

```
xtract -insd CDS gene product feat_location sub_sequence
```

Table Operations

Tab-delimited tables can be piped through **filter-columns** for numeric and substring matching:

```
filter-columns '10 <= $2 && $2 <= 30 && $5 ~ peptide'
```

and sent to **print-columns** for flexible modification of text and numbers in the final output:

```
print-columns '$1, $2+1, $3+$4-1, "\042" $5 "\042", tolower($6), total += $2'
```

Both scripts are front-ends to the **awk** data manipulation utility. In awk, a **dollar sign** ("") followed by a digit indicates an input data column, not a Unix command-line argument position.

In order to prevent misinterpretation of awk dollar signs by the Unix shell interpreter, both scripts require their argument to be inside apostrophes instead of double quotation marks. The standard NF and NR awk variables can be used, as can YR (for year) and DT (for date in YYYY-MM-DD format).

Along with **sort-table**, these scripts allow novice users to do useful things on tabular data without first having to know arcane setup details (such as how to specify the tab character as the column separator).

The **align-columns** script has several arguments to reformat tables and make them easier to interpret.

The **intersect-uids**, **combine-uids**, and **exclude-uids** scripts merge unique identifier files with Boolean operations (AND, OR, and NOT, respectively). Use **compare-uids** to see UID differences.

Biological Data in Entrez

EDirect provides additional functions, scripts, and exploration constructs to simplify the extraction of complex data obtained from the interconnected Entrez biological databases.

Sequence Qualifiers

The NCBI data model for sequence records (5) is based on the central dogma of molecular biology. Sequences, including genomic DNA, messenger RNAs, and protein products, are "instantiated" with the actual sequence letters, and are assigned accession numbers for reference.

Features contain information about the biology of a region, including the transformations involved in gene expression. Qualifiers store specific details about a feature, such as the name of the gene, genetic code used for protein translation, or accession of the product sequence.

A **gene** feature indicates the location of a heritable region of nucleic acid that confers a measurable phenotype. An **mRNA** feature on genomic DNA represents the exonic and untranslated regions that remain after message transcription and intron splicing. A coding region (**CDS**) feature has a product reference to the translated protein sequence record:

A search for cone snail venom mature peptides:

```

esearch -db protein -query "conotoxin" -feature mat_peptide |
efetch -format gpc |
xtract -insd complete mat_peptide "%peptide" product mol_wt peptide |

```

uses the xtract -insd function to print the accession number, mature peptide length, product name, calculated molecular weight, and amino acid sequence for a sample of neurotoxic peptides:

AAD31916.1	24	0-superfamily conotoxin Tx03	2555	CYDGGTSCDSGIQCCSGWCIFVCF
AIC77084.1	17	conotoxin Bt14.7	1802	CAHSVPGMHPCKNNTC
AAD31921.1	24	omega conotoxin SVIA mutant 2	2419	CRPSGSPCGVTSICCGRCSRKCT
ADB43131.1	15	conotoxin Cal 1b	1708	LCCKRHHGCHPCGRT
ABE27008.1	25	conotoxin p114.2	2773	GPGSAICNMACRLEHGHLYPFCHCR
ABE27011.1	25	conotoxin fe14.2	2697	SSGSTVCKMMMLRGYGHLYPSCGCR
ABW16858.1	15	marmophin	1915	DWEYHAHPKPNFSWT
AIC77085.1	21	conotoxin Bt14.8	2574	NECDNCMRSFCSMIYEKCRK

...

Piping the results to a series of Unix commands and EDirect scripts:

```

grep -i conotoxin |
filter-columns '10 <= $2 && $2 <= 30' |
sort-table -u -k 2,2n |
align-columns -

```

filters by product name, limits the results to a specified range of peptide lengths, sorts the table by peptide length, chooses one example for each length, and aligns the columns for cleaner printing:

ADB43131.1	15	conotoxin Cal 1b	1708	LCCKRHHGCHPCGRT
ADB43128.1	16	conotoxin Cal 5.1	1829	DPAPCCQHPIETCCRR
AIC77105.1	17	conotoxin Lt1.4	1705	GCCSHAPACDVNNPDICG
ADB43129.1	18	conotoxin Cal 5.2	2008	MIQRSQCCAVKKNCCCHVG
ADD97803.1	20	conotoxin Cal 1.2	2206	AGCCPTIMYKTGACRTNRCR
AIC77085.1	21	conotoxin Bt14.8	2574	NECDNCMRSFCSMIYEKCRK
ADB43125.1	22	conotoxin Cal 14.2	2157	GCPADCPNTCDSSNKCSPGFP
AIC77154.1	23	conotoxin Bt14.19	2578	VREKDCPPHPVPGMHKCVCLKTC

...

The xtract -insdx variant:

```

esearch -db protein -query "conotoxin" | efilter -query "mat_peptide [FKEY]" |
efetch -format gpc |
xtract -insdx complete mat_peptide "%peptide" product mol_wt peptide |
xtract -pattern Rec -select product -contains conotoxin |
xtract -pattern Rec -sort mol_wt

```

saves the output table directly as XML, with the XML tag names taken from the original qualifier names:

...

```

<Rec>
  <accession>AA033169.1</accession>
  <feature_key>mat_peptide</feature_key>
  <peptide_Len>16</peptide_Len>
  <product>alpha-conotoxin GIC</product>
  <mol_wt>1615</mol_wt>
  <peptide>GCCSHAPACAGNNQHIC</peptide>
</Rec>
<Rec>
  <accession>AIC77099.1</accession>
  <feature_key>mat_peptide</feature_key>
  <peptide_Len>16</peptide_Len>

```

```

<product>conotoxin Im1.2</product>
<mol_wt>1669</mol_wt>
<peptide>GCCSHPACNVNPHIC</peptide>
</Rec>
...

```

Qualifier names with prefix shortcuts "#" and "%" are modified to use "_Num" and "_Len" suffixes, respectively.

Missing Qualifiers

For records where a particular qualifier is missing:

```

esearch -db protein -query "RAG1 [GENE] AND Mus musculus [ORGN]" |
efetch -format gpc |
xtract -insd source organism strain |
sort-table -u -k 2,3

```

a dash is inserted as a placeholder:

P15919.2	Mus musculus	-
AA061776.1	Mus musculus	129/Sv
NP_033045.2	Mus musculus	C57BL/6
EDL27655.1	Mus musculus	mixed
BAD69530.1	Mus musculus castaneus	-
BAD69531.1	Mus musculus domesticus	BALB/c
BAD69532.1	Mus musculus molossinus	MOA

Recursive Taxonomy Data

Certain XML objects returned by efetch are recursively defined, including Taxon in -db taxonomy and Gene-commentary in -db gene. Thus, they can contain nested objects with the same XML tag.

Retrieving a set of taxonomy records:

```
efetch -db taxonomy -id 9615,9606 -format xml
```

produces XML with nested Taxon objects (marked below with line references) for each rank in the taxonomic lineage:

```

1  <TaxaSet>
   1  <Taxon>
      <TaxId>9606</TaxId>
      <ScientificName>Homo sapiens</ScientificName>
      ...
      <LineageEx>
2     2  <Taxon>
        <TaxId>131567</TaxId>
        <ScientificName>cellular organisms</ScientificName>
        <Rank>no rank</Rank>
3     </Taxon>
4     4  <Taxon>
        <TaxId>2759</TaxId>
        <ScientificName>Eukaryota</ScientificName>
        <Rank>superkingdom</Rank>
5     </Taxon>
      ...
      </LineageEx>
      ...
6  </Taxon>
7  <Taxon>

```

```

    <TaxId>9615</TaxId>
    <ScientificName>Canis lupus familiaris</ScientificName>
    ...
8    </Taxon>
    </TaxaSet>

```

Xtract tracks XML object nesting to determine that the <Taxon> start tag on line 1 is closed by the </Taxon> stop tag on line 6, and not by the first </Taxon> encountered on line 3.

To accommodate recursively-defined data, entry (by -element) to an internal object is blocked when its name matches the current exploration container.

The **star** / **child** construct bypasses the search constraint:

```

efetch -db taxonomy -id 9606,7227 -format xml |
xtract -pattern Taxon -block "**/Taxon" \
  -tab "\n" -element TaxId,ScientificName

```

to allow controlled descent to the next level:

```

131567    cellular organisms
2759     Eukaryota

```

Using **double star** / **child** recursively visits every object regardless of depth, and can flatten a complex structure into a linear set of elements in a single step:

```

efetch -db taxonomy -id 9606 -format xml |
xtract -pattern Taxon \
  -first TaxId -tab "\n" -element ScientificName \
  -block "***/Taxon" -if Rank -is-not "no rank" -and Rank -excludes "root" \
  -tab "\n" -element Rank,ScientificName

```

This prints all of the individual internal lineage nodes:

```

9606      Homo sapiens
domain    Eukaryota
clade     Opisthokonta
kingdom    Metazoa
clade     Eumetazoa
clade     Bilateria
clade     Deuterostomia
phylum   Chordata
subphylum Craniata
clade     Vertebrata
clade     Gnathostomata
clade     Teleostomi
clade     Euteleostomi
superclass Sarcopterygii
clade     Dipnotetrapodomorpha
clade     Tetrapoda
clade     Amniota
class     Mammalia
...

```

Genes in a Region

Records for protein-coding genes on the human X chromosome are retrieved by running:

```

esearch -db gene -query "Homo sapiens [ORGN] AND X [CHR]" |
efilter -status alive -type coding | efetch -format docsum |

```

Gene names and chromosomal positions are extracted by piping the records to:

```
xtract -pattern DocumentSummary -NAME Name -DESC Description \
-block GenomicInfoType -if ChrLoc -equals X \
-min ChrStart,ChrStop -element "&NAME" "&DESC" |
```

Exploring each GenomicInfoType is needed because of pseudoautosomal regions at the ends of the X and Y chromosomes:

```
...
<GenomicInfo>
  <GenomicInfoType>
    <ChrLoc>X</ChrLoc>
    <ChrAccVer>NC_000023.11</ChrAccVer>
    <ChrStart>155997630</ChrStart>
    <ChrStop>156013016</ChrStop>
    <ExonCount>14</ExonCount>
  </GenomicInfoType>
  <GenomicInfoType>
    <ChrLoc>Y</ChrLoc>
    <ChrAccVer>NC_000024.10</ChrAccVer>
    <ChrStart>57184150</ChrStart>
    <ChrStop>57199536</ChrStop>
    <ExonCount>14</ExonCount>
  </GenomicInfoType>
</GenomicInfo>
...
```

Without limiting to chromosome X, the copy of IL9R near the "q" telomere of chromosome Y would be erroneously placed with genes that are near the X chromosome centromere, shown here in between SPIN2A and ZXDB:

```
...
57121860    FAAH2      fatty acid amide hydrolase 2
57133042    SPIN2A     spindlin family member 2A
57184150    IL9R       interleukin 9 receptor
57592010    ZXDB       zinc finger X-linked duplicated B
...
```

The -if statement eliminates coordinates from pseudoautosomal gene copies present on Y chromosome telomeres. Results can now be sorted by position, and then filtered and partitioned:

```
sort-table -k 1,1n | cut -f 2- |
grep -v pseudogene | grep -v uncharacterized | grep -v hypothetical |
between-two-genes AMER1 FAAH2
```

to produce an ordered table of known genes located between two markers flanking the centromere:

```
FAAH2      fatty acid amide hydrolase 2
SPIN2A     spindlin family member 2A
ZXDB       zinc finger X-linked duplicated B
NLRP2B     NLR family pyrin domain containing 2B
ZXDA       zinc finger X-linked duplicated A
SPIN4      spindlin family member 4
ARHGEF9    Cdc42 guanine nucleotide exchange factor 9
AMER1      APC membrane recruitment protein 1
```

SNP-Modified Product Pairs

Single nucleotide polymorphisms can represent different substitutions at the same position:

```
efetch -db snp -id 11549407 -format docsum |
```

but variation records do not explicitly match a specific CDS modification to its altered protein product.

The `hgvs2spdi` script converts 1-based HGVS (6) data ("NM_000518.5:c.118C>T") into 0-based SPDI (7) format ("NM_000518.5:167:C:T"). For SNPs on cDNA transcripts the position is CDS-relative, and the script retrieves the GenBank record in order to calculate the absolute sequence offset:

```
snp2hgvs | hgvs2spdi | spdi2tbl | tbl2prod
```

The `tbl2prod` step translates the coding region locations (after nucleotide substitution), and sorts them with protein sequences (after residue replacement) to produce adjacent matching CDS/protein pairs:

```
rs11549407    NM_000518.5:167:C:T    MVHLTPEEKSAVTALWGKVNDEVGGEALGRLLVVYPWT*R ...
rs11549407    NP_000509.1:39:Q:*    MVHLTPEEKSAVTALWGKVNDEVGGEALGRLLVVYPWT*R ...
rs11549407    NM_000518.5:167:C:G    MVHLTPEEKSAVTALWGKVNDEVGGEALGRLLVVYPWTER ...
rs11549407    NP_000509.1:39:Q:E    MVHLTPEEKSAVTALWGKVNDEVGGEALGRLLVVYPWTER ...
rs11549407    NM_000518.5:167:C:A    MVHLTPEEKSAVTALWGKVNDEVGGEALGRLLVVYPWTKR ...
rs11549407    NP_000509.1:39:Q:K    MVHLTPEEKSAVTALWGKVNDEVGGEALGRLLVVYPWTKR ...
...
```

Sequence Analysis

EDirect sequence processing functions are provided by the `transmute` program. They can handle huge sequences as normal strings, without requiring any special coding techniques or custom data structures.

Reverse Complementation

A GenBank sequence can be converted to FASTA, reverse-complemented, and printed as FASTA with:

```
efetch -db nuccore -id U00096 -format gb |
gbf2fsa | transmute -revcomp | transmute -fasta -width 50
```

Sequence Editing

The pBR322 cloning vector is a circular plasmid with unique restriction sites in two antibiotic resistance genes. The `transmute -replace` function introduces a second BamHI restriction enzyme recognition site (based on a site-directed mutagenesis experiment) by modifying two bases in the *rop* gene:

```
efetch -db nuccore -id J01749 -format fasta |
transmute -replace -offset 1907 -delete GG -insert TC |
...
```

Pattern Searching

The modified sequence from above is then passed to `transmute -search`, which takes a list of sequence patterns (with optional labels) and uses a finite-state algorithm to simultaneously search for all patterns:

```
...
transmute -search -circular GGATCC:BamHI GAATTC:EcoRI CTGCAG:PstI |
align-columns -g 4 -a rl
```

The (0-based) starting positions and restriction enzyme names for each match are printed in a table:

```
374    BamHI
1904   BamHI
3606   PstI
4358   EcoRI
```

The **disambiguate-nucleotides** and **systematic-mutations** scripts can generate all possible single-base substitutions in a pattern for use in a relaxed-stringency search.

Six-Frame Translation

Protein translation uses a finite-state machine to slide a triplet-codon window along the sequence. State values are offsets into an amino acid lookup table. The next-state and reverse-complement transition tables, and an amino acid table for every genetic code, are all precomputed. A nucleotide sequence can then be translated in all six possible reading frames by an extremely fast loop of indexed lookups:

```
efetch -db nuccore -id U54469.1 -format fasta |
transmute -cds2prot -gcode 1 -all |
```

This produces regions of translated protein separated by asterisks representing stop codons:

```
>U54469.1-1+
RLLGFYNISQ*QAFPELPCSTIDSLWPPKSQT*LKN*IIRIIIKPSNLR ...
>U54469.1-2+
GCLGFITSVSDRHFQSCPVQQSIAAFGHQNPKN*RIK*FE**LSPVTYA ...
...
```

The FASTAs are then converted to XML, and **xtract -pept** splits the translated frames at each stop codon (asterisk), sequence gap (hyphen), or ambiguous translation ("X") character:

```
fsa2xml |
xtract -rec FASTA -pattern FASTA \
-group "FASTA/*" -element "*" \
-group FASTA -wrp FRAG -pept Seq
```

That appends individual peptide fragments to the original fields and creates 6 new XML records:

```
...
<FRAG>rllgfynisq</FRAG>
<FRAG>qafpelpcstidslwppksqt</FRAG>
<FRAG>lkn</FRAG>
...
```

Feature Locations

A table of coding region locations and gene names can be saved directly as XML with **xtract -insdx**:

```
efetch -db nuccore -id NC_000011 -format gb -style master |
xtract -insdx CDS gene feat_location > cds_loc.xml
```

The human β -globin coding region location is then retrieved and stored in a Unix shell variable with:

```
loc=$( xtract -input cds_loc.xml -pattern Rec \
-if gene -equals HBB -element feat_location )
```

Location intervals are shown in biological order, where start is greater than stop on the minus strand:

```
5227021..5226930, 5226799..5226577, 5225726..5225598
```

Sequence Transformations

The cascading effects of a genomic SNP can be reproduced with **transmute** functions: **-replace** applies the substitution, **-extract** uses the location intervals from above to isolate the altered coding sequence, and **-cds2prot** translates the modified CDS into protein with the designated genetic code:

```
efetch -db nuccore -id NC_000011 -format fasta |
transmute -replace -offset 5226773 -delete G -insert A |
transmute -extract -1-based "$loc" |
transmute -cds2prot -gcode 1 -frame 0 -every -trim
```

The genomic G to A transition on human chromosome 11 corresponds to the C to T substitution on the minus-strand-encoded β -globin mRNA in the earlier SNP example:

```
MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWT*R ...
```

Gene Positions

An understanding of sequence coordinate conventions is necessary in order to use gene positions to retrieve the corresponding chromosome subregion with `efetch` or with the UCSC browser.

Sequence records displayed in GenBank or GenPept formats use a **"one-based"** coordinate system, with sequence position numbers starting at "1":

```
1 catgccattc gttgagttgg aaacaaactt gccggctagc cgcatacccg cggggctgga
61 gaaccggctg tgtgcggcca cagccaccat cctggacaaa cccgaagacg tgagtgaggg
121 tcggcgagaa cttgtgggct agggtcggac ctcccaatga cccgttccca tccccagga
181 cccactccc ctggtaacct ctgaccttc gtgtcctatc ctcccttct agatcccttc
...
```

Under this convention, positions refer to the sequence letters themselves:

C	A	T	G	C	C	A	T	T	C
1	2	3	4	5	6	7	8	9	10

and the position of the last base or residue is equal to the length of the sequence. The ATG initiation codon above is at positions 2 through 4, inclusive.

For computer programs, however, using **"zero-based"** coordinates can simplify the arithmetic used for calculations on sequence positions. The ATG codon in the 0-based representation is at positions 1 through 3. (The UCSC browser uses a hybrid, half-open representation, where the start position is 0-based and the stop position is 1-based.)

Software at NCBI will typically convert positions to 0-based coordinates upon input, perform whatever calculations are desired, and then convert the results to a 1-based representation for display. These transformations are done by simply subtracting 1 from the 1-based value or adding 1 to the 0-based value.

Coordinate Conversions

Retrieving the docsum for a particular gene:

```
esearch -db gene -query "BRCA2 [GENE] AND human [ORGN]" |
efetch -format docsum |
```

returns the chromosomal position of that gene in "0-based" coordinates:

```
...
<GenomicInfoType>
  <ChrLoc>13</ChrLoc>
  <ChrAccVer>NC_000013.11</ChrAccVer>
  <ChrStart>32315479</ChrStart>
  <ChrStop>32399671</ChrStop>
  <ExonCount>27</ExonCount>
</GenomicInfoType>
...
```

Piping the document summary to an xtract command using `-element`:

```
xtract -pattern GenomicInfoType -element ChrAccVer ChrStart ChrStop
```

obtains the accession and 0-based coordinate values:

```
NC_000013.11    32315479    32399671
```

Efetch has `-seq_start` and `-seq_stop` arguments to retrieve a gene segment, but these expect the sequence subrange to be in 1-based coordinates.

To address this problem, two additional efetch arguments, `-chr_start` and `-chr_stop`, were created to allow direct use of the 0-based coordinates:

```
efetch -db nuccore -format gb -id NC_000013.11 \  
-chr_start 32315479 -chr_stop 32399671
```

Xtract numeric extraction commands can also assist with coordinate conversion. Using `xtract -inc`:

```
xtract -pattern GenomicInfoType -element ChrAccVer -inc ChrStart ChrStop
```

obtains the accession and 0-based coordinates, then increments the positions to produce 1-based values:

```
NC_000013.11    32315480    32399672
```

EDirect knows the policies for sequence positions in all relevant Entrez databases (e.g., gene, snp, dbvar), and provides additional shortcuts for converting these to other conventions. For example:

```
xtract -pattern GenomicInfoType -element ChrAccVer -1-based ChrStart ChrStop
```

understands that gene docsum ChrStart and ChrStop fields are 0-based, sees that the desired output is 1-based, and translates the command to convert coordinates internally using the `-inc` logic. Similarly:

```
-element ChrAccVer -ucsc-based ChrStart ChrStop
```

leaves the 0-based start value unchanged but increments the original stop value to produce the half-open form that can be passed to the UCSC browser:

```
NC_000013.11    32315479    32399672
```

Gene Sequence

Genes encoded on the minus strand of a sequence:

```
esearch -db gene -query "DDT [GENE] AND mouse [ORGN]" |  
efetch -format docsum |  
xtract -pattern GenomicInfoType -element ChrAccVer ChrStart ChrStop |
```

have coordinates ("0-based" in docsums) where the start position is greater than the stop:

```
NC_000076.6     75773373     75771232
```

These values can be read into Unix variables by a "while" loop:

```
while IFS=$'\t' read acn str stp  
do  
  efetch -db nuccore -format gb \  
    -id "$acn" -chr_start "$str" -chr_stop "$stp"  
done
```

The variables can then be used to obtain the reverse-complemented subregion in GenBank format:

```

LOCUS      NC_000076                2142 bp    DNA        linear    CON 08-AUG-2019
DEFINITION Mus musculus strain C57BL/6J chromosome 10, GRCm38.p6 C57BL/6J.
ACCESSION  NC_000076 REGION: complement(75771233..75773374)

```

```

...
    gene      1..2142
              /gene="Ddt"
    mRNA      join(1..159,462..637,1869..2142)
              /gene="Ddt"
              /product="D-dopachrome tautomerase"
              /transcript_id="NM_010027.1"
    CDS       join(52..159,462..637,1869..1941)
              /gene="Ddt"
              /codon_start=1
              /product="D-dopachrome decarboxylase"
              /protein_id="NP_034157.1"
              /translation="MPFVELETNLPASRIPAGLENRLCAATATILDKPEDRVSVTIRP
GMTLLMNKSTEPCAHLLVSSIGVVGTAEQNRTHSASFCKFLTEELSLDQDRIVIRFFP
...

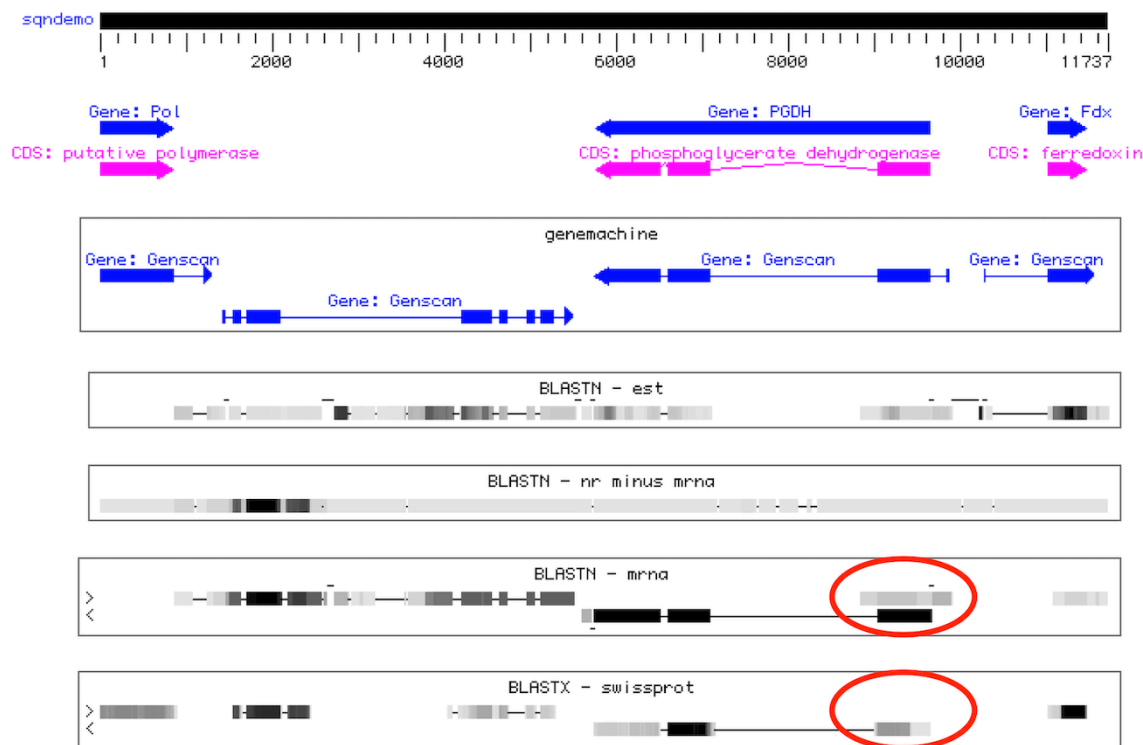
```

The reverse complement of a plus-strand sequence range can be selected with `efetch -revcomp`.

Alignment Excavation

NHGRI's experimental GeneMachine program could run several gene prediction algorithms and BLAST searches, gathering the results to assemble a richly-annotated sequence record.

NCBI's Sequin program could read the record and display the combined ranges of separate alignments in a "smear" that made it possible to see an emerging image of the underlying gene structures:



While interesting patterns (e.g., potential antisense regulation transcripts) might occasionally be noticed by visual inspection, examination by eye is tedious, error-prone, and does not scale to chromosome size. However, the process of converting raw BLAST alignment data in ASN.1 format to a computable, tab-delimited data table, is easily automated with a chain of simple `xtract` commands.

The first xtract reads the ASN.1 sequence record, converts it internally to XML, and uses **-select** to conditionally filter out everything but the BLASTN-mRNA annotation:

```
xtract -pattern annot_E -select label/str -equals "BLASTN - mrna" |
```

For each alignment, the segment start positions (and strands) are implicitly paired - one for the genomic sequence followed by one for the mRNA sequence. One particular alignment has three segments (here showing the original ASN.1 on the right, and the automatically-derived XML on the left):

<starts>	starts {
<starts_E>5613</starts_E>	5613,
<starts_E>842</starts_E>	842,
<starts_E>-1</starts_E>	-1,
<starts_E>961</starts_E>	961,
<starts_E>5599</starts_E>	5599,
<starts_E>962</starts_E>	962
</starts>	

The "-1" start position indicates an internal gap in the genomic assembly relative to the cloned mRNA sequence, and the length of "1" (below) says that it is a single base deletion. The "underscore-E" suffix indicates that the object was derived from an unlabeled element within a named ASN.1 sequence or set.

A second xtract explores each alignment, using **-odd** to get the first (genomic) value in each start pair:

```
... -pattern align_E -block starts -wrp Start -odd starts_E ... |
```

Each element is wrapped with a descriptive name, and the fields are packed into a novel structure:

```
<Start>5613</Start>
<Start>-1</Start>
<Start>5599</Start>
<Length>119</Length>
<Length>1</Length>
<Length>14</Length>
```

The range of an alignment can be calculated from the first and last start positions, and either the first or the last segment length value, depending upon the strand. Those values are obtained using **-first** and **-last** positional arguments:

```
... -wrp FirstPos -first Start -wrp LastPos -last Start ... |
```

and the new fields are packaged in a different ad hoc XML structure:

```
<FirstPos>5613</FirstPos>
<LastPos>5599</LastPos>
<FirstLen>119</FirstLen>
<LastLen>14</LastLen>
<Strand>minus</Strand>
```

A final xtract command performs two parallel explorations, one for each strand:

```
...
-block Rec -if Strand -equals plus -def "-" \
-element Accn Score FirstPos LastPos LastLen Strand \
-block Rec -if Strand -equals minus -def "-" \
-element Accn Score LastPos FirstPos FirstLen Strand |
```

producing a tab-delimited intermediate table:

```
AY046051.1    126    5599    5613    119    minus
```

A `print-columns` command calculates the end position and total length of each alignment, and the results are sorted by strand and genomic location. The "0-based" position fields are then incremented to produce "1-based" final values:

```
print-columns '$1, $2, $3, $4 + $5 - 1, $4 + $5 - $3, $6' |
sort-table -k 6,6fr -k 3,3n -k 4,4nr -k 1,1f |
print-columns '$1, $2, $3 + 1, $4 + 1, $5, $6'
```

The adjusted table values, which are suitable for further computation:

```
AY046051.1    126    5600    5732    133    minus
```

now match the coordinates in the standard BLAST report:

```
5600  AACTTAATACATAA-GTTGGTAGCCACAAATGTGAAAGATTAAATTAATACTCATCCATT 5658
      |||
976   AACTTAATACATAATGTTGGTAGCCACAAATGTGAAAGATTAAATTAATACTCATCCATT 917
      ...
```

Local PubMed Archive

Fetching data from Entrez works well when a few thousand records are needed, but it does not scale for much larger sets of data, where the time it takes to download becomes a limiting factor.

Local Record Cache

EDirect can now preload 40 million live PubMed records onto an inexpensive external 1 TB solid-state drive (SSD), storing them in indexed archive files. For example, PMID 2539356 would be entry 9356 in:

```
/pubmed/Archive/00/02/000253.archive
```

Each archive contains up to 10,000 individually-compressed source data records in a specific range of identifiers. Preceding those concatenated, independent blocks is a binary array of the cumulative block lengths. This allows rapid random access to any or all of the compressed records in the archive.

The local archive is a completely self-contained turnkey product, with no need to download, configure, and maintain complicated third-party database software.

Set an environment variable in your configuration file(s) to reference a section of your external drive:

```
export EDIRECT_LOCAL_ARCHIVE=/Volumes/external_drive_name/
```

Then run **archive-pubmed** to download the PubMed release files and save each record on the drive. The initial download may take several hours, depending on your network connection, with initial archiving taking another few hours. Subsequent updates are incremental, and should finish in minutes.

Fetching a set of 144,129 PubMed records from NCBI's network service takes a bit over 30 minutes:

```
esearch -db pubmed -query "PNAS [JOUR]" -pub abstract | efetch -format xml
```

Searching the local indices (see next section) with **xsearch**, and then using **xfetch** to retrieve and decompress the XML records from the local archive, runs in under 7 seconds:

```
xsearch -db pubmed -query "PNAS [JOUR] AND has abstract [PROP]" | xfetch
```

This is more than 250 times faster than using the EUtils servers, and avoids network reliability issues.

Piping to **xfetch -stream** returns the original compressed records, for a roughly four-fold size reduction of data to send over a network. This would be followed by decompression on the client with **gunzip -c**.

Alternatively, using `xfetch -turbo` precedes each record with an object containing its size in bytes:

```
<NEXT_RECORD_SIZE>4027</NEXT_RECORD_SIZE>
```

Piping these enhanced records to `xtract -turbo` approximately doubles the speed of the (rate-limiting) record partitioning step, which otherwise uses the Boyer-Moore-Horspool fast string search algorithm.

Running `xfetch -db pubmed -all` retrieves the entire database by piping each ".archive" file through:

```
dd ibs=8000 skip=10 2> /dev/null | gunzip -c 2> /dev/null
```

Local Search Index

A similar strategy was used to create a local information retrieval system suitable for large data mining queries. Run `archive-pubmed -index` to populate retrieval index files from records stored in the local archive. The initial indexing will also take a few hours. To build a small subset for evaluation, run:

```
archive-pubmed -index -custom "J Exp Med [JOUR] AND 2000:2026 [PDAT]"
```

For PubMed titles and primary abstracts, the indexing process deletes hyphens after specific prefixes, removes accents and diacritical marks, splits words at punctuation characters, corrects encoding artifacts, and spells out Greek letters for easier searching on scientific terms. It then prepares inverted indices with term positions, and uses them to build distributed term lists and postings files.

For example, the term list that includes "cancer" in the title or abstract would be located at:

```
/pubmed/Postings/TIAB/c/a/n/c/canc.TIAB.trm
```

A query on cancer thus only needs to load a very small subset of the total index. The software supports expression evaluation, wildcard truncation, phrase queries, proximity searches, and partial matches.

The `xinfo`, `xsearch`, `xlink`, and `xfilter` scripts provide access to the local search system.

Names of indexed fields, all terms for a given field, and terms plus record counts, are shown by:

```
xinfo -fields
```

```
xinfo -terms TITL
```

```
xinfo -totals PROP
```

Terms are truncated with a trailing asterisk, and can be expanded to show individual postings counts:

```
xinfo -count "catabolite repress*"
```

```
xinfo -counts "catabolite repress*"
```

Query evaluation includes Boolean operations and parenthetical expressions:

```
xsearch -query "(literacy AND numeracy) NOT (adolescent OR child)"
```

Adjacent words in title or abstract fields are treated as a contiguous phrase:

```
xsearch -query "selective serotonin reuptake inhibitor [TITL]"
```

Each plus sign will replace a single word inside a phrase, and runs of tildes indicate the maximum distance between sequential phrases:

```
xsearch -query "vitamin c + + common cold"
```

```
xsearch -query "vitamin c ~ ~ common cold"
```

Ranked partial term matching is available in any field with **-match**:

```
xsearch -match "tn3 transposition immunity [PAIR]" | just-top-hits 1
```

An exact substring match, without special processing of Boolean operators or indexed field names, can be obtained with **-title** (on the article title) or **-exact** (on the title or abstract):

```
xsearch -title "Genetic Control of Biochemical Reactions in Neurospora."
```

MeSH identifier code, MeSH hierarchy key, and year of publication are also indexed, and MESH field queries are supported by internally mapping to the appropriate CODE or TREE entries:

```
xsearch -db pubmed -query "C14.907.617.812* [TREE] AND 2015:2019 [YEAR]"
```

PMIDs processed through an external source can be reintroduced to a local query pipeline with **xfilter**:

```
| xfilter -db pubmed -query "Monoamine Oxidase [MESH] AND Deficiency [SUBH]" |
```

Data Analysis

All query commands return a structured message containing the database name and a list of UIDs, which can be piped directly to **xfetch** to retrieve the uncompressed records. For example:

```
xsearch -db pubmed -query "selective serotonin ~ ~ ~ reuptake inhibit*" |
xfetch |
xtract -pattern PubMedArticle -num AuthorList/Author |
sort-uniq-count -n |
reorder-columns 2 1 |
align-columns -g 4 -a lr
```

performs a proximity search with dynamic wildcard expansion (matching phrases like "selective serotonin and norepinephrine reuptake inhibitors") and fetches 16,088 PubMed records from the local archive. It then prints a frequency table of the number of papers per number of authors (a consortium is treated as a single author):

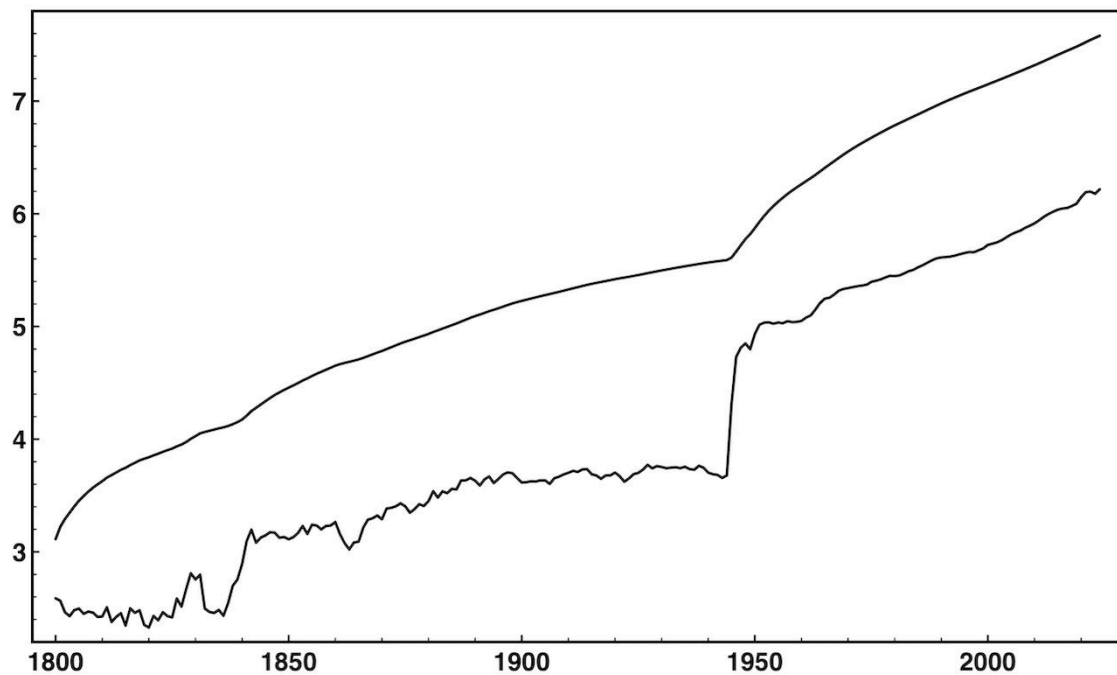
```
0      56
1     1494
2     2144
3     2228
...
```

Data Visualization

The cumulative size of PubMed can be calculated with a running sum of the annual record counts:

```
xinfo -db pubmed -totals YEAR |
print-columns '$2, $1, total += $1' |
print-columns '$1, log($2)/log(10), log($3)/log(10)' |
filter-columns '$1 >= 1800 && $1 < YR' |
xy-plot annual-and-cumulative.png
```

Exponential growth over time will appear as a roughly linear curve on a semi-logarithmic graph:



The sharp jump after World War II was caused by several factors, including the release of declassified papers, a policy of expanding biomedical research in postwar America, and the introduction of computers that could keep up with the indexing of articles from a broader range of subjects.

Natural Language Processing

NLM's Biomedical Text Mining Group performs computational analysis to extract chemical, disease, and gene references from article contents (8). NLM indexing of PubMed records assigns Gene Reference into Function (GeneRIF) mappings (9).

Running **archive-nlmnlp-index** periodically (monthly) will automatically refresh any out-of-date support files and then index the connections in CHEM, DISZ, GENE, GRIF, GSYN, and PREF fields:

```
xinfo -terms DISZ | grep -i Raynaud
xinfo -counts "Raynaud* [DISZ]"
xinfo -query "Raynaud Disease [DISZ]"
```

Following Citation Links

Running **archive-nihocc-index** will download the latest NIH Open Citation Collection monthly release and build CITED and CITES indices, the local equivalent of **elink -cited** and **-cites** commands.

Citation links are retrieved by piping one or more PMIDs to **xlink -target**:

```
xsearch -db pubmed -query "Haselkorn R* [AUTH]" |
xlink -target CITED |
```

This returns PMIDs for 8,356 articles that cite the original 225 papers. The results are then restricted to a range of recent years, and those records are fetched. The **xtract -histogram** shortcut builds a journal frequency table from the subsequent articles:

```
xfilter -query "2020:2025 [YEAR]" | xfetch |
xtract -pattern PubmedArticle -histogram Journal/ISOAbbreviation |
sort-table -nr | head -n 10
```

The **archive-pids -index** command reads the PubMed local archive's incremental inverted index files, and builds a PMCID index that allows xlink to return PubMed Central identifiers from PMIDs:

```
xlink -db pubmed -id 12372140 -target PMCID | xfetch
```

The xlink **-ranked** flag produces the same ranked output format as xsearch **-match**.

Additional Experimental Archives

Legacy PMC release files are downloaded via FTP by running **archive-pmc -index**, which collects primary author names, citation details, section titles, and full-text paragraphs. It then converts them to the more tractable PMCInfo object, and builds an archive from those derived records.

Similarly, **archive-taxonomy -index** archives novel records assembled from NCBI taxonomy data tables retrieved from the FTP site.

A new database is ready for use once its autonomous archiving script has finished all downloading, conversion, validation, caching, indexing, inversion, collection, merging, and posting steps.

Adding a New Local Archive

New database domains can be added using records obtained from external public data resources, or with private information taken from a laboratory's research results or a hospital's clinical trial data.

Archive Scripts

The first step is to create a one-line archive script that sends command-line arguments to **xbuild**. A sample database indexed by peptide sequence fragments is built with **archive-peptide**, which runs:

```
xbuild -db peptide -project peptide "$@"
```

A primary database has the same values for the **-db** and **-project** arguments. It may import structured records, optionally caching simpler derived records, or it may assemble records from tabular data.

Secondary projects add new indexed fields or links to primary databases. The **archive-nihocc** script:

```
xbuild -db pubmed -project nihocc "$@"
```

adds reciprocal citation reference links to the pubmed database, but does not create its own records.

Project Directories

A new database or project next requires adding a project directory within the **edirect** / **extern** folder. The **edirect** / **extern** / **peptide** subfolder contains several helper scripts. By convention, their names are composed of a formal step in the build process, a hyphen, and the project name:

```
download-peptide  
populate-peptide  
index-peptide
```

A single configuration file named for the project is also present:

```
peptide.ini
```

Secondary project files go in an **edirect** / **extern** / **{database}-{project}** subfolder.

Configuration Files

Project-specific configuration files guide the build process. The **peptide.ini** file provides sections for archiving and posting, as well as information used by xfetch to retrieve the derived records:

```
[info]
db=peptide
project=peptide

[archive]
name=PeptInfo
index=PeptInfo/UID

[fetch]
name=PeptInfo
set=PeptInfoSet
```

In contrast, the **nihocc.ini** configuration file adds link flags in the merge and posting sections:

```
[info]
db=pubmed
project=nihocc

[merge]
link=true

[posting]
fields="CITED CITES"
link=true
```

while **pubmed.ini** has a links section with destination databases for xlink **-target** arguments:

```
...
[links]
CITED=pubmed
CITES=pubmed
PMCID=pmc
```

"Download" Helper

The actual helper files needed for any given project depends on the source of the records, and on any accessory data files that must be retrieved separately. For each archive step, the helper file is "sourced" by an intermediate script, which executes it in the context of properly initialized variables.

An abridged version of the **download-peptide** file is shown below:

```
FilterByDivision() {
    dir="$1"

    if [ "$dir" = "archaea" ]
    then
        grep "wp_"
    elif [ "$dir" = "bacteria" ] || [ "$dir" = "plasmid" ]
    then
        grep "."
    else
        grep -v "wp_"
    fi
}
```

```

}

if [ -d "${sourceBase}" ]
then
  cd "${sourceBase}"

  if [ -n "$custom" ]
  then
    nquire -lst ftp.ncbi.nih.gov/refseq/release "$custom" < /dev/null |
    grep gpff | FilterByDivision "$custom" | sort -V | skip-if-file-exists |
    while read fl
    do
      echo "$fl" | nquire -asp ftp.ncbi.nih.gov/refseq/release "$custom"
    done
  fi
fi

```

For this example, a RefSeq division folder is passed by archive-peptide using the **-custom** argument:

```
archive-peptide -custom "viral"
```

In the real download-peptide file, the custom variable contains one or more division abbreviations:

```
archive-peptide -custom "ARC FUN PRO VRL"
```

Individual build steps can be executed with **-step**, or bypassed with **-skip**:

```
archive-peptide -step IDX
archive-peptide -index -skip "SET CHK DAT DWN GEN POP RES IDX COL MRG PST CLR"
```

For the full download-peptide helper file, running just the download step without a **-custom** argument prints a table displaying the division abbreviation, actual subfolder name, number of files, and total number of bytes, for each taxonomic division (formatted by align-columns **-h 2 -g 4 -a llrm**):

ARC	archaea	9	1,956,632,000
BCT	bacteria	825	181,043,837,778
FUN	fungi	36	4,029,672,534
INV	invertebrate	93	6,206,662,948
MAM	vertebrate_mammalian	121	5,892,072,442
MIT	mitochondrion	1	113,495,942
PLN	plant	67	4,368,670,110
PRO	protozoa	7	786,600,954
PSM	plasmid	9	1,388,949,422
PST	plastid	3	512,697,988
VRL	viral	1	227,714,284
VRT	vertebrate_other	178	10,748,398,867

The counts refer to data files run through the FilterByDivision function. This favors the non-redundant "WP" protein sequence files for archaea and bacteria, which were introduced for prokaryotes in 2013.

"Populate" Helper

Unless the source records are perfect for your intended use, you can design a more convenient structure containing just the information you want.

The **populate-peptide** script originally converted the downloaded GenPept flatfiles to INSDSeq XML. The current record number, which was obtained by using a **plus sign** ("+") as an xtract -element argument, then became the integer access key for caching the derived record:

```

base=${fl%.gz}
if [ ! -f "${sentinelsBase}/${base.snt}" ] && [ -s "$fl" ]
then

    gunzip -c "$fl" | gbf2xml |
    xtract -rec PeptInfo -pattern INSDSeq \
        -wrp UID -element "+" \
        ...
        -wrp Sequence -element INSDSeq_sequence |
    transmute -format |
    rchive -gzip -archive -db "$dbase" -index UID -pattern PeptInfo

    touch "${sentinelsBase}/${base.snt}"
fi

```

Each protein record is converted into a simple PeptInfo structure with a unique identifier value:

```

<PeptInfo>
  <UID>302720946</UID>
  <Accession>WP_003047541</Accession>
  <Organism div="BCT" taxid="271">Thermus aquaticus</Organism>
  <Product>type I DNA topoisomerase</Product>
  <Sequence>mpkkpktqgaahlgeggpkaeearattlvvvespakarsiqkmlgp ... </Sequence>
...

```

(The current code calls a direct conversion function to bypass the INSDSeq intermediate. It also adds the previous maximum UID value, which allows separate divisions to be processed at different times.)

"Index" Helper

Given a protein sequence, **-element** will show it as a polypeptide, **-letters** will split it into individual amino acids, and **-pentamers** will create overlapping oligopeptides (residues 1–5, 2–6, 3–7, etc.):

```

-element      mpkkpktqgaahlgeggpkaeearattlvvvespakarsiqk ...
-letters      m p k k p k t q g a a h l g e g g p k a e ...
-pentamers    mpkkp pkkpk kpkpt kpktq pktqg ktqga tqgaa ...

```

The **-letters** option resembles words in a sentence, which is the starting point for building the positional indices that support phrase and proximity searches. But a standard phrase search will drop candidates that have even a single mismatch to the query sequence.

A better approach for finding the most similar sequences is to use **xsearch -match** on the pentamers. Indexing short overlapping peptides would make more efficient use of the local archive's directory hierarchy. The choice of 5–mers was based on results from NCBI's early SEQR indexed search experiment (10). For convenience, when peptide fragments are indexed using the PENT field, **-match** will internally split the protein query sequence into overlapping pentamers before running the search:

```

xsearch -db peptide -match "rlgrdtadmiqlikefdaaggvavrfiddgistdgdmgqmv [PENT]"

```

The result is a set of record keys ranked by the number of unique overlapping fragments matched.

For primary databases, index helpers embed an **xtract** command in a Unix "HERE" document. This is then passed as an argument for execution within an incremental indexing function. (Secondary projects do not need this invalidated index repopulation mechanism.)

A simplified excerpt of the **index-peptide** script:

```

read -r -d '' idxtxt <<- EOS
xtract -set IdxDocumentSet -rec IdxDocument \

```

```

-pattern PeptInfo -UID PeptInfo/UID \
-wrp IdxUid -element "&UID" -clr -rst -tab "" \
-group PeptInfo -pkg IdxSearchFields \
...
-block Product -wrp PROD -indexer Product \
-block Organism -wrp ORGN -element Organism -rst \
-wrp DIV -element @div \
-block Sequence ... -wrp PENT -pentamers Sequence
EOS

```

IncrementalIndex "\${idxtxt}"

will split the overlapping pentamers into separate terms for regular indexing:

```

...
<IdxDocument>
  <IdxUid>302720946</IdxUid>
  <IdxSearchFields>
    <UID>0302720946</UID>
    <ACCN-WP>003047541</ACCN-WP>
    <PROD pos="3">dna</PROD>
    <PROD pos="2">i</PROD>
    <PROD pos="4">topoisomerase</PROD>
    <PROD pos="1">type</PROD>
    <ORGN>Thermus aquaticus</ORGN>
    <DIV>BCT</DIV>
    <SLEN>833</SLEN>
    <MLWT>93298</MLWT>
    <PENT>mpkkp</PENT>
    <PENT>pkkpk</PENT>
  ...

```

Text fields can use **-indexer** to create positional indices, but it also removes stop words, which would discard valid pentapeptides like "apply", "every", and "never". Those attributes will likely be decoupled by adding new configuration options to allow finer adjustment of field-specific behavior.

Later build steps, starting with index inversion, are standardized, and do not need external helper files.

Importing Data Tables

Large data tables are processed by programs written in the Go programming language. They are compiled and executed on-the-fly by helpers calling **"go run"**. The central loop in **prep-nihocc.go**, simplified for brevity, and without the usual reality checks and periodic output buffer flushing, is:

```

var bldr strings.Builder

wrtr := bufio.NewWriter(os.Stdout)
scanr := bufio.NewScanner(os.Stdin)

for scanr.Scan() {
    line := scanr.Text()
    if line == "citing, referenced" {
        continue
    }

    cols := strings.Split(line, ",")
    fst, scd := cols[0], cols[1]

```

```

    pdFst, pdScd := padNumericID(fst), padNumericID(scd)

    bldr.WriteString(fst + "\tCITED\t" + pdScd + "\n")
    bldr.WriteString(scd + "\tCITES\t" + pdFst + "\n")

    txt := bldr.String()
    wrtr.WriteString(txt[:])
    bldr.Reset()
}

wrtr.Flush()

```

Archive Configuration File

More flexible drive management uses an environment variable that points to a configuration file:

```
export EDIRECT_LOCAL_CONFIG="${HOME}/.edirectrc"
```

(For quick evaluation, you can instead place an **edirect.ini** configuration file into the edirect folder.)

The configuration file has one section per database, with entries pointing to solid-state drives:

```
[pubmed]
```

An **ARCHIVE** entry is required, and defaults to having all folders in the absence of other entries, but it is primarily for the **Archive**, **Data**, and **Posting** folders needed for record retrieval and indexed queries:

```
ARCHIVE=/Volumes/archive/
```

If set, a **WORKING** entry will move **Extras**, **Invert**, **Merged**, **Scratch**, and **Source** folders, used for building the archive and search indices but not for active queries, to another drive:

```
WORKING=/Volumes/builder-pool-pm/
```

(**Scratch** prevents secondary projects from colliding with primary incremental inverted index files.)

Independently, **POSTING** and **SOURCES** can move their respective folders to other drives. The source file repository can even use a large, rotating hard disk, since it only needs sequential streaming access:

```
POSTING=/Volumes/pm-posting/
SOURCES=/Volumes/common-sources-1/
```

An optional **[default]** section applies to any databases that do not have their own named sections.

With this configuration mechanism, each separate database can point to its own set of unique drives.

Solid-State Drive Preparation

To initialize a solid-state drive for hosting the local archive on a Mac, log into an admin account, run Disk Utility, choose View → Show All Devices, select the top-level external drive, and press the Erase icon. Set the Scheme popup to GUID Partition Map, and APFS will appear as a format choice. Set the Format popup to APFS, enter the desired name for the volume, and click the Erase button.

To finish the drive configuration, disable Spotlight indexing on the drive with:

```
sudo mdutil -i off "${EDIRECT_LOCAL_ARCHIVE}"
sudo mdutil -E "${EDIRECT_LOCAL_ARCHIVE}"
```

and disable FSEvents logging with:

```
sudo touch "${EDIRECT_LOCAL_ARCHIVE}/.fseventsd/no_log"
```

Also exclude the drive from being backed up by Time Machine or scanned by a virus checker, and, in Apple → System Settings → Privacy & Security → Full Disk Access, turn on the Terminal slide switch.

Python Integration

Controlling EDirect from Python scripts is easily done with assistance from the **edirect.py** library file, which is included in the EDirect archive.

At the beginning of your program, import the edirect module with the following commands:

```
#!/usr/bin/env python3

import sys
import os
import shutil

sys.path.insert(1, os.path.dirname(shutil.which('xtract')))
import edirect
```

The first argument to **edirect.execute** is the Unix command you wish to run. It can be a string:

```
("efetch -db nuccore -id NM_000518.5 -format fasta")
```

or a sequence of strings, which allows a variable's value to be substituted for a specific parameter:

```
accession = "NM_000518.5"
(('efetch', '-db', 'nuccore', '-id', accession, '-format', 'fasta'))
```

An optional second argument accepts data to be passed to the Unix command through stdin. Multiple steps are chained together by using the result of the previous command as the data argument in the next command:

```
seq = edirect.execute("efetch -db nuccore -id NM_000518.5 -format fasta")
sub = edirect.execute("transmute -extract -1-based -loc 51..494", seq)
prt = edirect.execute(('transmute', '-cds2prot', '-every', '-trim'), sub)
```

Data piped to the script itself is relayed by using **sys.stdin.read()** as the second argument.

Alternatively, the **edirect.pipeline** function can execute a string containing several piped commands:

```
edirect.pipeline('''efetch -db nuccore -id NM_000518.5 -format gb |
                  xtract -insd CDS gene product feat_location''')
```

or can accept a sequence of individual command strings to be piped together for execution:

```
edirect.pipeline(('efetch -db protein -id NP_000509.1 -format gp',
                  'xtract -insd Protein mol_wt sub_sequence'))
```

Hiding details (e.g., `isinstance`, `shlex.join`, `shlex.split`, and `subprocess.run`) inside a common module means that biologists who are new to coding could control an entire analysis pipeline from their first Python program.

An **edirect.efetch** shortcut that uses named arguments is also available:

```
edirect.efetch(db="nuccore", id="NM_000518.5", format="fasta")
```

To run a custom shell script, make sure the execute permission bit is set, supply the full execution path, and follow it with any command-line arguments:

```
db = "pubmed"
res = edirect.execute("./datefields.sh", db), "")
```

Shell Scripting

The Unix "shell" is a command interpreter that supports user-defined variables, conditional statements, and repetitive execution loops. Shell scripts are saved in files, and are referenced by file name.

Comments start with a pound sign ("#") and are ignored. Quotation marks within quoted strings are entered by "escaping" with a backslash ("\"). Subroutines (functions) can be used to collect common code or simplify the organization of the script.

Given a tab-delimited file of feature keys and values, where each gene is followed by its coding regions:

```
gene      matK
CDS       maturase K
gene      ATP2B1
CDS       ATPase 1 isoform 2
CDS       ATPase 1 isoform 7
```

the **cat** command can pipe the file contents to a shell script that reads the data one line at a time.

Dissecting the code, the first line selects the Bash shell for executing the script on the user's machine:

```
#!/bin/bash
```

The latest gene name is stored in the "gene" variable, which is first initialized to an empty string:

```
gene=""
```

The **while** command sequentially reads each line of the input file, **IFS** indicates tab-delimited fields, and **read** saves the first field in the "feature" variable and the remaining text in the "product" variable. The statements between the **do** and **done** commands are then executed separately for each input line:

```
while IFS=$'\t' read feature product
do
```

The **if** command retrieves the current value stored in the feature variable (indicated by placing a dollar sign in front of the variable name) and compares it to the word "gene":

```
if [ "$feature" = "gene" ]
```

If the feature key was "gene", it runs the **then** section, which copies the contents of the current line's "product" value into the persistent "gene" variable. Otherwise, the **else** section prints the saved gene name and the current coding region product name, separated by a tab character. The conditional block is terminated with a **fi** instruction ("if" in reverse):

```
then
    gene="$product"
else
    echo -e "$gene\t$product"
fi
done
```

(Prior to **else**, a series of **elif** / **then** commands can encode multiple conditional blocks.)

The resulting output has paired gene and CDS product names in separate columns on individual rows:

```
matK      maturase K
ATP2B1     ATPase 1 isoform 2
ATP2B1     ATPase 1 isoform 7
```

Compiled Go Programs

A program written in a compiled language is translated into a computer's native machine instruction code, and will run much faster than an interpreted script. Piping FASTA data to the basecount binary executable (compiled from the basecount.go source code file, below):

```
efetch -db nuccore -id J01749,U54469 -format fasta | basecount
```

will return rows containing an accession number followed by counts for each base:

```
J01749.1    A 983    C 1210    G 1134    T 1034
U54469.1    A 849    C 699     G 585     T 748
```

Google's Go language ("golang") has a straightforward build process that eliminates the complexity of typical package management systems. There are no complicated makefiles or separate header files. Circular dependencies between packages are not permitted.

Compilation of Go programs is fast, and error messages are clear and informative. For run-time failures (such as a nil pointer dereference, out-of-bounds array access, or thread deadlock), the error message includes the relevant stack traces with source code line numbers.

Programs in Go start with **package main** and then **import** additional software libraries (many included with Go, others residing in commercial repositories like github.com):

```
package main

import (
    "cmp"
    "utils"
    "fmt"
    "maps"
    "os"
    "slices"
)
```

Each compiled Go binary has a single **main** function, which is where program execution begins:

```
func main() {
```

The fsta **variable** is assigned to a data **channel** that streams individual FASTA records one at a time:

```
fsta := utils.FASTAConverter(os.Stdin, false)
```

The countLetters **subroutine** will be called with the identifier and sequence of each FASTA record:

```
countLetters := func(id, seq string) {
```

An empty counts **map** is created for each sequence, and its memory is freed when the subroutine exits:

```
counts := make(map[rune]int)
```

A **for** loop on the **range** of the sequence string visits each sequence letter. The map keeps a running count for each base or residue, with "++" incrementing the current value of the letter's map entry:

```
for _, base := range seq {
    counts[base]++
}
```

A sorted keys **array** is produced by calling **slices.SortFunc**. The alphabetical sort order is determined by the second argument, which is an **anonymous** function literal:

```
keys := slices.SortedFunc(maps.Keys(counts),
    func(i, j rune) int { return cmp.Compare(i, j) })
```

The sequence identifier is printed in the first column:

```
fmt.Fprintf(os.Stdout, "%s", id)
```

Iterating over the array prints letters and base counts in alphabetical order, with tabs between columns:

```
for _, base := range keys {
    num := counts[base]
    fmt.Fprintf(os.Stdout, "\t%c %d", base, num)
}
```

A newline is printed at the end of the row, and then the subroutine exits, clearing the map and array:

```
fmt.Fprintf(os.Stdout, "\n")
}
```

The remainder of the main function uses a loop to **drain** the fsta channel, passing the identifier and sequence string of each successive FASTA record to the countLetters function. The main function then ends with a final closing brace:

```
for fsa := range fsta {
    countLetters(fsa.SeqID, fsa.Sequence)
}
}
```

Save the following script to a file named **build.sh**, in the same directory as the **basecount.go** file. Adjust optional GOOS and GOARCH environment variables to cross-compile for a different platform:

```
#!/bin/bash

if [ ! -f "go.mod" ]
then
    go mod init "$( basename $PWD )"
    echo "replace eutils => $HOME/edirect/eutils" >> go.mod
    go get eutils
fi
if [ ! -f "go.sum" ]
then
    go mod tidy
fi
if [ ! -d "vendor" ]
then
    go mod vendor -e
fi

for fl in *.go do
    env GOOS=darwin GOARCH=arm64 go build -o "${fl%.go}" "$fl"
done
```

Other build flags can show escape analysis decisions or strip debugging symbols from the executable.

The build script creates module files used to track dependencies and retrieve imported packages. It also computes the path for finding the local **eutils** helper library included with EDirect. Set the Unix execution permission bit for the build script and compile the program(s) by running:

```
chmod +x build.sh
./build.sh
```

This produces a self-contained binary executable. With no reliance on local dynamic-link libraries, programs will not break when users update their development environments.

Additional Examples

EDirect examples demonstrate how to answer ad hoc questions in several Entrez databases. The detailed examples have been moved to a separate document, which can be viewed by clicking on the [ADDITIONAL EXAMPLES](#) link.

Summary

Entrez Direct continues to grow in popularity for simplifying access to Entrez data. It generates over half a billion EUtils service requests per year, with a peak of 15.8 million network hits on its busiest day. Over 180,000 users (measured by unique IP addresses) made EDirect queries in 2025, and it is referenced by more than 360 links in Google Scholar and over 225 citations in PubMed Central.

Recent increases in computer speed, processor count, and memory size, along with the transition to solid-state storage drives and improved file systems, provide an opportunity to simplify data analysis pipelines and add new capabilities in ways that would not have been practical even a few years ago.

EDirect local archives keep derived records, search indices, and cross-domain link tables together in an integrated system. Self-contained native binary executables eliminate external library load delays and version conflicts. The software is modular and easy to maintain, and it has evolved gradually through careful, selective refactoring.

This design helps EDirect avoid tangled package dependencies and sensitivity to network latency that can occur with other architectures. Absence of a third-party database eliminates common system administration and hosted infrastructure costs. The mature archive build system removes the need for redundant infrastructure development by every group that wants a searchable Entrez-style database.

Postings files use variable-byte-length encoding of UIDs to efficiently reference trillions of records. This ensures the archive's long-term ability to keep pace with the expanding scale of biological databases.

An archive can be built on a single well-configured personal computer in a reasonable amount of time without exceeding available memory. The 423-million-record peptide database takes around 1 hour for archiving and 11 hours for indexing and inversion on a Mac Studio M4 Max computer with 16 CPUs, 128 GB of internal RAM, and an external 4 TB Thunderbolt 5 SSD.

A university could run **archive-peptide-index** on an existing server to build a master copy of a searchable archive. Once finished, the **Archive**, **Data**, and **Posting** folders could be copied onto external solid-state drives (e.g., 2 TB Thunderbolt 3) provided by individual laboratories. By connecting an SSD to a lab's computer (and updating the configuration file), that lab will have unlimited use of the derived dataset, with the only expense being the purchase of one solid-state drive.

The ability to process a wide variety of scientific information (e.g., biochemical pathway components or mass spectrometry profiles) gives EDirect the potential to accelerate research through independent, cumulative data integration efforts. With these new local archives, pipelines can use **xsearch** queries to help look for unexpected but biologically relevant connections between records in different domains.

Additional search fields can be populated by importing curated references that map biological concepts (e.g., metabolic pathway classes) to an archive's source records. Using these in **xfilter** queries may help prevent combinatorial explosions when adapting multistep navigation protocols (11) for automated exploration tools that make discoveries by traversing a series of precomputed database links.

As EDirect begins to provide computable information from a growing set of public databases, scientists will be able to take those combined resources in directions that are limited only by their imaginations.

Appendices

Each EDirect program has a **-help** command that prints detailed information about available command-line arguments. These include **-sort** values for esearch, **-format** and **-mode** choices for efetch, and **-cmd** options for elink.

Info Data

Info field data contains status flags for several term list index properties:

```
<Field>
  <Name>ALL</Name>
  <FullName>All Fields</FullName>
  <Description>All terms from all searchable fields</Description>
  <TermCount>280005319</TermCount>
  <IsDate>N</IsDate>
  <IsNumerical>N</IsNumerical>
  <SingleToken>N</SingleToken>
  <Hierarchy>N</Hierarchy>
  <IsHidden>N</IsHidden>
  <IsTruncatable>Y</IsTruncatable>
  <IsRangable>N</IsRangable>
</Field>
```

Additional Elink Options

Elink has several additional modes that can be specified with the **-cmd** argument. When not using the default "neighbor" command ("neighbor_history" has been retired), elink will return an eLinkResult XML object, with the links for each UID presented in separate blocks. For example, the "neighbor" command:

```
esearch -db pubmed -query "Hoffmann PC [AUTH] AND dopamine [MAJR]" |
elink -related -cmd neighbor |
xtract -pattern LinkSetDb -element Id
```

will show the original PMID in the first column and related article PMIDs in subsequent columns:

```
1504781    11754494    3815119    1684029    14614914    12128255    ...
1684029    3815119    1504781    8097798    17161385    14755628    ...
2572612    2903614    6152036    2905789    9483560    1352865    ...
...
```

The "acheck" command returns all available link names for each record:

```
esearch -db pubmed -query "Federhen S [AUTH]" |
elink -cmd acheck |
xtract -pattern LinkSet -tab "\n" -element IdLinkSet/Id \
  -block LinkInfo -tab "\n" -element LinkName
```

printing each on its own line:

```
25510495
pubmed_images
pubmed_pmc
pubmed_pmc_local
pubmed_pmc_refs
pubmed_pubmed
```

```
pubmed_pubmed_citedin
...
```

The "prlinks" command can obtain the URL reference to the publisher web page for an article. The Unix "xargs" command calls elink separately for each identifier:

```
epost -db pubmed -id 22966225,19880848 |
efetch -format uid |
xargs -n 1 elink -db pubmed -cmd prlinks -id |
xtract -pattern LinkSet -first Id -element ObjUrl/Url
```

Nquire Arguments

The nquire program uses command-line arguments to obtain data from RESTful, CGI, or FTP servers. Queries are built up from command-line arguments. Paths can be separated into components, which are combined with slashes. Remaining arguments (starting with a dash) are tag/value pairs, with multiple values between tags combined with commas.

For example, a POST request (using -url):

```
nquire -url https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ espell.fcgi \
-db pubmed -term "vitamen" |
xtract -pattern eSpellResult -element CorrectedQuery
```

returns the corrected spelling of "vitamen":

```
vitamin
```

while a GET query:

```
nquire -get https://eutils.ncbi.nlm.nih.gov/entrez/eutils/ einfo.fcgi -db pubmed |
xtract -pattern DbInfo -block Field -element Name FullName -deq "\n"
```

returns information about the fields indexed for the PubMed database:

```
ALL      All Fields
UID      UID
FILT     Filter
TITL     Title
MESH     MeSH Terms
MAJR     MeSH Major Topic
...
```

and an FTP request:

```
nquire -ftp ftp.ncbi.nlm.nih.gov pub/gdp ideogram_9606_GCF_000001305.14_850_V1 |
grep acen | cut -f 1,2,6,7 | awk '/^X\t/'
```

returns data with the (estimated) sequence coordinates of the human X chromosome centromere (here showing where the p and q arms meet):

```
X      p      58100001      61000000
X      q      61000001      63800000
```

Nquire can also produce a list of files in an FTP server directory:

```
nquire -lst ftp://nlmpubs.nlm.nih.gov online/mesh/MESH_FILES/xmlmesh
```

or a list of FTP file names preceded by a column with the file sizes:

```
nquire -dir ftp.ncbi.nlm.nih.gov gene/DATA
```

Finally, `nquire` can download FTP files to the local disk:

```
nquire -dwn ftp.nlm.nih.gov online/mesh/MESH_FILES/xmlmesh desc2026.zip
```

If Aspera Connect is installed, the `nquire -asp` command will provide faster retrieval from NCBI servers:

```
nquire -asp ftp.ncbi.nlm.nih.gov pubmed baseline pubmed26n0001.xml.gz
```

Without Aspera Connect, `nquire -asp` defaults to using the `-dwn` logic.

References

1. Schuler GD, Epstein JA, Ohkawa H, Kans JA. Entrez: molecular biology database and retrieval system. *Methods Enzymol.* 1996. [https://doi.org/10.1016/s0076-6879\(96\)66012-1](https://doi.org/10.1016/s0076-6879(96)66012-1). (PMID 8743683.)
2. Hutchins BI, Baker KL, Davis MT, Diwersy MA, Haque E, Harriman RM, Hoppe TA, Leicht SA, Meyer P, Santangelo GM. The NIH Open Citation Collection: A public access, broad coverage resource. *PLoS Biol.* 2019. <https://doi.org/10.1371/journal.pbio.3000385>. (PMID 31600197.)
3. Lin J, Wilbur WJ. PubMed related articles: a probabilistic topic-based model for content similarity. *BMC Bioinformatics.* 2007. <https://doi.org/10.1186/1471-2105-8-423>. (PMID 17971238.)
4. Wu C, Macleod I, Su AI. BioGPS and MyGene.info: organizing online, gene-centric information. *Nucleic Acids Res.* 2013. <https://doi.org/10.1093/nar/gks1114>. (PMID 23175613.)
5. Ostell JM, Wheelan SJ, Kans JA. The NCBI data model. *Methods Biochem Anal.* 2001. <https://doi.org/10.1002/0471223921.ch2>. (PMID 11449725.)
6. den Dunnen JT, Dalgleish R, Maglott DR, Hart RK, Greenblatt MS, McGowan-Jordan J, Roux AF, Smith T, Antonarakis SE, Taschner PE. HGVS Recommendations for the Description of Sequence Variants: 2016 Update. *Hum Mutat.* 2016. <https://doi.org/10.1002/humu.22981>. (PMID 26931183.)
7. Holmes JB, Moyer E, Phan L, Maglott D, Kattman B. SPDI: data model for variants and applications at NCBI. *Bioinformatics.* 2020. <https://doi.org/10.1093/bioinformatics/btz856>. (PMID 31738401.)
8. Wei C-H, Allot A, Leaman R, Lu Z. PubTator central: automated concept annotation for biomedical full text articles. *Nucleic Acids Res.* 2019. <https://doi.org/10.1093/nar/gkz389>. (PMID 31114887.)
9. Mitchell JA, Aronson AR, Mork JG, Folk LC, Humphrey SM, Ward JM. Gene indexing: characterization and analysis of NLM's GeneRIFs. *AMIA Annu Symp Proc.* 2003:460-4. (PMID 14728215.)
10. Hurwitz DI, Han L, Geer LY. Searching by index for similar sequences: the SEQR algorithm. *arXiv.* 2018. <https://arxiv.org/pdf/1811.00931>.
11. Xin J, Afrasiabi C, Lelong S, Adesara J, Tsueng G, Su AI, Wu C. Cross-linking BioThings APIs through JSON-LD to facilitate knowledge exploration. *BMC Bioinformatics.* 2018. <https://doi.org/10.1186/s12859-018-2041-5>. (PMID 29390967.)

Release Notes

EDirect release notes describe the history of incremental development and refactoring, from the original implementation in Perl, through the redesign in Go and shell script, and to the maturation of the local archive. The detailed notes have also been moved to a separate document, which can be accessed through the [RELEASE NOTES](#) link.

For More Information

Announcement Mailing List

NCBI posts general announcements regarding the E-utilities to the [utilities-announce announcement mailing list](#). This mailing list is an announcement list only; individual subscribers may **not** send mail to the list. Also, the list of subscribers is private and is not shared or used in any other way except for providing announcements to list members. The list receives about one posting per month. Please subscribe at the above link.

Documentation

EDirect navigation functions call the URL-based Entrez Programming Utilities:

<https://www.ncbi.nlm.nih.gov/books/NBK25501>

NCBI database resources are described by:

<https://www.ncbi.nlm.nih.gov/pubmed/37994677>

Information on how to obtain an API Key is described in this NCBI blogpost:

<https://ncbiinsights.ncbi.nlm.nih.gov/2017/11/02/new-api-keys-for-the-e-utilities>

An introduction to shell scripting for non-programmers is at:

<https://missing.csail.mit.edu/2020/shell-tools/>

An article on the Go programming language, written by its creators, is at:

<https://cacm.acm.org/research/the-go-programming-language-and-environment/>

and transcripts of talks on design philosophy and retrospective experience of Go are at:

<https://commandcenter.blogspot.com/2012/06/less-is-exponentially-more.html>

<https://commandcenter.blogspot.com/2024/01/what-we-got-right-what-we-got-wrong.html>

Instructions for downloading and installing the Go compiler are at:

<https://golang.org/doc/install#download>

Additional NCBI website and data usage policy and disclaimer information is located at:

<https://www.ncbi.nlm.nih.gov/home/about/policies/>

Public Domain Notice

A copy of the NCBI Public Domain Notice, which applies to EDirect, is shown below:

PUBLIC DOMAIN NOTICE
National Center for Biotechnology Information

This software/database is a "United States Government Work" under the terms of the United States Copyright Act. It was written as part of the author's official duties as a United States Government employee and thus cannot be copyrighted. This software/database is freely available to the public for use. The National Library of Medicine and the U.S. Government have not placed any restriction on its use or reproduction.

Although all reasonable efforts have been taken to ensure the accuracy

and reliability of the software and data, the NLM and the U.S. Government do not and cannot warrant the performance or results that may be obtained by using this software or data. The NLM and the U.S. Government disclaim all warranties, express or implied, including warranties of performance, merchantability or fitness for any particular purpose.

Please cite the author in any work or product based on this material.

Getting Help

Please refer to the [PubMed](#) and [Entrez](#) help documents for more information about search queries, database indexing, field limitations and database content.

Suggestions, comments, and questions specifically relating to the EUtility programs may be sent to eutilities@ncbi.nlm.nih.gov.

Entrez Direct Release Notes

Jonathan Kans, PhD¹

Created: April 23, 2013; Updated: October 8, 2026.

EDirect was conceived in 2012, prototyped in Perl, and released to the public in 2014.

Xtract was subsequently rewritten in the compiled Go programming language, for a hundredfold speed improvement on modern multi-processor computers. Transmute and rchive are also provided as platform-specific Go executables.

A major refactoring, for ease of maintenance and long-term stability, was completed in 2020. EDirect now exists as a set of Unix shell scripts plus the trio of utilities implemented in Go. The original Perl code has been retired.

2026

Version 26.4: October 08, 2026

- Added directory synchronization after file rename for local archive on Linux.
- Xtract -element "@@" shows attribute names and values, "@@@" shows <name>value</name> form.

Version 26.3: September 23, 2026

- Recompiled executables with Go 1.27.
- Variable-length decoding counts end-of-integer bytes (MSB = 0) to preallocate array of correct size.
- Fixed spurious message in local archive test for rotating hard drives on Linux.
- Local archive on Linux internally sets -ceph flag to handle file server latency.

Version 26.2: August 24, 2026

- Local PMC archive uses last legacy source file release on FTP site.
- Sequence record filter example updated to "dbxref flybase [TEXT]" from "[PROP]".

Version 26.1: August 6, 2026

- New -ceph flag (undocumented) controls additional locks needed for local archive build on a Ceph file system that includes hard disk drives.
- Local archive build process now retries the RES step if any modified archive files were not reconstructed.
- Removed retired gap, gapplus, and grasp databases from test-eutils.
- Transmute -plain and eutils.TransformAccents no longer absorb ASCII backspace through carriage return characters.

Version 26.0: July 13, 2026

- Added file locking, file synchronization, and a mutex-protected map of in-progress file names to prevent race conditions when building a local archive on a Ceph distributed file system.

Version 25.9: June 9, 2026

- Improved speed of local archive inverted index collection step.
- Added initial support for huge databases to use longer padded UID strings.
- Removed deprecated filter-record script (original name of filter-records).

Version 25.8: June 2, 2026

- Posting UIDs promoted to unsigned 64-bit integers (variable byte length encoded in files).

Version 25.7: June 1, 2026

- Implemented variable byte length encoding of UIDs in postings files.
- Improved eutils.TranslateCdRegion efficiency.
- Remove warning about left-over tildes if no phrase matched the proximity query.
- Allow PENT field to be hyphenated in future with name of PAM or BLOSUM matrix used for indexing.

Version 25.6: May 21, 2026

- Updated archive-nihocc helpers to handle new data file convention.
- Minor refactor in preparation for variable byte length encoded UIDs.

Version 25.5: May 12, 2026

- Added split-records to split text files -by-num or -by-len, based on xtract -split-by- commands for XML sets.
- Pluralized filter-record to filter-records for consistency with split-records.
- Deprecated filter-record script prints warning that it will be removed in next release.
- Restored strings.Builder in PartitionText, accidentally deleted when simplifying from PartitionXML.

Version 25.4: April 27, 2026

- Xtract -split-by-id article-id -prefix PMC handles old and new capitalizations.
- Updated PMC download address to /pub/pmc/deprecated/oa_bulk.
- Local archive indexer/inverter code adjusted for high performance and low memory pressure.

Version 25.3: April 7, 2026

- Xfetch -all unpacks and streams all records in a local archive.
- Using -custom flag with archive-pubmed or archive-pmc builds local subset from Entrez query.
- Placing an edirect.ini file inside edirect folder allows quick testing of local archive configuration.

- Updated pmc2info to handle both old and new article-id convention.
- Added [SLEN] and [MLWT] fields to peptide database local archive.

Version 25.2: March 9, 2026

- Recompiled executables with Go 1.26.1.
- Efilter drops use of Entrez history.

Version 25.1: February 5, 2026

- Elink no longer uses the Entrez history mechanism to store intermediate results.
- By default, elink samples 10,000 evenly-spaced UIDs, in chunks of 50 at a time, to avoid server stress.
- Use elink -complete flag to override default sampling and link every input UID, still in chunks of 50.

Version 25.0: January 21, 2026

- EDirect will automatically start using SOLR limit workaround for PMC when new server is deployed.
- Raised chunk size to increase retrieval speed of document summaries for pubmed and sequence databases.

Version 24.9: January 11, 2026

- Preparation for PubMed Central switch to SOLR server.

Version 24.8: January 10, 2026

- EDirect now stores 10,000 individually-compressed records from a specific UID range, preceded by a byte offset index, in each local archive file.
- New archive design gave xfetch another order of magnitude speed improvement for bulk record retrieval compared to efetch.

2025

Version 24.7: August 25, 2025

- Recompiled executables with Go 1.25.
- Local archive subproject folders now use extern/{database}-{subproject} naming convention.
- Xlink -ranked produces same output as xsearch -match
- Xtract -indexer only removes stop words for specific text fields.
- Reorganized and expanded EDIRECT.pdf and its plain text README.

Version 24.6: August 12, 2025

- Xtract allows multiple -if clauses within a single exploration, but only for printing.
- Retired deprecated xtract exploration variants with initial capital letters.
- Added getMixedContentString function to consolidate -mixed mode support.

Version 24.5: August 7, 2025

- Xsearch -match "{sequence} [PENT]" constructs overlapping pentamers.
- Local archive code uses os.Stat to check for zero-length record files.
- Added test for empty slice to avoid deadlock on partially erased archive.
- Rchive -link now precedes -merge or -promote to build local archive link fields.
- Refactored local archive build system to use task-specific helper scripts.

Version 24.4: July 8, 2025

- Xtract conditional test supports question mark (for object tag) and tilde (for object value).
- Simplified gff2xml script using -if test on question mark in heterogeneous record exploration.
- Fixed xtract caret prefix support (for element depth).
- Transmute -gbf (or filter-genbank) now supports -min and -max arguments.
- Transmute -normalize pmc internally sets -mixed flag.

Version 24.3: July 3, 2025

- Renamed xtract record partitioning commands to -split-by-num and -split-by-len.
- Extraction by filter-record -min / -max range now ignores leading text.
- Added -info flag to archive-nlmnlp and archive-nihocc scripts to check for out-of-date release files.
- Updated path and file names for archive-nlmnlp release data.

Version 24.2: June 20, 2025

- Refactored and consolidated local archive path configuration code for better efficiency and easier maintenance.
- PubMed Central converter pmc2info creates separate SECTION/TEXT containers for subsection titles.
- Modified gff-sort to support records using Name instead of ID in GFF3 Attributes column.
- Partitioning records with xtract -split (by number) or -allot (by size) starts with file number 001.

Version 24.1: June 3, 2025

- Reverted prep-taxoninfo.go after the new "realm" column was removed from rankedlineage.dmp.
- Moved xfetch.ini and xlink.ini to new config subfolder.

Version 24.0: May 22, 2025

- Xlink uses extensible xlink.ini configuration file to determine destination database from -db and -target.
- Xfetch removes default values, obtains wrapper field names only from xfetch.ini configuration file.

Version 23.9: May 12, 2025

- Nquire -asp supports conventions of free Aspera Connect 4.2 client.

- Xtract -prose and -indexer separate superscripts or subscripts from text if adjacent characters are both letters or both digits.
- Added -download, -verify, and -missing commands to archive-pubmed and archive-pmc scripts.
- Align-columns -g flag takes "t" argument to keep tabs as column separator.

Version 23.8: April 8, 2025

- Added eutils.ParentsToLineage function to generate column for sorting a hierarchy into proper order.
- Transmute -p2l calls ParentsToLineage function.
- Added gff-sort script to put GFF3 lines in hierarchical order for single-pass processing.
- Modified prep-taxoninfo.go for separate "realm" column in the rankedlineage.dmp file.
- Using -element "tilde" in a heterogeneous exploration unescapes html-encoded characters in anonymous objects.
- The join-into-groups-of scripts defaults to groups of 10000 lines if no argument is given.

Version 23.7: March 27, 2025

- Efetch -format asn also sets -immediate flag to avoid unnecessary delays on large records.
- Xtract -mimics command replaces non-alphanumeric characters with space before running substring test.
- Several internal xtract maps consolidated into a structure argument.
- Removed deprecated fetch-*, stream-*, and phrase-search scripts.
- Replaced remaining instances of deprecated io.ioutil file read functions.

Version 23.6: March 3, 2025

- Recompiled executables with Go 1.24.
- Local PubMed archive indexes Chemical/NameOfSubstance as [SUBS] field.
- Gbf2xml creates /calculated_mol_wt qualifier from CDS /translation sequence.
- Obsolete fetch-*, stream-*, and phrase-search scripts now print deprecation warnings.

Version 23.5: February 19, 2025

- Added xinfo, xsearch, xlink, xfilter, and xfetch navigation scripts to replace phrase-search for local archive access.
- Local (xsearch) and remote (esearch) navigation scripts interoperate by using same ENTREZ_DIRECT message.
- Term position indices now use variable byte length encoding to efficiently support PubMed and PMC.
- New environment variable code simplifies configuration of directories used for local archive.
- File referenced by EDIRECT_LOCAL_CONFIG still takes precedence over individual path variables.
- Xfetch can use extensible xfetch.ini configuration file for database-specific arguments (e.g., doctype).

- Added date argument support in local archive for fetching results from delayed SOLR network query.
- Added download-flatfile -ftp and -https flags to select GenBank format download method.
- Added illustrative figures to PDF version of readme documentation file.
- Added "-with -split" and "-transform -translate" examples to readme.pdf.
- Fixed bug when archive scripts checked for new annual release and the source folder was already empty.
- Fixed -mindate integer conversion typo added during move to separate ecollect process.
- Corrected error message that reports use of internal -retmax argument.
- Removed test-eutils references to retired popset database.

Version 23.4: January 3, 2025

- Xtract subrange uses 32-bit Unicode to avoid cutting in middle of multi-byte UTF-8 character.
- Xtract exploration argument now accepts ampersand-prefixed variables (undocumented).

2024

Version 23.3: December 23, 2024

- Filter-record adds -min and -max arguments to isolate records by position without parsing.
- Fixed bug parsing XML comment in PMC that starts with right angle bracket.

Version 23.2: December 18, 2024

- Efetch sequence retrieval supports "-format asn" and "-mode binary" arguments.
- Esearch sorts and uniques ErrorList elements to suppress multiple PhraseNotFound warnings.
- Transmute -outline accepts an optional integer argument for the maximum depth to show.
- Added Interactions Table example to Gene section of Additional Examples document.

Version 23.1: November 20, 2024

- Ecollect helper script retrieves current UIDs in separate process to avoid internal state conflicts.
- Efilter after esearch in PubMed adjusts <Query> or date values in ENTREZ_DIRECT message.
- Elink into history splits input into chunks, but now truncates each chunk to avoid backend server overflow, while still sampling from the full range of input IDs. Keeping results in history allows efficient modification by efilter.
- Added elink -cmd uid to process in small chunks without truncation and return a comprehensive list of link UIDs. Subsequent filtering of results can be done outside of history with the intersect-uid-lists script.
- Added EDIRECT_NO_RETRY and EDIRECT_NO_ERRORS environment variables.
- Test-eutils -alive runs one elink query 10 times to look for failing servers.
- Combining Independent Queries example uses intersect-uid-lists instead of history '#' convention.

Version 23.0: October 28, 2024

- Elink SOLR workaround uses history for results if target is not PubMed, even if linking from PubMed.
- Gbf2xml now recognizes qualifier continuation lines that start with an unexpected slash character.
- Gbf2info wraps qualifiers in a tag based on the feature key for easy selection by type.
- Fetch-pubmed restores -turbo functionality by passing "\$@" arguments to common fetch-local script.

Version 22.9: October 21, 2024

- Added retry and verify steps to improve download-flatfile script reliability.
- Gbf2xml gains 3-fold speed improvement on release files with multithreading.
- Archive-pubmed indexes "Bad Reference [PROP]" if citation PMID is longer than 8 digits.
- Run "export EDIRECT_TRACE=true" to enable "set -x" tracing.
- All composite literals in code are now created with explicit field keys.
- Rearranged structure field order to minimize allocated object size.

Version 22.8: October 10, 2024

- Lowered efetch retrieval chunk size for -db sra and -format runinfo.
- Xtract adds -consists-of {string of allowed characters} conditional test.
- Archive-pubmed indexes "Suspicious Date [PROP]" if MedlineDate resembles split PMID.
- Xtract -year "PubDate/*" ignores MedlineDate artifact.
- Rchive -join leaves fusion of adjacent inverted records with same identifier to subsequent -merge step.
- Archive scripts -erase flag controls deletion of Extras directory contents.
- Renamed archive-ncbinlp to archive-nlmnlp due to internal reorganization.
- Additional identifiers recognized by xml2fsa and xml2tbl for patent submissions.
- Added download-flatfile variant of download-sequence script.

Version 22.7: September 30, 2024

- Efilter places parentheses around -query argument clause to "AND" with history key.
- Efilter detects an empty list piped in from a previous step and exits without an error message.
- Xtract -bkt shortcut wraps data in parsable FASTA define bracketed fields.
- Added gbf2facds script to generate fasta_cds_na and fasta_cds_aa formats from GenBank files.
- Updated list of Unicode spaces for HasBadSpace and CleanupBadSpaces functions.
- Xtract -mask sets objects to exclude from printing with -element "*" (undocumented).
- PMCInfo generator excises table and formula contents, and normalizes Unicode space characters.

Version 22.6: September 12, 2024

- Updated URL for efetch -format bioc with -db pubmed or -db pmc.
- ASN.1 to XML conversion removes non-printing characters.
- Refactored local archive incremental inversion for faster performance.
- Internal join step collects incremental inversions for final merging.
- Added archive-pids to support phrase-search -link PMCID after archive-pubmed.

Version 22.5: August 29, 2024

- Elink creates new web environment to avoid history overflow.
- Improved speed of accession to integer identifier conversion.
- Recompiled executables with Go 1.23.
- Updated go.mod and go.sum module files for Go executables.
- Adopted new range-over-function style in incremental for-loops.
- Transitioned from sort.Slice to slices.Sort family for simpler code.
- Using maps.Keys iterator as input for slices.Sorted.
- Term sort logic moved to CompareAlphaOrNumericKeys function.

Version 22.4: July 17, 2024

- Efetch -db nuccore -format docsum now works on VDB accessions.
- Elink adjusts batch size to remain within history element limits.

Version 22.3: July 1, 2024

- Elink record count bug fixed in code for the SOLR workaround.
- Updated go.mod and go.sum module files for Go executables.
- Local archive-ncbinlp and archive-nihocc builds now require local Go compiler.

Version 22.2: June 24, 2024

- Efilter supports both -country and -geo_loc_name shortcuts.
- Xtract -insd accepts country and geo_loc_name arguments.

Version 22.1: May 13, 2024

- Xtract and transmute high-level functions remove leading Byte Order Marks.
- Transmute -bom removes Byte Order Mark at the beginning of strings.
- Phrase-search -match returns all UIDs that are matched more than once.
- Replaced just-first-key script with just-top-hits plus numeric argument.

Version 22.0: April 30, 2024

- Recompiled executables with Go 1.22.
- Restored ability to use `-len` with `"*"` to get length of XML record.
- Added `SIZE` index to `archive-pubmed` and `archive-pmc`.

Version 21.9: April 16, 2024

- Added support for 64-bit Linux-aarch64 variant of 32-bit Linux-`*arm*`.
- Consolidated extraction customization arguments into a structure.

Version 21.8: April 4, 2024

- `Xtract -split` takes substring delimiter from `-with` argument.
- `Xtract -cds2prot` adds explicit `-frame0` and `-frame1` to default (1-based) `-frame`.
- `Transmute -cds2prot` uses `-gcode` for consistency with `xtract -cds2prot`.
- `Transmute -replace` also adds explicit `-offset0` and `-offset1` to default (0-based) `-offset`.
- `Transmute -r2p verify` does full lookup if original PMID is in "erratum for", etc., exclusion list.
- Improved phrase-search logic for recognizing parentheses inside chemical names.
- `Archive-pubmed` and `archive-pmc` add `-info` flag to check for out-of-date release files.

Version 21.7: March 21, 2024

- `Xtract -molwt` variants `-molwt-m` and `-molwt-f` retain the initial methionine or fMet.
- `Xtract -pkg` exploration idiosyncrasy is now bypassed internally.
- Added `bsmp2info` for easier extraction of BioSample data.
- Experimental `gbf2info` script is alternative to `xtract -insdx flatfile` field extraction.
- New systematic-mutations script can follow `disambiguate-nucleotides`.
- `Archive-pubmed` warns if release files for previous years are present.
- Added indexed DOI field to local PubMed archive, with mirror-image terms.
- Phrase-search reverses order of characters in query term for DOI field.
- Phrase-search recognizes and skips parentheses inside chemical names.
- Jaccard test for best citation match candidate no longer removes stop words.
- Citation matcher excludes records marked "erratum for", "retraction in", etc.

Version 21.6: March 5, 2024

- Navigation command `-help` functions report if new version of EDirect is available.

Version 21.5: February 20, 2024

- Added multi-step transformations and sequence analysis sections to readme.pdf.
- Xtract checks for commands starting with a Unicode dash instead of an ASCII hyphen.
- Xtract -insdx convenience function wraps -insd table in XML.
- Transmute -fasta reformats sequences into fixed-width lines.

Version 21.4: February 12, 2024

- Xtract variables can now record the result of an -element variant on an object.
- Xtract -numeric only accepts items that are entirely digits.
- Xtract -avg joined by -geo, -hrm, and -rms integer mean functions.
- Documented xtract -lge, lg2, and -log integer logarithm functions.
- Xtract -med for even-sized lists now uses the average of the middle elements.
- Preferred environment variable name is EDIRECT_LOCAL_ARCHIVE.
- New BioSample section added to the Additional Examples web page.

Version 21.3: January 29, 2024

- Nquire accepts leading -http-version, -content-type, and -user-agent arguments.
- Nquire -datasets implements GET queries using RESTful arguments.
- Added jsonl2xml script to convert JSON Lines output from NCBI Datasets.
- Added refseq-nm-cds to consolidate and replace earlier scripts.
- Use align-columns -a "w" to report widths of each column.
- Increased tbl2xml buffer size to handle titin protein and cDNA sequences.
- Removed retired homogene database from test files.

Version 21.2: January 17, 2024

- Phrase-search -terms and -totals now implemented in compiled code.
- Local archive positional indices no longer treat integers as stop words.
- Navigation commands now map -db nucleotide, which has no links, to nuccore.
- Nquire -citmatch shortcut extracts matching PMID from JSON result.

2023

Version 21.1: December 27, 2023

- New -zap argument in archive-pubmed and archive-pmc removes ALL files.
- Added refseq-nm-subseq to extract coding region sequences from FTP release files.

- Added refseq-nm-starts to get CDS offsets needed by hgvs2spdi in advance.

Version 21.0: December 20, 2023

- Archive-pubmed time delay increases before each failed download retry attempt.

Version 20.9: December 6, 2023

- Archive-pubmed no longer needs year of MeSH and journal data files.
- Transmute -cds2prot -all adds -orf flag, with capital letters marking start codon products.
- Transmute -gbf extended with -accession(s), -taxid(s), and -organism filter arguments.
- Transmute -gbf includes -require and -exclude arguments to filter based on string value.
- Transmute -txf uses -pattern to identify start of text record for filtering.
- Added filter-genbank and filter-record shortcut scripts for transmute -gbf and -txf.
- Esearch recognizes -geo_loc_name, which will replace the -country shortcut.
- Phrase-search -match queries individual terms and sorts by number of hits.
- Added just-first-key script to return lines where first column matches value in first row.
- Added indexed KYWD field to local PubMed archive.
- Added -even and -odd element variants to xtract.
- Renamed Clean functions to Normalize, to avoid confusion with Cleanup methods.
- Removed code for indexing retired Global Network of Biomedical Relationships project.

Version 20.8: November 13, 2023

- Continued refactoring to allow extension of local archive without modifying base source code.
- Code for downloading original records from network now contained in each archive script.
- Xtract instructions for indexing local records also reside within archive scripts.
- NCBI NLP terms and NIH OCC links are indexed by Go language source code "scripts".
- Archive scripts use "go run" to dynamically compile and execute the stand-alone Go programs.
- Large Unicode-to-ASCII mapping files moved to data subfolder.

Version 20.7: November 6, 2023

- Refactored local archive creation scripts for ease of future expansion.
- Split archive-ncbinlp and archive-nihocc out of archive-pubmed script.
- Added idx-grant helper script for custom-index.

Version 20.6: October 20, 2023

- EDirect suppresses "root" email name to compensate for server issue.

- Nquire clears bold red text highlights after reporting file download failure.
- Xtract -insd recognizes geo_loc_name source qualifier, which will replace country.
- New mask argument in eutils.VisitElements blocks recursive exploration.
- Inverted index subdirectory groups now controlled by argument.
- Added StringToXML function to consolidate common logic for several format converters.
- Refactored fetch-local and stream-local to simplify record retrieval code.

Version 20.5: September 27, 2023

- Transmute -f2x wraps FASTA stream in simple XML.
- Transmute -cds2prot adds -max for FASTA line length, and -all for six-frame translation.
- Xtract -gcode and -frame values used by -cds2prot protein translation.
- Xtract -pept splits amino acid runs at *, -, x, or X.
- Changed baseToIdx from map to array to speed up translation time.

Version 20.4: September 15, 2023

- Improved xtract internal conversion speed of JSON, ASN.1, and GenBank formats.
- Efetch -db pubmed regains ability to use -format asn.1.
- Phrase-search -debug prints environment variables, tests PubMed record fetch.
- Added toml2xml script to parse TOML configuration file contents into XML.
- All genetic code translation tables are precomputed in gdata.go source file.

Version 20.3: September 6, 2023

- Navigation functions print warning for unrecognized argument as well as unrecognized option.
- Xtract -indexer uses -wrp value as field name for positional indices.
- Xtract -indexer supersedes earlier field-specific positional indexing functions.
- Xtract -pentamers generates 5-letter overlapping sequence segments.
- Local archive 16-bit position values are now unsigned to contain longest protein sequence.

Version 20.2: August 22, 2023

- Recompiled executables with Go 1.21.
- Reverted archive-pubmed and archive-taxonomy to use FTP by default.
- Added archive-pubmed -stem flag to optionally populate the STEM indexed field.
- Xtract -journal improves journal title capitalization for APA converter.
- Xtract "[^]" element construct handles truncation guide strings with vertical bars.
- Added yaml2xml script to parse YAML configuration file contents into XML.

- Added YR (year) and DT (date) variables to print-columns and filter-columns scripts.

Version 20.1: July 31, 2023

- Refactored ERROR and WARNING messages to consolidate highlighting code.

Version 20.0: July 17, 2023

- EDirect navigation shell scripts now use bash interpreter instead of sh, since SOLR workaround needs bash "local" variables in recursive function.
- Archive-pmc adds -https and -ftp download flags, default reverted to FTP (Aspera Connect, if present).
- Common PMC archive wrappers for TEXT include TITLE, ABSTRACT, INTRODUCTION, MATERIALS, METHODS, RESULTS, DISCUSSION, and CONCLUSION, with PARAGRAPH for unrecognized.
- Select abstract paragraphs with xtract -pattern PMCInfo -sep "\n" -element ABSTRACT/TEXT.

Version 19.9: June 30, 2023

- Esearch -db pubmed with both -query and -mindate/-maxdate works with SOLR server.
- Efetch -db gene -format xml retrieves in chunks of 200 to avoid server errors.
- Nquire on Cygwin uses wget by default, otherwise prefers curl.

Version 19.8: June 12, 2023

- Local archive scripts for pubmed, pmc, and taxonomy use HTTPS by default.
- Archive first tries EDIRECT_LOCAL_CONFIG environment variable to find INI file.
- Phrase-search author queries automatically apply appropriate wildcard truncation.
- Disable middle initial expansion with a dash, e.g., "Jefferson T- [AUTH]".

Version 19.7: May 31, 2023

- New environment variables (EDIRECT_LOCAL_MASTER and EDIRECT_LOCAL_WORKING) point to common volumes with pubmed, pmc, and taxonomy subfolders, superseding database-specific variables.
- Archive-pubmed PDAT field replaced by DATE, has better consistency with older records.
- Added ini2xml script to parse INI configuration file contents into XML.

Version 19.6: May 25, 2023

- Switched archive-pmc script to use PMC bulk retrieval service with daily updates.
- PMC title, abstract, and regular text paragraphs are indexed in PMCInfo object.

Version 19.5: May 8, 2023

- Database is case-insensitive to prevent -db PMC, SNP, or SRA from bypassing normalization.

Version 19.4: April 24, 2023

- Added archive-taxonomy to build local index of NCBI taxonomy data.

- Integrates data from new_taxdump.tar.gz release and introduces new TaxonInfo record.
- Initial taxoninfo.xml product can also be analyzed by xtract.
- Still need to import files containing type material and biological host information.
- Use phrase-search -db taxonomy for queries, fetch-taxonomy to retrieve records.
- Added fetch-local, which takes a -db argument, to start reversing script proliferation.

Version 19.3: April 17, 2023

- XML parsing tolerates <? processing instructions ?> with embedded HTML error messages.
- Xtract -tag, -att, -atr, -cls, -slf, and -end commands added for stepwise attribute construction.
- Xtract -tag sets internal -wrp flag to reencode angle brackets and ampersands in new XML.
- Xtract exploration on "*" searches inside current object without needing to be given its name.
- Raised default maximum nesting depth of xtract -verify to allow complex MathML markup in PMC.
- Added xtract -verify -max argument to control reporting depth threshold.
- Xtract -test reports visible Combining Accent strings and invisible Unicode characters.
- Idx-errors helper for custom-index adds ERRS field for records with problematic characters.
- Added pma2apa script to convert PubmedArticle XML to APA format.
- Updated download-ncbi-software -datasets URL address to v2.
- For Apple Silicon, install-edirect.sh script also retrieves Darwin (x86) executables.
- Installation script will place new edirect folder at head of PATH, to bypass any older version.
- Installation command and installation script use HTTPS instead of FTP.

Version 19.2: March 23, 2023

- Xtract -insd accepts GO_component, GO_function, and GO_process qualifiers.

Version 19.1: March 16, 2023

- Esearch -db pubmed saves <Query> item in ENTREZ_DIRECT message even for small result counts.
- The -quick argument reverses this, forcing use of history mechanism regardless of SOLR truncation.
- Elink performs additional -target and -related argument reality checks.
- Xtract detects multiple -if or -unless conditional arguments in same exploration block.

Version 19.0: February 16, 2023

- SOLR workaround uses "all [SB] NOT pubmed books [SB]" to avoid sorting artifact.
- Esearch -query removes apostrophes in text before processing.
- Xtract -unit PubDate -date "*" works with document summaries.

Version 18.9: February 12, 2023

- Maximum PMID determination for SOLR workaround compensates for "all [SB]" sorting artifact.
- Archive-pubmed -https -index downloads extra data files without use of ftp protocol.
- Archive-pubmed adds initial support for zero-padded UID index.

Version 18.8: January 27, 2023

- SOLR workaround uses binary search on PMID range instead of sliding CRDT window.
- Local PubMed archive adds "conflict of interest statement" to PROP field.
- Moved PublicationType out of PROP and into new PTYP field.
- Added RDAT field for DateRevised in local archive.
- New archive-pubmed -ftp and -https flags select release file download method.
- Added JOUR and SRC fields to archive-pmc for journal capture.
- Esearch -tranquil suppresses "No items found" error, skips automatic retries.
- Xtract -sort expanded to -sort-fwd and -sort-rev variants.
- Xtract -initials parses initials from first and middle names.

Version 18.7: January 4, 2023

- Xtract -unit PubDate -date "*" construct returns date in YYYY/MM/DD format.
- Local PubMed archive adds PDAT index for wildcard searching on publication date.
- Added archive-pmc and fetch-pmc scripts for initial support of local PMC archive.
- Phrase-search -db pmc selects use of local PMC search index.
- PMC indexes positional TITL, ABST, and TEXT fields, plus AUTH and YEAR.

2022

Version 18.6: December 20, 2022

- Refactored local archive incremental indexing, reusing existing XML multi-threaded processing components for simpler code and faster performance.

Version 18.5: December 12, 2022

- EUtils queries check XML results for messages packaged in WarningList.
- Epost prints XML result on failure to combine loaded components.
- Esearch no longer re-encodes double quote to " with SOLR server.
- Archive-pubmed relocates Sentinels directory inside Archive folder.
- Download-pubmed adds a second retry attempt, reports failure.

- Erase and repopulate local archive now that 2023 PubMed is released.

Version 18.4: December 5, 2022

- Warning printed if expected and actual counts from SOLR backend disagree.
- Added -verbose flag to show details of SOLR truncation workaround.
- Elink -cited and -cites instantiate results in message due to epost issue.
- Phrase-search -link CITED and CITES logic improved for faster lookup.

Version 18.3: December 2, 2022

- Renamed recent colliding variable in SOLR truncation limit workaround code.
- Changed SOLR code to work with Debian and Ubuntu shell variants.
- Local links fields now index zero-padded PMIDs to prepare for speed improvement.
- Download-ncbi-data retrieves 2023 release of MeSH files.

Version 18.2: November 23, 2022

- Code modified to support newly-deployed SOLR backend for PubMed database.
- Added -quick flag to esearch, elink, and efetch to override truncation limit workaround.
- Added -chunk argument to elink and efetch to control internal batch size.
- Nquire -len returns Content-Length of HTTP file without downloading.
- Xtract -prose removes HTML decorations and converts newlines to spaces in PMC paragraphs.
- Xtract -sort now handles unsigned real numbers with a decimal point.
- Archive-pubmed -nihocc adds CITED and CITES link fields, from NIH Open Citation Collection data.
- Phrase-search -link CITED or CITES looks up links for a set of PMIDs sent through stdin.
- Archive-pubmed -scrub now removes all -extras and -nihocc postings.

Version 18.1: October 21, 2022

- Archive-pubmed -extras GENE indexing adds PREF (preferred gene name), GSYN (gene synonym), and GRIF (gene reference-into-function) fields.
- Ref2pmid -options test uses citmatch service without requiring local archive.

Version 18.0: October 12, 2022

- Efetch -chunk overrides number of records to retrieve in each network request.
- Gbf2ref script can read EMBL/UniProt format in addition to GenBank/GenPept.
- Gbf2ref populates ATHR field with string of all authors in citation.
- Ref2pmid -options "verify" first determines if existing PMID in citation is correct.
- Ref2pmid -options "remote" calls citmatch service if local algorithm failed to find an unambiguous match.

- Use `ref2pmid -options "strict,remote,verify"` for best citation matching accuracy and performance.
- `Edict server preload -file` reads precomputed `ref2pmid` results, for use by `nquire -edict match -citation`.
- `Xtract -jour` calls `CleanJournal` to handle encoded ampersands and apostrophes.
- `Download-ncbi-data -journals` uses `xtract -jour` for better journal title cleanup.

Version 17.9: September 20, 2022

- Consolidated local archive cache maintenance functions.
- Moved first-level inverted index cache files out of `Archive` directory.
- Moved PubMed sentinel files to a new `Sentinels` directory.
- Above changes will make it feasible to have separate "live" and "next" copies of local archive files, eliminating nightly down-time if a 2 TB solid-state drive is available.

Version 17.8: September 6, 2022

- Refactored XML parser improves speed of "parent / star" processing.
- Restored proper archiving order for adjacent versions of the same publication.
- Adjusted code for downloading `desc2022.xml` MeSH descriptor file.

Version 17.7: August 29, 2022

- Citation matcher maps journal to standard name, checks for unresolvable ambiguity.
- `Xtract -wrp` delays reencoding if `-replace`, allowing `-rep "&"` instead of `-rep "&"`.

Version 17.6: August 15, 2022

- Recompiled with Go 1.19 for faster execution on all processors.
- `Download-ncbi-data "journals"` choice generates journal title lookup files with and without a leading article, allowing both "Journal of Immunology" and "The Journal of Immunology" to match the expected "J Immunol" record regardless of original cataloging.
- Similar ampersand expansion (of encoded "&" to " and ") is done for journal title lookup keys, but not for the local archive `JOUR` index, so that it matches the current term list convention in PubMed.
- Precomputed journal tables attempt to resolve ambiguous name or alias collisions by favoring journals that are currently indexed in MEDLINE. This removes two competing entries with "Journal of Immunology" aliases.
- `Journal sets` file includes multiple choices for 2671 abbreviations that cannot be resolved automatically, separated by vertical bars, so "dmj" maps to "Danish medical journal | Diabetes & metabolism journal".
- `Edict.go` server adds support for remote journal title queries.
- Local archive citation matcher caches most recent unique query results to avoid repeatedly recalculating the same reference on all components of a `pop/phy/mut/eco` set.
- `Xtract -reg` and `-exp`, regular expression patterns for `-replace`, now protected by `mutex`.
- `Xtract -aliases` reads mapping file to support `-classify` argument, which uses multiple whole word or phrase substring matching to populate custom Entrez indices.

- Deprecated index-pubmed script now just calls archive-pubmed -index.
- Custom-index runs collect and expand steps, previously performed by index-pubmed.
- Added idx-affil custom indexing helper script for affiliation experiments.
- Added idx-journals to index Journal/Title in JRNL field, skipping the ISOAbbreviation, ISSN, ISSNLinking, MedlineTA, and NlmUniqueID elements also indexed in JOUR field.
- Transmute -gbf uses file of accessions for filtering a GenBank flatfile stream.
- Speed of the (rate-limiting) -gbf flatfile partitioning step (using an uncompressed GenBank release file on a dedicated machine) was just under 40,000 records per second. This is sufficient to support the possibility of applying the local PubMed archive approach to the orders-of-magnitude larger set of sequence records.
- Fixed bug in RepairUnicodeMarkup that produced unbalanced symbols when runs of Unicode superscripts or subscripts were immediately followed by another type of non-ASCII character, such as a Greek letter.

Version 17.5: August 2, 2022

- Added edict.go remote server for local archive (RESTful equivalent of phrase-search commands), with support for search, fetch, (compressed) stream, and (citation) match operations. The -host and -port arguments override the default "localhost:8080" address used for testing.
- Nquire -edict is a shortcut for access to a running edict server, defaulting to "localhost:8080".
- Set the NQUIRE_EDICT_SERVER environment variable to override the nquire -edict address.
- Nquire -preview is a shortcut for the upcoming PubMed SOLR server.
- Nquire -get and -url send explicit curl -X GET and -X POST arguments.
- Archive-pubmed -extras flag indexes chemical, disease, and gene references (extracted from article contents by NCBI text mining and NLM indexing groups) as CHEM, DISZ, and GENE fields. This supersedes the index-extras script, and only runs if any relevant data file is out of date or not yet downloaded.
- Archive-pubmed writes two independently-compressed blocks (xml+DOCTYPE header and PubmedArticle XML record data) to each file. When streaming sets of compressed files for efficient network transfer, the headers before each record are skipped by advancing a fixed number of bytes.
- New asn2ref script helps with citation matching from Seq-entry ASN.1 records.
- Elink internal chunk size lowered to 400 to avoid server timeouts.
- Test-eutils -preview runs the basic -alive tests on the SOLR server.

Version 17.4: July 18, 2022

- Efilter handles <Query> or <Id> items in the ENTREZ_DIRECT message.
- Archive-pubmed second-level inverted index cache moved to Increment folder.
- Downloading each PubMed ftp release file validates contents, retries on failure.
- Normalization removes combining accents in author affiliation field.
- Remaining functionality needed for full EDirect support of PubMed SOLR server includes combining queries (in esearch) and specifying a range of output records (in efetch).

Version 17.3: June 29, 2022

- Esearch -title queries individual words with [TITL] to bypass automatic term mapping.
- Elink supports <Query> or <Id> items in the ENTREZ_DIRECT message.
- Local index restores PAIR field to accelerate inexact citation matching.
- Removed TLEN and TNUM fields from local index.
- Local archive adds xml and DOCTYPE lines to every PubmedArticle record. This will allow biopython's Bio.Entrez subpackage to use the archive files directly.
- Fetch-pubmed removes the xml and DOCTYPE prefix from individual PubmedArticle records. A single pair precedes the <PubmedArticleSet> wrapper.
- Fetch-pubmed -turbo places <NEXT_RECORD_SIZE> objects in front of each XML record, to allow faster XML data extraction with xtract -turbo.
- Xtract -mirror reverses the characters in a string.

Version 17.2: June 13, 2022

- Esearch passes a <Query> field in the ENTREZ_DIRECT message when using the upcoming PubMed SOLR backend. This allows efetch to circumvent the 10,000 PMID-per-query limit by repetitive searching with a sliding window of dynamically-resized create date ranges.
- Efetch temporarily calls "transmute -mixed -normalize pubmed" twice, as a quick fix to compensate for the SOLR server's unexpected encoding of non-ASCII characters with the "&#x...;" construct.
- Internal esearch -count and -uids added to access PubMed SOLR without using history.
- Esearch -translate and -components now handle SOLR output variant.
- Local indexing adds bad date, future date, medline date, has abstract, and versioned to the PROP field.
- Local JOUR field now indexes MedlineTA, NlmUniqueID, and ISSNLinking values.
- Local YEAR field standardizes on 4 directory levels for all dates (e.g., /1/8/9/2/), which includes 104,824 citations from the eighteenth and nineteenth centuries.
- Run "archive-pubmed -clear -index" to refresh the inverted index cache after changes to indexing code.
- Phrase-search -totals prints the term list with document counts for the given field.
- Improved local citation matcher parsing of "in press" journal references.

Version 17.1: May 16, 2022

- Esearch -translate and -components, instead of -query, return the full query translation or the individual translation components, respectively. For -db pubmed they show the automatic term mapping expansions.
- Archive-pubmed adds a second level of caching for faster search index rebuilding. Use -clean to remove Inverted folder intermediate files, or -clear to also remove Archive inverted index cache files.
- Improved author name normalization and indexing of apostrophes and combining accents.

- New cit2pmid script calls "https://pubmed.ncbi.nlm.nih.gov/api/citmatch" service, or with "-local" flag performs citation matching using EDirect local archive and search system.
- Added ncbi::edirect:Execute function to control EDirect from NCBI C++ toolkit programs.

Version 17.0: April 18, 2022

- Removed STEM field (Porter2 algorithm) from standard local indexing.
- Run `rchive -e2delete "${EDIRECT_PUBMED_MASTER}/Archive"` to clear the incremental cache. This is needed to remove residual STEM postings.
- Next execution of `archive-pubmed -daily` will reinitialize the pre-indexed cache in under 90 minutes. Future daily updates should take around 3 minutes.
- `Archive-pubmed -index` can subsequently rebuild local search indices on demand in under 2 hours.
- `Xtract` adds `-stemmed` argument to index Porter2-processed sentences.
- Use `custom-index $( which idx-stemmed ) STEM` to manually restore stemmed index.

Version 16.9: April 7, 2022

- Recompiled with Go 1.18, which should execute faster on ARM and Apple Silicon processors.
- `Nquire -pubchem`, previously a legacy alias for `-pugrest`, is now repurposed for more convenient access to PubChem Pathways, which replaces the retired Entrez BioSystems database.
- `Xtract` automatic detection of JSON input accidentally conflated the default record names for top-level objects and arrays. Now it uses the same policy as `transmute -j2x` ("opt" and "anon", respectively).
- Local search index adds PROP field for publication types.
- Using parallel `pgzip` (de)compression library for faster search index construction.
- `Archive-pubmed -daily` keeps an incremental cache of pre-indexed files up to date.
- `Archive-pubmed -index` uses the cache to repopulate all local retrieval system files in under 3 hours, superseding the slower `index-pubmed` process.

Version 16.8: March 17, 2022

- Phrase-search supports MESH queries by mapping to indexed TREE or CODE fields.
- Phrase-search reads an expandable file of JOUR field aliases (e.g., PNAS).
- `Xtract -pairx` extends `-pairs` to include isolated single words.
- `Idx-pairs` helper for `custom-index` reintroduces modified non-positional title word PAIR index.
- `EDIRECT_PREVIEW` environment variable allows testing of the upcoming PubMed API.
- `Elink` stores identifiers in `ENTREZ_DIRECT` message instead of history with new PubMed server.
- `Efilter` accepts `ENTREZ_DIRECT` instantiated identifiers when using the new PubMed API.

Version 16.7: March 1, 2022

- Added `ds2pme` script to convert PubMed DocumentSummary XML to Pubmed-entry ASN.1.

- Phrase-search adds -words (replacing -partial) and -pairs, for local index citation matching test.
- Filter-stop-words script adds optional -plus argument to support phrase-search -pairs.
- Index-pubmed script adds TLEN and TNUM indices, phrase-search can query those fields by range.
- Xtract -trim removes leading and trailing spaces, and leading zeros.
- Xtract -wct counts the number of words in a string, obeying -stops and -stems modifiers.
- Transmute -g2r (gbf2ref) tracks previously-encountered titles and authors, suppressing duplicate citations, which allows xtract -select STAT to create non-redundant inpress or unpub subsets.
- Transmute -r2p (and ref2pmid shortcut) reads gbf2ref subsets for local citation matching.

Version 16.6: February 14, 2022

- Nquire -pugwait polls asynchronous PubChem PUG-REST searches, returning ENTREZ_DIRECT structure with instantiated compound identifiers. (Support for this form was included in the 2020 EDirect redesign.)
- Nquire -timer prints milliseconds between initial service request and completion of network data transfer.
- Xtract warns that capitalized exploration arguments (e.g., -Block), which were needed for recursive data in the original (retired) Perl implementation, are now deprecated, and will be removed in the near future.
- Added pma2pme script to convert PubmedArticle XML to Pubmed-entry ASN.1.
- Archive-pubmed script takes optional -asn argument to save Pubmed-entry ASN.1 files in the local archive, coexisting with the PubmedArticle XML records. Use fetch-pubmed -asn to retrieve.
- Index-pubmed script now creates indices for author (AUTH, ANUM, FAUT, LAUT, CSRT, INVR) and citation (JOUR, VOL, ISS, PAGE, LANG) fields, eliminating the need for a separate custom-index step. To be used with TITL, TIAB, and YEAR fields for local citation matching experiments with phrase-search script.
- Xtract modifies -plain (removal of mixed-content sections) and adds -simple (normalization of accented letters), both derived from -basic (cleanup of superscripts and subscripts).
- Xtract repurposes -author and adds -prose, to correct commonly misused lookalike characters (sharp S and lower-case beta) and remove accents and markup, for use in generating search indices and ASN.1.
- Xtract -auth replaces commas, periods, and hyphens to allow queries with GenBank reference authors.
- Xtract -month "PubDate/*" finds first month name or abbreviation, returning a number from 1 to 12.
- Xtract -page extracts first page (digits and letters) from a page range.
- Xtract adds -hex (hexadecimal), -oct (octal), and -acc (running total accumulator) numeric arguments.
- Xtract -element "." (period) generates text ASN.1 from customized XML records. Underscores in XML tag names show unlabeled braces around SEQUENCE OF components (single), unquoted INTEGER or ENUMERATED values (trailing), or alternative CHOICE selections (internal).
- Xtract -element "%" (percent) generates JSON from customized XML records. Underscores in XML tag names show unlabeled brackets (single), named arrays in brackets (leading), or unquoted values (trailing).
- Transmute -g2r (and gbf2ref shortcut) extract reference fields from sequence flatfiles for citation matching.
- Phrase-search -partial queries title words individually and combines results for ranked matches.

- EDirect now uses a custom internal Unicode-to-ASCII conversion map.

Version 16.5: January 3, 2022

- Using efetch.fcgi instead of esearch.fcgi to retrieve UID lists, in preparation for new PubMed API.
- Nquire adds initial implementation of -puglist and -pugwait helper functions for PubChem.
- Xtract -verify raises maxDepth limit to 30 to avoid warning about depth of PMC XML records.
- Rchive -invert uses a separate map for each initial character, now runs in 1/3 less time.
- Added indexing of INVR investigators to idx-authors helper script.

2021

Version 16.4: December 20, 2021

- Nquire -pugrest -inchi silently adds "InChI=" prefix if it is missing in the argument value.
- Xtract -year "PubDate/*" construct, broken in recent refactoring, again returns a single 4-digit year.
- Separate EDIRECT_PUBMED_MASTER and EDIRECT_PUBMED_WORKING environment variables also apply to index-extras and custom-index scripts.

Version 16.3: December 15, 2021

- Esummary -format docsum has a clearer "redundant argument" message.
- Efilter -db assembly adds -status shortcut.
- Nquire adds -pugrest and -pugview shortcuts for PubChem Power User Gateway services.
- Nquire allows Windows version of curl to recognize Cygwin paths.
- Xtract supports multiple -insd feature clauses for output on a single line.
- Xtract -insd source taxid qualifier extracts integer from "taxon:###" db_xref.
- Xtract -accent no longer needs -strict or -mixed processing flags.
- Transmute -plain removes accents and diacritical marks from text.
- Local archive scripts use 2022 PubMed release files and 2022 MeSH data files.
- Local archive Extras directory added to store original MeSH data files and NLP downloads. Separate environment variables (EDIRECT_PUBMED_MASTER and EDIRECT_PUBMED_WORKING) can place Archive, Data, and Postings folders on the computer's internal drive, while keeping release files and indexing intermediates on the external SSD.
- Renamed alternative edirect-install.sh script to update-edirect.sh.

Version 16.2: October 28, 2021

- Xtract automatically detects and processes JSON, text ASN.1, and GenBank/GenPept formats. An explicit transmute command is only needed if you wish to inspect the intermediate XML, or to override the default conversion arguments.

- Transmute -j2x -nest "element" choice adds "_E" suffixes to multi-dimensional array components. This is now the default for both transmute and the xtract JSON converter, assigning a distinct tag name to each level.
- Transmute -g2x parses the DBLINK section to capture BioProject and BioSample identifiers.
- Using printf instead of echo to generate configuration commands in the install-edirect.sh script.
- Efetch -db pubmed -format uid retrieves PMIDs in chunks of 10,000 in preparation for new PubMed API.
- Added idx-metadata script for indexing JOUR, LANG, MAJR, MESH, and SUBH fields in the local archive.
- Added edirect.py module file for import by Python programs. Use edirect.execute to run individual EDirect commands, Unix tools, or shell scripts, controlling multiple steps by passing the previous result to the next command. Or use edirect.pipeline to launch a chain of several commands contained in a single argument. An edirect.efetch shortcut that takes named arguments is also provided.

Version 16.1: October 13, 2021

- Efetch -strand argument can accept "+" or "-". The leading backslash is required.
- Efetch -express flag works like -immediate, but on several sequence records at a time.
- Nucleotide accession lookup is only needed for master sequences.
- Epost looks up all nucleotide accessions to silently skip replaced records.
- Xtract -self applies a default value to allow detection of empty self-closing tags.
- Xtract uses both time interval and record count to trigger an output buffer flush.
- Minor refactoring of xtract argument parsing code.
- Removed obsolete setup.sh script.
- Updated cacert.pem file.

Version 16.0: October 4, 2021

- Xtract -insd feat_intervals is a version of feat_location that generates 0-based positions.
- Transmute -extract accepts -0-based and -1-based modifiers, defaulting to the 1-based feat_location form.
- The eutils.SequenceExtract library function has a new isOneBased boolean argument.
- Added fuse-segments to the set of scripts for post-processing magicblast -outfmt asn.
- Renamed uniq-columns script to uniq-table.

Version 15.9: September 27, 2021

- Redesigned install-edirect.sh script, which no longer calls setup.sh.
- Alternative edirect-install.sh script does not offer to edit configuration files.
- Installation prints PATH command to use for current terminal session.
- Successfully installed and ran test-eutils and test-edirect on Cloud.
- Minor refactoring of xtract -element variant code.

- Modified snp2hgvs script to remove records where the Id and SNP_ID do not match.

Version 15.8: September 16, 2021

- Efetch -db clinvar leaves VCV prefix for -format vcv.
- Added gene column to spdi2tbl component of snp2tbl.
- Renamed spdi2prod script to tbl2prod, now reads output of snp2tbl.
- Xtract -hgvs allows R and Y nucleotide ambiguity characters.
- Xtract -author replaces commas and periods in GenBank reference authors.
- Refactored eutils.FASTAConverter into tokenizer and streamer components.
- Added uniq-columns script.

Version 15.7: September 9, 2021

- Efetch -immediate flag avoids memory overflow on sets of extremely large sequences.
- Efetch -format fasta chunk size reduced to 50 records at a time.
- Xtract -backward presents elements in reverse order.
- Modified spdi2prod script to print "wild-type" protein and CDS translation.
- Renamed filter-table script to filter-columns.
- Moved all help text to a new help subfolder.
- Recompiled with Go 1.17, which generates smaller binaries that execute faster.

Version 15.6: September 1, 2021

- Added datasets and sra-toolkit choices to download-ncbi-software script.
- Added efilter -keyword and -purpose shortcuts for "purposeofsampling" keywords.
- Restored missing efilter -pub and -released shortcuts.
- Xtract -ncbi2na and -ncbi4na functionality available in common eutils library.
- Added vendor option to Go build.sh scripts to cache source code for all external library dependencies.

Version 15.5: August 16, 2021

- Documentation introduces Go compiled language, dependency management with modules, and EDirect's local eutils library package.
- Now including go.mod and go.sum module files for eutils and cmd project directories.
- Transmute -counts implements Go base count example.
- Added xml2fsa script for converting INSDSeq XML to FASTA.
- Xtract -fasta, used by xml2fsa script, now generates conventional 70 uppercase characters per line.
- Xtract -element "~" for object contents joins "?" for object name.

- Added download-ncbi-software script, initially for magic-blast on linux, macosx, and win64 platforms.
- Added blst2tkns, split-at-intron, fuse-ranges, and find-in-gene scripts for merging overlapping reference coordinate matches from magicblast -outfmt asn.
- Nucleotide accession lookups may use [PACC] or [ACCN] fields.
- Nquire supports NQUIRE_IPV4 environment variable for diagnosing suspected IPv6 problem.

Version 15.4: July 16, 2021

- Added efilter -source select as shortcut for RefSeq Select dataset.
- Efetch -id removes PMC prefix in front of PubMed Central identifiers.
- Transmute -align -a argument adds m choice for using commas to separate integers into groups of 3 digits.
- Added transmute -align -w argument to specify minimum width for all columns.
- Improved scripting introduction in automation section of documentation.

Version 15.3: June 21, 2021

- Transmute -separate replaced by -combine, default is not to remove internal top-level objects, which result from retrieving large sets of data in smaller chunks.
- Added transmute -search for finding positions of patterns in sequence data (e.g., restriction sites), -circular flag allows match to pattern spanning origin of circular molecule.
- Added disambiguate-nucleotides script to expand degenerate base letters for restriction enzymes like AccI.
- Added transmute -find for searching non-sequence text that includes digits, spaces, and punctuation, -relaxed flag ignores spacing differences, punctuation.
- Added idx-words helper script to split words at punctuation for indexing [WORD] field.
- Local indexing speed improvements.
- Removed deprecated rchive -phrase function, not needed with positional indices.

Version 15.2: June 3, 2021

- Expand-current prepares decompressed nonredundant PubMed archives with xtract -index, which places <NEXT_RECORD_SIZE> objects in front of each XML record.
- Xtract -turbo uses precomputed <NEXT_RECORD_SIZE> information to double the speed of the (rate-limiting) partitioning step, allowing additional CPU cores to participate in XML data extraction.
- Added custom-index, which calls a user-supplied helper script to build an initial PMID-term index, and then completes the inversion and posting steps to integrate the new field into the local search system.
- Added idx-author sample helper script, with boilerplate xtract instructions for generating an IdxDocument set, and specific commands for populating a novel [ANUM] index.
- Rchive -invert converts input strings to lower-case if that was not already done in the indexing step.
- Added xtract -includes, like -contains but requiring the substring match to align on word boundaries.
- Index-pubmed adds a new [TITL] field, which indexes only the article title.

- Phrase-search -title performs an exact search on the local [TITL] field.
- Index-pubmed replaces the [NORM] field with [TIAB], the conventional code for title and abstract. Legacy queries using [NORM] will be internally redirected to [TIAB].
- README and readme.pdf files are more concise, with several sections now residing only in the more detailed web documentation.

Version 15.1: May 20, 2021

- Xtract and transmit now run as native executables on Apple Silicon machines.
- Setup.sh script can add edirect folder to PATH in both .bash_profile and .zshrc for MacOS.
- Documented efetch and elink reading large lists of identifiers from stdin or file, bypassing need for epost.
- Nquire supports -ncbi, -eutils, and -pubchem URL shortcuts.
- Nquire uses --cacert instead of --capath in curl command.
- Efetch -db snp uses -format -self flag to keep self-closing attribute-free PAIRED or SINGLE items in XML.
- SNP processing adds Gene, Hgvs, and Spdi fields, and (when different from Id) adds OldId field.
- Added transmute -codons to display nucleotide codons above protein residues.
- Transmute -a2x, -c2x, -g2x, -j2x, and -t2x have shortcut scripts asn2xml, csv2xml, gbf2xml, json2xml, and tbl2xml, respectively.

Version 15.0: April 29, 2021

- Removed edirect.pl and setup-deps.pl scripts containing original Perl implementation.
- Deprecated -oldmode and -newmode arguments, and USE_NEW_EDIRECT environment variable.
- Simplified -hgvs output structure, and converted to 0-based positions for SPDI processing.
- Added snp2hgvs script as shortcut for -hgvs extraction from SNP docsum.
- Added hgvs2spdi script to adjust CDS-relative -hgvs offsets into sequence-relative positions suitable for use by transmute -replace.
- Added spdi2prod script to pair modified NM translation with modified NP sequence.
- Added snp2tbl script to produce tab-delimited table of adjusted SNP values in one step.
- Transmute -replace and -extract use -lower to specify lower-case output.
- Transmute -g2x writes INSDAltSeqItem_first-accn and INSDAltSeqItem_last-accn objects for WGS master.
- Xtract -pkg and -enc XML generators accept multiple slash-delimited object names.

Version 14.9: April 15, 2021

- EDirect runtime errors display an "ERROR:" tag in inverse bold red text on the terminal.
- Efetch -id now works on WGS master accessions in nucleotide databases.
- Lookup of rs/ss numbers supported in -id argument for -db snp.

- Epost uses chunks of 1000 to avoid server truncation.
- Transmute -hgvs parsing now supports ".g" genomic and ".c" coding sequences, in addition to ".p" proteins. The ".c" item location uses <Offset> rather than <Position>, since it is relative to the first base of the CDS initiation codon, not the sequence. Additional operations will be required to adjust this value for SPDI processing.
- The next release will retire the edirect.pl and setup-deps.pl scripts, and will ignore the old/new mode arguments and environment variable.

Version 14.8: March 24, 2021

- Esearch has improved logic to recognize [FILT], [PROP], and [ORGN] controlled-vocabulary phrases without the need for extra quotation marks to protect the query.
- Elink runs -cmd acheck to confirm empty history result, can detect stale links to deleted records.
- Elink writes structured message on empty history input to avoid breaking pipeline of multiple link operations.
- Nquire adds -dir command to show ftp directory listing with column of file sizes.
- Blank lines are ignored by sort-uniq-count, sort-uniq-count-rank, and sort-table scripts.
- Removed eblast script after URL-based BLAST service was disabled.
- Use of edirect.pl script (through -oldmode command-line argument or USE_NEW_EDIRECT=false environment variable setting) prints DEPRECATED warning message.

Version 14.7: March 8, 2021

- Xtract -insd handles qualifiers without values (e.g., pseudo, transgenic).
- Xtract -insd adds feat_location qualifier to print feature intervals.
- Transmute -extract reads xtract -insd feat_location interval format.
- Transmute -remove -first and -last arguments can use sequence letters instead of count.
- Efetch -format ipg processes potentially large records one at a time.
- Error message printed if non-integer -id argument is used with -db taxonomy.
- Nquire -dwn and -asp file download failure report improved.
- Refactored common.go and transmute.go functions into reusable local "eutils" package.
- Go compiler uses "replace eutils => ../eutils" line in go.mod file to import from local eutils package.
- Transmute -degenerate > gdata.go recreates library file with updated genetic code mapping tables.
- Removed obsolete and deprecated scripts.
- Added sort-table, filter-table, and print-columns scripts.
- Revised eblast script as prelude for running on cloud.

Version 14.6: February 3, 2021

- Transmute -cds2prot added to translate coding regions with nucleotide substitutions.
- Transmute -revcomp and -molwt added, sharing code with xtract functions.

- Transmute -remove, -retain, and -replace allow script-driven editing of sequences and insertion of SNP bases, with argument values obtained by -hgvs parsing of HGVS data.
- Transmute -diff simplifies visualization of sequence point mutations.
- Esearch and efilter enforce restriction of -country shortcut to sequence databases.

Version 14.5: January 19, 2021

- Added -hgvs support for genomic single nucleotide substitutions.
- Sort by RefSeq accession numbers within categories of -hgvs output.
- Updated efilter -db snp -class frameshift mapping to FXN entry.
- Updated test-eutils upon retirement of Entrez sparcle database.
- Updated Amino Acid Substitutions example.
- Added Reference Formatting example.
- Xtract -replace uses -reg and -exp values for regular expression substitution.

Version 14.4: January 13, 2021

- HGVS format parsed into XML by xtract and transmute -hgvs commands. Initial implementation supports amino acid missense and nonsense variations. More coding to follow.
- Transmute -align -a argument adds z choice for padding numbers with leading zeros.
- Fixed bug in efetch -start and -stop subrange arguments.
- Restored trailing newline lost in switch from echo to printf.

Version 14.3: January 7, 2021

- Major overhaul of EDirect documentation completed.
- Release notes and additional examples moved to separate web pages.
- Xtract -doi cleans DOI data and generates complete URL.
- Transmute -align -a argument adds n and N choices for aligning to decimal point.
- Added align-columns script as preferred front-end to transmute -align.
- Transmute -j2p works on a concatenated stream of JSON records.
- Transmute -aa1to3 and -aa3to1 amino acid abbreviation converters added for HGVS processing.
- Improved lookup of accessions in -id argument, now includes 21 Entrez databases with 10 fields.
- Improved code that maps PDB protein accessions with case-sensitive chain letters.
- Efetch -db bioproject no longer converts formats in order to handle accession input.
- Efetch -format fasta adds newline if missing before angle bracket.
- Uses printf "%s" instead of echo to avoid misinterpreting backslash in retrieved records.
- Nquire keeps error messages from leaking into output file.

- Confirmed that EDirect will run on Apple Silicon under Rosetta translation environment.

2020

Version 14.2: December 14, 2020

- Transmute -normalize now handles unexpected DocumentSummary attributes.
- Transmute -t2x -heading takes tags from columns in first row of file.
- Transmute -align converts a tab-delimited table to columns padded with spaces.
- Nquire -raw does minimal URL encoding for GeneOntology query.
- Efetch -db bioproject maps -format docsum to -format xml.
- Download-ncbi-data script updated to use desc2021.xml and supp20201.xml for mesh lookup table.

Version 14.1: November 30, 2020

- Nquire always uses new implementation.
- Scripts updated to use new nquire features.
- Efetch and esummary warn about -db mismatches and collisions between -format choices.
- Efetch supports PDB accessions with case-sensitive chain letters in the -id argument for -db protein.
- Efetch supports GCA and GCF accessions in the -id argument for -db assembly.
- Esearch -mindate and -maxdate can be used alone, with defaults filling in the missing argument.
- Xtract -set, -rec, -pkg, and -enc shortcuts join -wrp for creating XML objects at appropriate levels during XML generation.
- Xtract -head can read separate arguments for individual column headings.
- Xtract uses double-hyphen to append value into variable.

Version 14.0: November 12, 2020

- Redesigned EDirect active by default, deselected by running "export USE_NEW_EDIRECT=false" to set environment variable.
- Old implementation also chosen by adding -oldmode as the first argument to individual efetch, efilter, einfo, elink, epost, esearch, esummary, and nquire commands.
- Automatic retry supported on empty result for all formats, not just structured data.
- Esearch protects [PROP], [FILT], and [ORGN] queries in biological databases from parsing artifact on controlled vocabulary entries with embedded "or" and "not".
- Efetch adds -showgaps flag.
- Xtract -wrp causes angle brackets, ampersands, quotation marks, and apostrophes to be reencoded in the new XML, without the need for explicitly using -encode.
- Xtract formatting, modification, normalization, and conversion functions moved to transmute, with old calls automatically redirected to avoid breaking user scripts.

- Transmute -format takes optional -comment and -cdata flags to keep XML comments and CDATA blocks.
- Transmute -a2x converts text ASN.1 data to XML.
- Xtract -ncbi2na and -ncbi4na decompress nucleotide sequence converted from text ASN.1 hex representation.

Version 13.9: September 14, 2020

- Xtract -fasta splits long sequences into groups of 50 letters.
- Xtract -select restores -in to indicate name of identifier file.
- Redesigned EDirect selected by running "export USE_NEW_EDIRECT=true" to set environment variable.
- New implementation also chosen by adding -newmode as the first argument to individual efetch, efilter, einfo, elink, epost, esearch, esummary, and nquire commands.

Version 13.8: August 27, 2020

- Deprecated econtact and eproxy utilities.
- Deprecated -alias argument.
- Added accn-at-a-time and skip-if-file-exists scripts.
- Xtract -g2x converts GenBank and GenPept to INSDSeq XML.
- Xtract -c2x is a CSV (comma-separated values) variant of -t2x.

Version 13.7: May 28, 2020

- Removed xtract -repair shortcut for -unicode conversion.
- Xtract -decode supports direct Base64 decoding.
- Xtract -normalize added for Efetch post-processing.

Version 13.6: April 17, 2020

- Efilter -db snp -class shortcuts mappings updated to reflect changes to FXN index.
- Xtract adds -matches and -resembles string conditional tests, -order string processing command.
- Minor change to term granularity for MeSH [TREE] index creation.

Version 13.5: February 21, 2020

- Xtract -select -streaming uses case-insensitive test, concurrent processing.
- Minor change to term granularity for reenabled [CODE] index creation.

Version 13.4: February 4, 2020

- Local [CONV] term list indexes GNBR theme plus relationship plus identifier pair.
- Theme indexing splits genes at semicolons, adds M prefix to OMIM, H to ChEBI identifiers.
- Run "source theme-aliases" to load commands for navigating theme identifier connections.

Version 13.3: January 28, 2020

- Index-extras downloads newest version of Global Network of Biomedical Relationships theme data.
- Xtract -contour replaces -synopsis -leaf variant.

Version 13.2: January 7, 2020

- Expand-current can be run after archive-pubmed as well as index-pubmed.

2019

Version 13.1: December 30, 2019

- Local index only builds TREE fields for A, C, D, E, F, G, and Z categories.
- Stop word list now includes "studies" and "study" in addition to "studied".
- Xtract -select driver file command names changed to -retaining, -excluding, and -appending.
- New efetch -format types documented for -db clinvar.
- Updated MeSH tree category descriptions in phrase-search -help.

Version 13.0: December 16, 2019

- Xtract -histogram collects data for sort-uniq-count on entire set of records.
- Xtract -wrp with empty string or dash resets -sep, -pfx, -sfx, -plg, and -elg customization values.
- Xtract -fwd and -awd print once before and once after a set of object instances.
- Xtract -t2x uses asterisk before column name to indicate XML contents, will not escape angle brackets in string.

Version 12.9: December 9, 2019

- Xtract -plain, an -element variant that removes embedded mixed-content markup, now also converts Unicode subscripts/superscripts, cleans bad spaces.
- Xtract -select -in changed to -select -using.
- Xtract -select -adding matches by identifier and appends XML metadata.
- Xtract -select -merging requires original records and identifier-metadata file to be in same order.
- Moved xtract -examples text to separate hlp-xtract.txt file.
- Efetch -format asn maps to -format asn.1.
- Disabled building of [CODE] field in local index.
- Minor changes to term granularity for faster local index creation.
- Index-pubmed takes optional [-collect | -index | -invert | -merge | -promote] argument to resume processing in an intermediate step.

Version 12.8: December 3, 2019

- Xtract -molwt sequence processing command added.
- Xtract -len triggers -def instead of returning 0 for missing object.
- Xtract -pairs does not drop the first component of hyphenated terms (e.g., site-specific).
- Xtract -synopsis with optional -leaf argument only reports content nodes.
- Adjusted test-eutils -esummary dbvar test.
- Local archive asks user to check TRIM status on slow update.
- Local archive checks that volume is actually a solid-state drive.

Version 12.7: November 21, 2019

- Local archive detects slow performance, reminds user about antivirus scanning and content indexing.
- Local archive warns if Mac file system is not APFS, prints instructions on how to reformat drive.
- Xtract -examples adds namespace prefix and Base64 decoding example.
- Efetch -db nucleotide/nucore -style withparts/conwithfeat processes -id accession list one record at a time.

Version 12.6: November 15, 2019

- Added support for automatic fallback to IPv4.
- Xtract -insd adds mol_wt as a synonym for calculated_mol_wt.
- Changed granularity of local index for [YEAR] field.
- If archiving is slow, ask user to ensure that antivirus scanning and content indexing are disabled.

Version 12.5: November 6, 2019

- Efetch -start and -stop subset retrieval arguments are now documented.
- Efetch -format docsum rescues uid attribute if no existing <Id> tag, now using case-sensitive test to prevent <id> from blocking.
- Added efetch -revcomp flag, sets -strand 2.
- Einfo sorts -fields and -links results by tag name.
- Fixed stdin-vs-argument bug in ftp-cp introduced when moving code into edirect.pl for Anaconda issue.
- Local search system [THME] field also indexes disambiguated themes - Jc (chemical-disease) and Jg (gene-disease) - under original code J (role in disease pathogenesis).

Version 12.4: October 28, 2019

- Expanded README file to cover more advanced features of potential interest to codeathon participants.
- Elink -released also accepts four-digit year.
- Local search system builds separate [CODE] and [TREE] indices for MeSH code and hierarchy values.

Version 12.3: October 23, 2019

- Added single-line copy-and-execute commands, in curl and wget flavors, as convenient options for EDirect installation.
- Elink -db pubmed supports -cited and -cites to follow reference connections from the NIH Open Citation Collection dataset.
- Added missing retry code to esearch and esummary.
- Xtract -t2x converts tab-delimited table to XML.
- Xtract -sort rearranges records by designated identifier.
- Xtract -split breaks up a large XML stream into multiple files.
- Xtract -chain changes _spaces_to_underscores.
- Efetch -db gtr -format docsum -mode json downloads in smaller groups to avoid timeouts.
- Nquire -get does not need -url command if followed immediately by URL argument.
- Consolidated code for perl-based commands (e.g., nquire, transmute) into master edirect.pl script.
- Wrappers to all perl-based commands now handle conflict with Anaconda installation.
- Moved external resource indexing code from xtract to rchive.
- Added phrase-search -filter command to pipe efetch -format uid results into local query.
- Download-ncbi-data script updated to use desc2020.xml and supp2020.xml for mesh lookup table.
- Experimental index-extras script loads natural language processing results into local retrieval system.
- Experimental fetch-extras scripts retrieves indexed NLP fields per PMID saved in local archive.

Version 12.2: September 27, 2019

- Added install-edirect.sh script, with download link in web documentation, for easier installation.
- Updated setup.sh script to ask permission to edit the PATH setting in the user's .bash_profile file.
- Xtract -is-before and -is-after tests compare order of strings.
- Xtract -mul, -div, and -mod numeric processing commands added.

Version 12.1: August 29, 2019

- Local query index now creates field-specific subdirectories immediately below Postings folder.
- Term position index files, needed for phrase and proximity searching in [NORM] and [STEM] fields, not made for [CODE] and [YEAR].
- Phrase-search -terms [field] prints complete term list for given field.
- Rchive -help gives example of how to sequentially ascend the MeSH [CODE] hierarchy index.
- Added expand-current script, to be run after index-pubmed, to prepare for fast scanning of all PubMed records.

- Added -repeat option to test-eutils monitoring script.

Version 12.0: August 14, 2019

- Efetch adds -format bioc for -db pubmed and -db pmc, retrieving annotated records from PubTator Central.
- Added download-ncbi-data script to consolidate and replace special-case scripts.

Version 11.9: August 5, 2019

- Updated test-eutils driver files due to retirement of unigene.
- Test-eutils progress line prints a period for success, x for failure.
- Test-eutils -timer prints response times in milliseconds for each query.
- Added exclude-uid-lists script.
- Added download-pmc-bioc script.
- Nquire supports -bioc-pubmed and -bioc-pmc shortcuts.
- Fetch-pubmed -fresh generates uncompressed PubMed files from local archive for fast data extraction.
- Index-pubmed retains compressed PubMed intermediate files for batch scanning.
- Xtract supports -select parent/element@attribute^version -in file_of_identifiers.

Version 11.8: July 23, 2019

- Xtract -j2x converts newline to space, compresses runs of spaces.
- Xtract -j2x supports JSON null tags.

Version 11.7: July 3, 2019

- Efetch -format docsum -mode json reads 500 records at a time, in conformance with the server limit.
- Fetch-pubmed -all sequentially streams all live records from the local cache.
- Xtract -plain removes embedded mixed-content markup tags.
- Added download-pmc-oa to fetch the open-access subset of PubMed Central.

Version 11.6: June 11, 2019

- Xtract -path implementation was simplified.
- Einfo -db all returns a combined eInfoResult XML, containing field and link names, and record and term counts, for all Entrez databases in one operation.

Version 11.5: June 7, 2019

- Xtract -is-equal-to and -differs-from conditional arguments compare values in two named elements.

Version 11.4: May 23, 2019

- Efetch -db snp -format json reads 10 records at a time for the initial server deployment.

- Updated SNP examples to use `efetch -format json` and `xtract -j2x`.
- Added Genes in Pathways example.
- Added `xml2tbl` script.

Version 11.3: May 3, 2019

- `Xtract -j2x` converts JSON stream to XML suitable for `-path` navigation.
- `Xtract -format -self` retains self-closing tags with no attributes.
- `Esample` replaces `xtract -samples`.

Version 11.2: April 15, 2019

- `Efetch -db snp` only supports `-format docsum` and `-format json`.
- `Efilter -db biosystems` has `-kind` and `-pathway` shortcuts.

Version 11.1: April 3, 2019

- `Xtract` optimizes performance for 6 CPUs with hyperthreading.

Version 11.0: March 11, 2019

- `Xtract -path` generates exploration commands from dotted object path.
- `Xtract -format -separate` retains internal `</parent><parent>`.

Version 10.9: February 1, 2019

- `Xtract -insd` supports a `sub_sequence` qualifier that uses `-nucleic` and produces upper-case sequence.
- `Xtract` now has an `-is-within` string conditional test.

Version 10.8: January 20, 2019

- `EDIRECT_DO_AUTO_ABBREV` environment variable restores relaxed matching of command-line arguments.
- `Efilter` shortcut `-journal` added for `-db pubmed`.
- `Efilter -pub last_*` shortcuts moved to `-released`.
- `Efilter -pub` and `-feature` can take comma-separated list of choices.
- `Transmute -docsum` command added.
- `Transmute -decode` and `-encode` commands renamed to `-unescape` and `-escape`.
- `Transmute -decode64` and `-encode64` commands added.

Version 10.7: January 14, 2019

- `Xtract -nucleic` uses bracketed range direction to determine whether to reverse complement the sequence.

2018

Version 10.6: December 13, 2018

- Local archive script creates command for saving data path in configuration file.
- Xtract -reverse returns -words output in reverse order.
- Efilter shortcut added for -db snp (e.g., -class missense).
- Efetch -format gbc (INSDSeq XML) supports -style withparts and -style conwithfeat.

Version 10.5: December 4, 2018

- EDirect commands and pipelines support faster access with API keys.
- Xtract attributes can be delimited by quotation marks or apostrophes.
- Transmute -encode and -decode commands added.
- Simplified processing of local inverted index intermediate files.

Version 10.4: November 13, 2018

- Rchive local indexing code refactored for faster performance.
- Xtract -deq deletes and replaces queued tab separator after the fact.
- Efilter -organism queries in [ORGN] field if argument is not in shortcut list.

Version 10.3: November 1, 2018

- Rchive -invert, -merge, -promote, and -query steps make better use of multiple processor cores.
- New phrase-search script replaces local-phrase-search.

Version 10.2: October 15, 2018

- Transmute -x2j joins -j2x to simplify the use of JSON-based services.
- Efetch -json converts adjusted XML output to JSON as a convenience.
- Xtract tag alphabet expanded to accommodate converted JSON data.
- Nquire -ftp takes server, directory, and filename arguments, sends data to stdout.

Version 10.1: October 9, 2018

- Xtract -mixed improves support for mixed-content XML.

Version 10.0: September 27, 2018

- Efilter can search for sequence records by sample collection location (e.g., -country "canada new brunswick").
- Xtract parsing code was refactored in preparation for improvements in handling mixed-content XML data.
- Added transmute script for format conversions (e.g., -j2x for JSON to XML).

Version 9.90: September 17, 2018

- Normalized archive path for low-value PMIDs in preparation for incremental indexing.

Version 9.80: September 4, 2018

- Xtract XML block reader can run on separate thread for improved performance on computers with surplus processor cores.
- Fixed bug in string cleanup when text starts with a non-ASCII Unicode character.
- Efetch regular expression pattern for detecting mixed-content tags was adjusted.

Version 9.70: August 22, 2018

- Local archive builds parallel stemmed and non-stemmed indices of terms in the title and abstract.
- Rchive and local-phrase-search use -query for evaluation of non-stemmed terms, -search for evaluation using the stemmed index.

Version 9.60: August 9, 2018

- Local archive script removes newlines inside PubMed text fields.
- Efetch adds missing newline at end of PubmedArticleSet XML.

Version 9.50: July 30, 2018

- Local indexing scripts adjusted to accommodate projected range of PMID values.
- Fixed inconsistency in positional indexing of terms with embedded non-alphanumeric characters.
- EDIRECT_PUBMED_WORKING environment variable keeps local archive intermediate files on a separate volume.
- Rchive and local-phrase-search use -exact to round-trip ArticleTitle contents without interpretation as a query formula.

Version 9.40: July 18, 2018

- Xtract handles misplaced spaces in attributes.
- Xtract -format repairs misplaced spaces in attributes.

Version 9.30: July 9, 2018

- Local data indexing retains intermediate products, allows rapid streaming of non-redundant current records.
- Index preparation removes apostrophe in trailing 's possessives.
- Wildcard minimum varies with prefix-driven posting character depth.

Version 9.20: June 26, 2018

- Portability and efficiency improvements to local data cache scripts.
- Xtract handles misplaced spaces in self-closing tags.

Version 9.10: June 18, 2018

- Added Parent/* element exploration construct to xtract.
- Xtract -year reliably obtains the year from "PubDate/*".

Version 9.00: June 6, 2018

- Fetch-pubmed -path supplies missing Archive directory if root path is given.
- Efetch cleanup of MathML markup properly handles parentheses.

Version 8.90: June 4, 2018

- Xtract -transform and -translate allow data value substitution.
- Xtract -wrp simplifies wrapping of extracted values in XML tags.

Version 8.80: May 29, 2018

- Efetch removes MathML tags from PubmedArticle XML contents, unless the -raw flag is used.

Version 8.70: May 14, 2018

- Local phrase indexing now uses positional indices instead of adjacent overlapping word pairs.
- Xtract -select uses conditional expressions to filter records.

Version 8.60: April 26, 2018

- Efetch -format uid pauses between groups, retries on failure.
- Fetch delay drops from 1/3 to 1/10 second if API key is used.
- Local phrase indexing uses smaller files to avoid memory contention.
- Phrase index removes hyphens from selected prefixes.

Version 8.50: April 13, 2018

- Efetch markup tag removal modified after change in server.
- Xtract -phrase filter split into -require and -exclude commands.

Version 8.40: April 9, 2018

- Efetch removes markup tags in all PubMed XML.
- Xtract without -strict prints warnings if markup tags are encountered.
- Xtract proximity search moved from -matches to -phrase.

Version 8.30: April 4, 2018

- Xtract is now available for ARM processors.

Version 8.20: March 12, 2018

- Minor changes to local record archiving scripts.

Version 8.10: March 2, 2018

- Xtract -strict and -mixed support MathML element tags in PubmedArticle XML.

Version 8.00: February 26, 2018

- Efetch -raw skips database-specific XML modifications.
- Added local-phrase-search script.
- Xtract -strict, -mixed, and -repair flag speed improvements.

Version 7.90: February 1, 2018

- Minor change to installation commands for tcsh.

Version 7.80: January 12, 2018

- Updated setup.sh script with additional error checking.

2017

Version 7.70: December 27, 2017

- Added archive-pubmed script to automate local record archiving.

Version 7.60: November 15, 2017

- Epost -id numeric argument bug fixed.
- Xtract conditional tests can now use subrange specifiers.
- Xtract -strict and -mixed use separate -repair flag to normalize Unicode superscripts and subscripts.

Version 7.50: October 31, 2017

- Setup instructions now work with the tcsh shell.
- API key value is taken from the NCBI_API_KEY environment variable.
- Efetch -format gb supports -style withparts and -style conwithfeat.
- Xtract supports optional element [min:max] substring extraction.
- Xtract -position supports [first|last|outer|inner|all] argument values.
- Added prepare-stash script for local record archive.

Version 7.40: September 27, 2017

- Xtract -hash reports checksums for local record archiving.
- Initial support for API keys.

Version 7.30: September 6, 2017

- Modified stash-pubmed script to work around Cygwin artifact.

- Removed unpack-pubmed script.
- Xtract -archive replaces -stash for local record archiving.
- Xtract -gzip allows compression of archived XML records.

Version 7.20: August 28, 2017

- Added download-pubmed, download-sequence, unpack-pubmed, stash-pubmed, and fetch-pubmed scripts, for experimental local record storage.
- Xtract -flags [strict|mixed] added to support new local storage scripts.
- Removed obsolete, original Perl implementation of xtract.pl.

Version 7.10: August 10, 2017

- Xtract -ascii converts non-ASCII Unicode to hexadecimal numeric character references.
- Setup script recognizes Cygwin running under the MinGW emulator.

Version 7.00: July 10, 2017

- Xtract -mixed and -strict handle multiply-escaped HTML tags.
- Efetch removes normal and escaped HTML tags from PubMed fields.
- Esearch -field processes individual query terms using the designated field, also removing stop words.
- Esearch -pairs splits the query phrase into adjacent overlapping word pairs.

Version 6.90: July 5, 2017

- Xtract -mixed replaces -relaxed, and -accent replaces -plain.
- Efetch uses larger chunks for -format uid, url, and acc.
- Esearch -log shows constructed URL and QueryTranslation result.

Version 6.80: June 8, 2017

- Modified download instructions to use edirect.tar.gz archive.
- The ftp-cp script can now read from stdin without the need for xargs.
- Rerunning ftp-cp or asp-cp only attempts to download missing files.

Version 6.70: May 8, 2017

- Added asp-cp script for faster download of NCBI ftp files using Aspera Connect.
- Xtract -strict and -relaxed handle empty HTML tag variants (e.g., and <sup/>).

Version 6.60: April 25, 2017

- Xtract -strict replaces -degloss to remove HTML <i>, , <u>, <sup> and <sub> tags from XML contents.
- Xtract -relaxed allows HTML tags in XML contents, to support current PubMed ftp release files.

- Xtract -plain removes Unicode accents.
- The setup.sh script prints an error message if it cannot fetch missing Perl modules.

Version 6.50: March 6, 2017

- Xtract -degloss replaces -html to remove HTML <i>, , <u>, <sup> and <sub> tags.

Version 6.40: March 1, 2017

- Epost detects accession.version input for sequence databases and sets -format acc.
- Xtract -html [remove|encode] converts <i> and tags embedded in XML contents.

Version 6.30: February 13, 2017

- Efetch -format docsum skips GI-less sequences without summaries.
- Xtract local indexing commands moved to -extras documentation.

Version 6.20: January 30, 2017

- Xtract -limit and -index allow extraction of selected records from XML file.

Version 6.10: January 19, 2017

- Added run-ncbi-converter script for processing ASN.1 release files.
- Xtract -format flush option added.
- Removed obsolete accession-dot-version conversion code.

2016

Version 6.00: December 27, 2016

- Efetch -format docsum removes eSummaryResult wrapper.
- Fixed content truncation bug when Xtract encounters very long sequences.

Version 5.90: December 21, 2016

- Efetch and Elink readied for switch to accession-dot-version sequence identifier.
- Xtract -insd recognizes INSDInterval_iscomp@value and other boolean attributes.
- Xtract adds experimental phrase processing commands for word index preparation.

Version 5.80: December 12, 2016

- Efilter adds shortcuts for -db gene (e.g., -status alive, -type coding).
- Xtract numeric conditional tests can use an element name for the second argument (e.g., -if ChrStop -lt ChrStart finds minus strand genes).

Version 5.70: November 30, 2016

- Xtract -format takes an optional [compact|indent|expand] argument. Processing compact XML is about 15% faster than indent form. Using expand places each attribute on a separate line for ease of reading.

Version 5.60: November 22, 2016

- Fixed bug in -datatype argument for Esearch and Efilter.
- Added optional argument to filter-stop-words script to indicate replacement.

Version 5.50: November 16, 2016

- Efetch -id allows non-numeric accessions only for sequence databases.
- Xtract element selection no longer considers fields in recursive sub-objects.
- Xtract introduces a double-star "**/Object" construct to flatten recursive child objects for linear exploration.
- Xtract conditional tests ignore empty self-closing tags.
- Xtract -else simplifies insertion of a placeholder to indicate missing data.

Version 5.40: November 7, 2016

- Added filter-stop-words and xy-plot scripts.

Version 5.30: October 31, 2016

- Added support for ecitmatch utility.
- Added amino-acid-composition and between-two-genes scripts.
- The sort-uniq-count and sort-uniq-count-rank scripts take an optional argument (e.g., -n for numeric comparisons, -r to reverse order).

Version 5.20: October 26, 2016

- Setup script no longer modifies the user's configuration file to update the PATH variable. Instead, it now prints customized instructions for the user to execute. The user may choose to run these commands, but is free to edit the .bash_profile file manually.
- Xtract deprecates -match and -avoid functions and the Element:Value conditional shortcut.
- Xtract -if and -unless commands use compound statements for conditional execution (e.g., -if Element -equals Value).
- Colon now separates namespace prefix from element name in xtract arguments (e.g., -block jats:abstract). Colon at start of element name matches any namespace prefix.
- Xtract -insd uses a dash as placeholder for missing field. Experimental -insdx command is deprecated.
- Precompiled versions of xtract are now provided for Darwin, Linux, and CYGWIN_NT platforms. The appropriate executable is downloaded by the setup script.

Version 5.10: October 13, 2016

- Xtract adds -0-based, -1-based, and -ucsc numeric extraction/conversion commands for sequence positions from several Entrez databases.

Version 5.00: September 26, 2016

- Efetch -format fasta removes blank lines between records.
- Xtract -insdx uses a dash to indicate a missing field.
- Xtract -insd no longer has blank lines between records.
- Xtract -input allows reading XML data from a file.

Version 4.90: September 14, 2016

- Epost -input allows reading from an input file instead of using data piped through stdin.
- Efilter now supports the -sort argument.
- Xtract -filter can recover information in XML comments and CDATA blocks.

Version 4.80: August 9, 2016

- Xtract -insd controlled vocabularies updated.

Version 4.70: August 4, 2016

- Einfo -db request can also display -fields and -links data summaries.
- Einfo -dbs prints database names instead of eInfoResult XML.

Version 4.60: July 18, 2016

- Elink -cmd acheck returns information on all available links for a record.
- Efilter -pub structured limits to articles with structured abstracts.

Version 4.50: July 1, 2016

- Esearch and Efilter detect and report -query phrase quotation errors.
- Efilter -pub shortcut adds last_week, last_month, and last_year choices.
- Efetch sets -strand 2 for minus strand if -seq_start > -seq_stop or if -chr_start > -chr_stop.

Version 4.40: June 21, 2016

- Transitioning to use of https for access to NCBI services.
- Epost -db assembly -format acc uses [ASAC] field instead of [ACCN].

Version 4.30: June 13, 2016

- Efilter -pub preprint limits results to ahead-of-print articles.
- Xtract -pattern Parent/* construct can now process catenated XML files.

Version 4.20: May 24, 2016

- Xtract command-line argument parsing improvements.
- Nquire -get supersedes -http get.

Version 4.10: May 3, 2016

- Xtract -format removes multi-line XML comments and CDATA blocks.

Version 4.00: April 4, 2016

- Esearch adds -spell to correct known misspellings of biological terms in the query string.
- Efilter adds -spell to correct query misspellings, and -pub, -feature, -location, -molecule, -organism, and -source shortcuts. Run efilter -help to see the choices available for each argument.

Version 3.90: March 21, 2016

- Code optimizations for increased Xtract speed.

Version 3.80: February 29, 2016

- Xtract can distribute its work among available processor cores for additional speed.

Version 3.70: February 8, 2016

- Xtract performance improvements.

Version 3.60: January 11, 2016

- The setup.sh configuration script now downloads a precompiled Xtract executable for selected platforms.

2015

Version 3.50: December 27, 2015

- Xtract reports error for element:value construct outside of -match or -avoid arguments.

Version 3.40: December 20, 2015

- Xtract -insd supports extraction from multiple features (e.g., CDS,mRNA).

Version 3.30: December 3, 2015

- Efetch -format docsum can accept a single sequence accession number in the -id argument.

Version 3.20: November 30, 2015

- Xtract supports -match conditional execution on values recorded in variables.

Version 3.10: November 18, 2015

- Efetch adds -chr_start and -chr_stop arguments to specify sequence range from 0-based coordinates in gene docsum GenomicInfoType object.

Version 3.00: October 30, 2015

- Xtract rewritten in the Go programming language for speed. The setup.sh configuration script installs an older Perl version (2.99) if a local Go compiler is unavailable.
- Efetch -format docsum only decodes HTML entity numbers in select situations.

Version 2.90: October 15, 2015

- Xtract warns on use of deprecated arguments -present, -absent, and -trim, in preparation for release of much faster version.

Version 2.80: September 9, 2015

- Xtract uses the "*/Child" construct for nested exploration into recursive structures, replacing the -trim argument.

Version 2.70: July 14, 2015

- Added entrez-phrase-search script to query on adjacent word pairs indexed in specific fields.

Version 2.60: June 23, 2015

- Xtract -match and -avoid support "Parent/Child" construct for BLAST XML.

Version 2.50: April 9, 2015

- Xtract capitalized -Pattern handles recursively-defined top-level objects.

Version 2.40: March 25, 2015

- EDirect programs use the http_proxy environment variable to work behind firewalls.

Version 2.30: March 11, 2015

- Cleaned up logic in setup.sh configuration script.
- EPost -format acc works properly on protein accessions.

Version 2.20: March 4, 2015

- Xtract -match and -avoid recognize "@attribute" without element or value.

Version 2.10: February 3, 2015

- Added ftp-ls and ftp-cp scripts for convenient access to the NCBI anonymous ftp server.

2014

Version 2.00: August 28, 2014

- Introduced copy-and-paste installation commands with setup.sh configuration script.

Version 1.90: August 8, 2014

- Xtract -format combines multiple XML results into a single valid object.

- Improved suppression of 0-count failure messages with -silent flag in scripts.

Version 1.80: July 15, 2014

- EPost -format acc accepts accessions in an -id argument on the command line.

Version 1.70: April 23, 2014

- EFetch -format docsum decodes HTML entity numbers embedded in the text.

Version 1.60: April 3, 2014

- Minor enhancements to xtract -insd.

Version 1.50: March 29, 2014

- Esearch -sort specifies the order of results when records are retrieved.
- Xtract exploration arguments (e.g., -block) now work on self-closing tags with data in attributes.

Version 1.40: March 17, 2014

- Xtract -format repairs XML line-wrapping and indentation.
- Implemented -help flag to display the list of command-line arguments for each function.

Version 1.30: March 3, 2014

- Xtract -insd partial logic was corrected to examine both 5' and 3' partial flags, and the location indicator recognizes "+" or "complete" and "-" or "partial".

Version 1.20: February 26, 2014

- Xtract -insd detects if it is part of an EDirect sequence record query, and dynamically executes the extraction request for specific qualifier values. When run in isolation it generates extraction instructions that can be incorporated (with modifications, if necessary) into other queries.

Version 1.10: February 10, 2014

- ESummary was replaced by "efetch -format docsum" to provide a single command for all document retrieval. The esummary command will continue to work for those who prefer it, and to avoid breaking existing scripts.
- Xtract processes each -pattern object immediately upon receipt, eliminating the need for using xargs and sh to split document retrieval into smaller units.

Version 1.00: February 6, 2014

- Initial public release.

2013

Version 0.00: April 23, 2013

- Initial check-in of web documentation page.

Version 0.00: March 20, 2013

- Initial check-in of edirect.pl script source code.

Entrez Direct Examples

Jonathan Kans, PhD¹

Created: April 23, 2013; Updated: October 8, 2026.

Some early EDirect examples required post-processing with shell script:

```
start=$(( start + 1 ))
stop=$(( stop + 1 ))
```

or awk commands:

```
awk -F '\t' -v 'OFS=\t' '{print $1, $2+1, $3+1}'
```

to increment 0-based sequence coordinates so that the proper region was retrieved:

```
efetch -db nuccore -id $accn -format fasta -seq_start $start -seq_stop $stop
```

This led to -element derivatives that could do simple adjustments within an xtract command:

```
-element ChrAccVer -inc ChrStart -1-based ChrStop
```

and to efetch arguments that could take 0-based coordinates directly:

```
efetch -db nuccore -id $accn -format fasta -chr_start $start -chr_stop $stop
```

These helped eliminate the need for scripts to perform otherwise trivial modifications on extracted data.

More recent work allows easier after-the-fact numeric and substring matching using filter-columns:

```
filter-columns '10 <= $2 && $2 <= 30 && $5 ~ peptide'
```

and text and number modification using print-columns:

```
print-columns '$1, $2+1, $3-1, "\042" $4 "\042", tolower($5), log($3), total += $2'
```

Both scripts accept column designators in an argument and pass them to internal awk commands. The NF (number of fields) and NR (current record number) built-in variables can also be used, as can YR (for year) and DT (for date in YYYY-MM-DD format).

PubMed

Author Frequency

Who are the most prolific authors on rattlesnake phospholipase?

```
esearch -db pubmed -query \
  "crotalid venoms [MAJR] AND phospholipase [TIAB]" |
efetch -format xml |
xtract -pattern PubmedArticle \
  -block Author -sep " " -tab "\n" -element LastName,Initials |
sort-uniq-count-rank
```

Placing author names on separate lines allows sort-uniq-count-rank to produce a frequency table:

```

87    Lomonte B
77    Gutiérrez JM
64    Soares AM
53    Marangoni S
43    Giglio JR
39    Bon C
...

```

Publication Distribution

When were the most papers about Legionnaires disease published?

```

esearch -db pubmed -query "legionnaires disease [TITL]" |
efetch -format docsum |
xtract -pattern DocumentSummary -element PubDate |
cut -c 1-4 |
sort-uniq-count-rank

```

In this case sort-uniq-count-rank reports the number of selected papers per year:

```

173    1979
102    1980
96     1978
92     1981
66     1983
...

```

Using -year "PubDate/*" on a PubmedArticle record takes the first four-digit number it encounters, and the result does not need to be trimmed:

```

esearch -db pubmed -query "legionnaires disease [TITL]" |
efetch -format xml |
xtract -pattern PubmedArticle -year "PubDate/*" |
sort-uniq-count-rank

```

Treatment Locations

What is the geographic distribution of sepsis treatment studies?

```

esearch -db pubmed -query \
"sepsis/therapy [MESH] AND geographic locations [MESH]" |
efetch -format xml |
xtract -pattern PubmedArticle \
-block MeshHeading -if DescriptorName@Type -equals Geographic \
-tab "\n" -element DescriptorName |
sort-uniq-count-rank

```

This returns the number of articles ranked by country or region:

```

660    United States
250    Spain
182    Germany
168    India
155    Taiwan
139    Japan
126    China
124    France
123    Europe
...

```

Note that England and United Kingdom will appear as two separate entries.

Indexed Date Fields

What date fields are indexed for PubMed?

```
einfo -db pubmed |
xtract -pattern Field \
  -if IsDate -equals Y -and IsHidden -equals N \
    -pfx "[" -sfx "]" -element Name \
    -pfx "" -sfx "" -element FullName |
sort -k 2f | expand
```

Indexed dates are shown with field abbreviations and descriptive names:

```
[CDAT] Date - Completion
[CRDT] Date - Create
[EDAT] Date - Entrez
[MHDA] Date - MeSH
[MDAT] Date - Modification
[PDAT] Date - Publication
```

Two other date fields have the IsHidden value set to "Y":

```
[EPDT] Electronic Publication Date
[PPDT] Print Publication Date
```

Digital Object Identifiers

How are digital object identifiers obtained from PubMed articles?

```
esearch -db pubmed -query "Rowley JD [AUTH]" |
efetch -format xml |
xtract -head '<html><body>' -tail '</body></html>' \
  -pattern PubmedArticle -PMID MedlineCitation/PMID \
  -block ArticleId -if @IdType -equals doi \
    -tab "" -pfx '<p><a href="' -sfx '"'>' -doi ArticleId \
    -tab "\n" -pfx '' -sfx '</a></p>' -element "&PMID" |
transmute -format
```

The `-doi` command extracts the DOIs and constructs URL references. The `-pfx` and `-sfx` arguments used here enclose each PMID with a clickable link to its DOI:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE html>
<html>
  <body>
    <p>
      <a href="https://doi.org/10.1038%2Fleu.2013.340">24496283</a>
    </p>
    <p>
      <a href="https://doi.org/10.1073%2Fpnas.1310656110">23818607</a>
    </p>
    ...
  </body>
</html>
```

Reference Formatting

How were references in the main EDirect documentation page formatted?

```
efetch -db pubmed -format docsum -id 8743683 31600197 17971238 \
23175613 11449725 26931183 31738401 31114887 14728215 29390967 |
```

Relevant document summary fields were collected with:

```
xtract -set Set -rec Rec -pattern DocumentSummary \
-sfp ", " -sfx "." -NM Name -clr \
-sfx "." -SRC Source -clr \
-pfx ":" -PG Pages -clr \
-pfx "(PMID " -sfx ".)" -ID Id -clr \
-group ArticleId -if IdType -equals doi \
-pfx "https://doi.org/" -sfx "." -DOI Value -PG "(.)" -clr \
-group DocumentSummary \
-wrp Sort -lower "&NM" \
-wrp Name -element "&NM" \
-wrp Title -element Title \
-wrp Source -element "&SRC" \
-wrp Year -year PubDate \
-wrp Pages -element "&PG" \
-wrp DOI -element "&DOI" \
-wrp Id -element "&ID" |
```

This generated intermediate XML with partial formatting:

```
...
<Rec>
  <Sort>wei ch, allot a, leaman r, lu z.</Sort>
  <Name>Wei CH, Allot A, Leaman R, Lu Z.</Name>
  <Title>PubTator central: automated concept ... full text articles.</Title>
  <Source>Nucleic Acids Res.</Source>
  <Year>2019</Year>
  <Pages>.</Pages>
  <DOI>https://doi.org/10.1093/nar/gkz389.</DOI>
  <Id>(PMID 31114887.)</Id>
</Rec>
...
```

References were sorted by the first author's last name:

```
xtract -set Set -rec Rec -pattern Rec -sort Sort |
```

and then printed with:

```
xtract -pattern Rec -deq "\n\n" -tab " " -sep "" \
-element Name Title Source Year,Pages DOI Id
```

followed by manual editing of hyphenated initials:

```
...
Wei C-H, Allot A, Leaman R, Lu Z. PubTator central: automated concept
annotation for biomedical full text articles. Nucleic Acids Res. 2019.
https://doi.org/10.1093/nar/gkz389. (PMID 31114887.)
...
```

Using xtract **-reg** and **-exp** regular expressions to guide xtract **-replace**:

```
efetch -db pubmed -id 31114887 -format xml |
xtract -pattern PubmedArticle -block Author -deq "\n" \
-element LastName -reg "[a-z .]" -exp "" -replace ForeName
```

can generate hyphenated initials directly from the PubmedArticle ForeName field:

```
<Author>
  <LastName>Wei</LastName>
  <ForeName>Chih-Hsuan</ForeName>
  <Initials>CH</Initials>
</Author>
```

by removing lower-case letters, spaces, and periods.

Nucleotide

Coding Sequences

What are the coding sequences in the *Escherichia coli* lac operon?

```
efetch -db nuccore -id J01636.1 -format gbc |
xtract -insd CDS gene sub_sequence
```

Sequence under a feature location is obtained with the `-insd "sub_sequence"` argument:

```
J01636.1    lacI    GTGAAACCAGTAACGTTATACGATGTCGCAGAGTATGCCG...
J01636.1    lacZ    ATGACCATGATTACGGATTCACTGGCCGTCGTTTTACAAC...
J01636.1    lacY    ATGTACTATTTAAAAAACACAACTTTTGGATGTTTCGGTT...
J01636.1    lacA    TTGAACATGCCAATGACCGAAAGAATAAGAGCAGGCAAGC...
```

Untranslated Region Sequences

What are the 5' and 3' UTR sequences for lycopene cyclase mRNAs?

```
SubSeq() {
  xtract -pattern INSDSeq -ACC INSDSeq_accession-version -SEQ INSDSeq_sequence \
    -group INSDFeature -if INSDFeature_key -equals CDS -PRD "(" ")" \
    -block INSDQualifier -if INSDQualifier_name -equals product \
      -PRD INSDQualifier_value \
    -block INSDFeature -pfc "\n" -element "&ACC" -rst \
      -first INSDInterval_from -last INSDInterval_to -element "&PRD" "&SEQ"
}

UTRs() {
  while IFS=$'\t' read acc fst lst prd seq
  do
    if [ $fst -gt 1 ]
    then
      echo -e ">$acc 5'UTR: 1..$((fst-1)) $prd"
      echo "${seq:1:$((fst-2))}" | fold -w 50
    else
      echo -e ">$acc NO 5'UTR"
    fi
    if [ $lst -lt ${#seq} ]
    then
      echo -e ">$acc 3'UTR: $((lst+1))..${#seq} $prd"
      echo "${seq:$lst}" | fold -w 50
    else
      echo -e ">$acc NO 3'UTR"
    fi
  done
}
```

```

esearch -db nuccore -query "5.5.1.19 [ECN0]" |
efilter -molecule mrna -feature cds -source refseq |
efetch -format gbc |
SubSeq | UTRs

```

Sequences before the start codon and after the stop codon are obtained with Unix substring commands:

```

>NM_001324787.1 NO 5'UTR
>NM_001324787.1 NO 3'UTR
>NM_001324979.1 5'UTR: 1..262 lycopene beta cyclase, chloroplastic/chromoplastic
attatagaaataacttaagatatatcattgccctttaatcatttattttta
actcttttaagtgtttaaagattgattctttgtacatgttctgcttcatt
tgtgttgaaaattgagttgttttcttgaattttgcaagaatataggggac
ccatttgtgttgaaaattgagcagctttctttgtgttttgttcgatttt
tcaagaatataggacccccattttctgttttcttgagataaattgcacctt
gttgggaaaat
>NM_001324979.1 3'UTR: 1760..1782 lycopene beta cyclase, chloroplastic/chromoplastic
attcgacttatctgggatcttgt
...

```

5-Column Feature Table

How can you generate a 5-column feature table from GenBank format?

```

XtoT() {
  xtract -pattern INSDSeq -pfx ">Feature " \
    -first INSDSeqid,INSDSeq_accession-version \
  -group INSDFeature -FKEY INSDFeature_key \
    -block INSDInterval -deq "\n" \
      -element INSDInterval_from INSDInterval_to \
        INSDInterval_point INSDInterval_point \
        "&FKEY" -FKEY "()" \
    -block INSDQualifier -deq "\n\t\t\t\t" \
      -element INSDQualifier_name INSDQualifier_value
}

efetch -db nuccore -id U54469 -format gb |
transmute -g2x |
XtoT

```

Exploring the INSDSeq XML hierarchy with a `-pattern {sequence} -group {feature} -block {qualifier}` construct, and adding proper indentation, can produce feature table submission format:

```

>Feature U54469.1
1      2881      source
                                organism      Drosophila melanogaster
                                mol_type      genomic DNA
                                db_xref      taxon:7227
                                chromosome    3
                                map          67A8-B2

80      2881      gene
                                gene          eIF4E

80      224      mRNA
892      1458
1550     1920
1986     2085
2317     2404
2466     2881

```

gene	eIF4E
product	eukaryotic initiation factor 4E-I
...	

An **xml2tbl** script containing this xtract command is now included with EDirect.

WGS Components

How can you get FASTA format for all components of a WGS project?

```
GenerateWGSAccessions() {
    gb=$( efetch -db nuccore -id "$1" -format gbc < /dev/null )
    ln=$( echo "$gb" | xtract -pattern INSDSeq -element INSDSeq_length )
    pf=$(
        echo "$gb" |
        xtract -pattern INSDSeq -element INSDSeq_locus |
        sed -e 's/010*/01/g'
    )
    seq -f "${pf}%07.0f" 1 "$ln"
}

GenerateWGSAccessions "JABRPF000000000" |
```

returns the expanded list of accessions from the WGS master:

```
JABRPF010000001
JABRPF010000002
JABRPF010000003
...
JABRPF010000061
JABRPF010000062
JABRPF010000063
```

Piping those accessions to:

```
efetch -db nuccore -format fasta
```

retrieves the individual components in FASTA format:

```
>JABRPF010000001.1 Enterococcus faecalis strain G109-2 contig00001, ...
ACACTAATATGTGTCTTTTACACTAGCTCACTAAAAAATAGTCATAATTTCTTCATTATTTAAAT
CCAACAATTGTGAAATCAATTTAATATCCGATGCTTTGAAAACAACCTCTCTTTTAATTTTTTGTAAT
CGTTGAAGCGGATATTGTTGCCGATGCAGTCATTTCAATTTGCCAACCACTCAACATTTTTTCCTTTC
...
```

Proteins in a Region

How do you get the protein records encoded in a specific region of a nucleotide sequence?

```
efetch -db nuccore -id NC_015162.1 -format gb -seq_start 9125 -seq_stop 103803 |
xtract -insd CDS protein_id |
cut -f 2 |
efetch -db protein -format fasta
```

extracts the protein_id accessions and retrieves those proteins in FASTA format:

```
>WP_013615770.1 hypothetical protein [Deinococcus proteolyticus]
MTQNASAGLTAETLMTETGLSAAVRRALKAYDAAFGLEHPTQDGLHLTPAEYDVLRRALQLTGGYAPG
LKLWFGEQQALALSTQPVASPREVQQLSQLYQQVESYRARPPEEPAQTLRALLDQAQALGWSGFWEMGRL
QGAALFVLGVLRFDGQRAKVSMQTPLSGAEALVLSRGVCALLQRVRTQPGSGQAWSWNEVLLEEVERIL
```

```
KDLTE
>WP_013615771.1 hypothetical protein [Deinococcus proteolyticus]
MNKDTNVDTAFAGGFFTTFENGECVFYDADGVCHSVTPDTVAAVVASGTPRDVVALHEDVQGRTDAVAQV
...
```

Reverse Complement GenBank

How do you reverse complement a nucleotide sequence in GenBank format?

```
efetch -db nuccore -id J01749 -format gb |
gbf2fsa |
transmute -revcomp |
transmute -fasta -width 50
```

converts GenBank to FASTA, reverse-complements the sequence, and saves it at the selected 50 characters per line:

```
TTCTTGAAGACGAAAGGGCCTCGTGATACGCCTATTTTTATAGGTTAATG
TCATGATAATAATGGTTTCTTAGACGTCAGGTGGCACTTTTCGGGGAAAT
GTGCGCGGAACCCCTATTTGTTTATTTTCTAAATACATTCAAATATGTA
...
ACGATGAGCGCATTGTTAGATTTTCATACACGGTGCCTGACTGCGTTAGCA
ATTTAACTGTGATAAACTACCGCATTAAAGCTTATCGATGATAAGCTGTC
AAACATGAGAA
```

Reverse Complement NCBI2NA

How do you reverse complement a 2-bit encoded nucleotide sequence?

```
efetch -db nuccore -id J01749 -format asn |
xtract -pattern Seq-entry \
-group seq -if ncbi2na \
-block inst -LEN length -SEQ -ncbi2na ncbi2na \
-SUB "&SEQ[:&LEN]" -REV -revcomp "&SUB" \
-sep "\n" -fasta "&REV"
```

expands the NCBI2NA representation to IUPACNA, truncates at the indicated length, reverse-complements that sequence, and saves it at the default 70 characters per line:

```
TTCTTGAAGACGAAAGGGCCTCGTGATACGCCTATTTTTATAGGTTAATGTCATGATAATAATGGTTTCT
TAGACGTCAGGTGGCACTTTTCGGGGAAATGTGCGCGGAACCCCTATTTGTTTATTTTCTAAATACATT
CAAATATGTATCCGCTCATGAGACAATAACCCTGATAAATGCTTCAATAATATTGAAAAAGGAAGAGTAT
...
GCATAACCAAGCCTATGCCTACAGCATCCAGGGTGACGGTGCCGAGGATGACGATGAGCGCATTGTTAGA
TTTCATACACGGTGCCTGACTGCGTTAGCAATTTAACTGTGATAAACTACCGCATTAAAGCTTATCGATG
ATAAGCTGTCAAACATGAGAA
```

Six-Frame Protein Translation

How do you translate all six reading frames of a nucleotide sequence?

```
efetch -db nuccore -id J01749 -format fasta |
transmute -cds2prot -circular -gcode 11 -all
```

will simultaneously translate three plus-strand and three minus-strand frames, including codons that span the origin if the molecule is circular:

```
>J01749.1-1+
FSCLTAYHR*ALMR*FITVKLLTQSGTVYE ... KCHLTSKKPLLS*H*PIKIGVSRGPFVFKN
>J01749.1-2+
```

```
SHV*QLIIDKL*CGSLSQLNC*RSQAPCMK ... SAT*RLRNHYHDINL*K*AYHEALSSSRI
>J01749.1-3+
LMFDSLSSISFNAVYHS*IANAVRHRV*N ... VPPDV*ETIIIMTLTYKNRRITRPFRLQEF
>J01749.1-1-
ILEDERAS*YAYFYRLMS***WFLRRQVAL ... FIHGA*LR*QFNCDKLPH*SLSMISCQT*E
>J01749.1-2-
NS*RRKGLVIRLFL*VNVMIIMVS*TSGGT ... FHTRCLTALAI*L**TTALKLIDDKLSNMR
>J01749.1-3-
EFLKTKGPRDTPIFIG*CHDNGFLDVRWH ... ISYTPDCVSNLTVINYRIKAYR**AVKHE
```

Pattern Searching

The pBR322 cloning vector is a circular plasmid with unique restriction sites in two antibiotic resistance genes.

The `transmute -replace` function introduces a second BamHI restriction enzyme recognition site by modifying two bases in the ribosomal binding site of the `rop` (restrictor of plasmid copy number) gene:

```
efetch -db nuccore -id J01749 -format fasta |
transmute -replace -offset 1907 -delete GG -insert TC |
```

The `transmute -search` function takes a list of sequence patterns with optional labels (such as restriction enzyme names), and uses a finite-state algorithm to simultaneously search for all patterns:

```
transmute -search -circular GGATCC:BamHI GAATTC:EcoRI CTGCAG:PstI |
align-columns -g 4 -a rl
```

The (0-based) starting positions and labels for each match are then printed in a two-column table:

```
374    BamHI
1904   BamHI
3606   PstI
4358   EcoRI
```

The `disambiguate-nucleotides` and `systematic-mutations` scripts can generate all possible single-base substitutions in a pattern for a more relaxed search.

Protein

Amino Acid Composition

What is the amino acid composition of human titin?

```
#!/bin/bash -norc

AAComp() {

  abbrev=( Ala Asx Cys Asp Glu Phe Gly His Ile \
            Xle Lys Leu Met Asn Pyl Pro Gln Arg \
            Ser Thr Sec Val Trp Xxx Tyr Glx )

  tr A-Z a-z |
  sed 's/[^a-z]//g' |
  fold -w 1 |
  sort-uniq-count |
  while read num lttr
  do
    idx=$(printf %i "$lttr")
    ofs=$((idx-97))
    echo -e "${abbrev[$ofs]}\t$num"
```

```

done |
sort
}

efetch -db protein -id Q8WZ42 -format gpc |
xtract -pattern INSDSeq -element INSDSeq_sequence |
AAComp

```

This produces a table of residue counts using the three-letter amino acid abbreviations:

Ala	2084
Arg	1640
Asn	1111
Asp	1720
Cys	513
Gln	942
Glu	3193
Gly	2066
His	478
Ile	2062
Leu	2117
Lys	2943
Met	398
Phe	908
Pro	2517
Ser	2463
Thr	2546
Trp	466
Tyr	999
Val	3184

Longest Sequences

What are the longest known insulin precursor molecules?

```

esearch -db protein -query "insulin [PROT]" |
efetch -format docsum |
xtract -pattern DocumentSummary -element Caption Slen Title |
grep -v receptor | sort -k 2,2nr | head -n 5 | cut -f 1 |
xargs -n 1 sh -c 'efetch -db protein -id "$0" -format gp > "$0".gpf'

```

Post-processing excludes the longer "insulin-like receptor" sequences, sorts by sequence length, and saves the GenPept results to individual files named by their sequence accessions, using the right angle bracket (">") Unix output redirection character:

```

EFN61235.gpf
EFN80340.gpf
EGW08477.gpf
EKC18433.gpf
ELK28555.gpf

```

Archaea Enzyme

Which archaeobacteria have chloramphenicol acetyltransferase?

```

esearch -db protein -organism archaea \
-query "chloramphenicol acetyltransferase [PROT]" |
efetch -format gpc |
xtract -pattern INSDSeq -element INSDSeq_organism INSDSeq_definition |

```

```
grep -i chloramphenicol | grep -v MULTISPECIES |
cut -f 1 | sort -f | uniq -i
```

Filtering on the definition line produces a list of archaeal organism names:

```
Euryarchaeota archaeon
Methanobrevibacter arboriphilus
Methanobrevibacter gottschalkii
Methanobrevibacter millerae
Methanobrevibacter oralis
...
```

Gene

Gene Counts

How many genes are on each human chromosome?

```
for chr in {1..22} X Y MT
do
  esearch -db gene -query "Homo sapiens [ORGN] AND $chr [CHR]" |
  efilter -status alive -type coding |
  efetch -format docsum |
  xtract -pattern DocumentSummary -NAME Name \
    -block GenomicInfoType -if ChrLoc -equals "$chr" \
    -tab "\n" -element ChrLoc,"&NAME" |
  sort | uniq | cut -f 1 |
  sort-uniq-count-rank |
  reorder-columns 2 1
done
```

This returns a count of unique protein-coding genes per chromosome:

1	2011
2	1224
3	1046
4	742
5	854
6	1015
7	911
8	666
9	763
10	718
11	1279
12	1012
13	327
14	601
15	585
16	835
17	1147
18	266
19	1391
20	528
21	232
22	429
X	839
Y	63
MT	13

The range construct cannot be used for Roman numerals, so the equivalent query on *Saccharomyces cerevisiae* would need to explicitly list all chromosomes, including the mitochondrion:

```
for chr in I II III IV V VI VII VIII IX X XI XII XIII XIV XV XVI MT
```

Plastid genes can be selected with "source plastid [PROP]" or -location plastid.

Chromosome Locations

Where are mammalian calmodulin genes located?

```
esearch -db gene -query "calmodulin * [PFN] AND mammalia [ORGN]" |
efetch -format docsum |
xtract -pattern DocumentSummary \
-def "-" -element Id Name MapLocation ScientificName
```

The -def command adds a dash to prevent missing data from shifting columns in the table:

801	CALM1	14q32.11	Homo sapiens
808	CALM3	19q13.32	Homo sapiens
805	CALM2	2p21	Homo sapiens
24242	Calm1	6q32	Rattus norvegicus
12313	Calm1	12 E	Mus musculus
50663	Calm2	6q12	Rattus norvegicus
24244	Calm3	1q21	Rattus norvegicus
12315	Calm3	7 9.15 cM	Mus musculus
12314	Calm2	17 E4	Mus musculus
80796	Calm4	13 A1	Mus musculus
617095	CALM1	-	Bos taurus
396838	CALM3	-	Sus scrofa
520277	CALM3	-	Bos taurus
364774	Calm15	17q12.2	Rattus norvegicus
...			

Exon Counts

How many exons are in each dystrophin transcript variant?

```
esearch -db gene -query "DMD [GENE] AND human [ORGN]" |
efetch -format docsum |
xtract -pattern DocumentSummary -block GenomicInfoType \
-tab "\n" -element ChrAccVer,ChrStart,ChrStop |
xargs -n 3 sh -c 'efetch -db nuccore -format gbc \
-id "$0" -chr_start "$1" -chr_stop "$2"' |
```

This retrieves an INSDSeq XML subset record for the indicated gene region, which contains a number of alternatively-spliced dystrophin mRNA and CDS features.

Data extraction computes the number of intervals for each mRNA location (individual exons plus non-adjacent UTRs), and obtains the transcript sequence accession, transcript length, and product name from qualifiers:

```
xtract -insd complete mRNA "#INSDInterval" \
transcript_id "%transcription" product |
```

Final filtering and sorting:

```
grep -i dystrophin |
sed 's/dystrophin, transcript variant //g' |
sort -k 2,2nr -k 4,4nr
```

results in a table of exon counts and transcript lengths:

NC_000023.11	79	NM_004010.3	14083	Dp427p2
NC_000023.11	79	NM_004009.3	14000	Dp427p1
NC_000023.11	79	NM_004006.3	13992	Dp427m
NC_000023.11	79	NM_000109.4	13854	Dp427c
NC_000023.11	78	XM_006724468.2	13923	X1
NC_000023.11	78	XM_017029328.1	13916	X4
NC_000023.11	78	XM_006724469.3	13897	X2
NC_000023.11	77	XM_006724470.3	13875	X3
...				

Upstream Sequences

What sequences are upstream of phenylalanine hydroxylase genes?

```
esearch -db nuccore -query "U49897 [ACCN]" |
elink -target gene |
elink -target homologene |
elink -target gene |
efetch -format docsum |
xtract -pattern DocumentSummary -if GenomicInfoType -element Id \
-block GenomicInfoType -element ChrAccVer -1-based ChrStart ChrStop |
```

This produces a table with 1-based sequence coordinates:

5053	NC_000012.12	102958441	102836889
18478	NC_000076.7	87357657	87419999
38871	NT_037436.4	7760453	7763166
24616	NC_005106.4	28066639	28129772
378962	NC_007115.7	17409367	17391680
...			

Then, given a shell script named "upstream.sh":

```
#!/bin/bash -norc

bases=1500
if [ -n "$1" ]
then
    bases="$1"
fi

while read id accn start stop
do
    if [ $start -eq 0 ] || [ $stop -eq 0 ] || [ $start -eq $stop ]
    then
        echo "Skipping $id due to ambiguous coordinates"
        continue
    fi
    if [ $start -gt $stop ]
    then
        stop=$(( start + bases ))
        start=$(( start + 1 ))
        strand=2
    else
        stop=$(( start - 1 ))
        start=$(( start - bases ))
        strand=1
    fi
fi
```

```

rslt=$( efetch -db nuccore -id $accn -format fasta \
        -seq_start $start -seq_stop $stop -strand $strand < /dev/null )
echo "$rslt"
done

```

the data lines can be piped through:

```
upstream.sh 500
```

to extract and print the 500 nucleotides immediately upstream of each gene:

```

>NC_000012.12:c102958941-102958442 Homo sapiens chromosome 12, GRCh38.p13 ...
TGAAGTCGAGAAGCTCCTGCTCCTCGGGGCTGAGCGGGTCGTAAGAGCCCTCGTCCGACGAGTAGGATGA
GACCGGCGAGCCGGCCATGGAGTTCAAGTCGTTGGAGTAGTTGGGGGAGATGGTGGGCGACAGGACGCCT
GCCTGGAAGGCGGCGCTACCGCGTCATGCTCGTCCAGCAGCTGCTGCAGCGCGGGATGTACTCGACCG
...

```

Interactions Table

What genes interact with human epidermal growth factor receptor?

The Interactions table for the human EGFR (epidermal growth factor receptor) gene at:

<https://www.ncbi.nlm.nih.gov/gene/?term=1956#interactions>

can be reconstructed as a tab-delimited table without hyperlinks using fresh data and a processing chain of xtract commands. After downloading the gene record in XML format:

```
efetch -db gene -id 1956 -format xml |
```

the first xtract command isolates the "Interactions" section and saves its components as individual entries:

```

xtract -set Set -pattern Entrezgene \
-wrp GENEID -GENEID Gene-track_geneid -wrp SYMBOL -SYMBOL Gene-ref_locus \
-division Entrezgene_comments \
-group Gene-commentary -if Gene-commentary_heading -equals Interactions \
-branch "*/Gene-commentary" -pkg Rec -element "&GENEID" "&SYMBOL" \
-block "Gene-commentary/*" -element "*" |

```

These are piped to a second xtract command that collects and reformats the source reference objects:

```

xtract -transform <( echo -e "1\tSOURCE\n2\tPRODUCT\n3\tOTHERGENE\n4\tINTERACT\n" ) \
-rec Rec -pattern Rec -if "#Other-source" -eq 4 \
-wrp DESCR -element Gene-commentary_text -wrp PMIDS -sep "|" -element PubMedId \
-group GENEID -element "*" \
-group SYMBOL -element "*" \
-group Other-source -PS "+" -DB "" -ID "" -AC "" \
-DB Dbtag_db -ID -first "Object-id_str,Object-id_id" -AC Other-source_anchor \
-branch Other-source -DBID "" -ACDBID "" \
-block Other-source -if "&DB" -and "&ID" \
-sep ":" -DBID "&DB,&ID" \
-block Other-source -if "&AC" -and "&DBID" \
-sep " [" -sfx "]" -ACDBID "&AC,&DBID" \
-block Other-source \
-pfx "<" -sfx ">" -translate "&PS" -rst \
-first "&ACDBID,&AC,&DBID" \
-pfx "</" -sfx ">" -translate "&PS" |

```

storing them in XML objects that use the name of the column:

```
...
<Rec>
  <DESCR>EGFR interacts with calmodulin</DESCR>
  <PMIDS>14960328</PMIDS>
  <GENEID>1956</GENEID>
  <SYMBOL>EGFR</SYMBOL>
  <SOURCE>BIND [BIND:58062]</SOURCE>
  <PRODUCT>NP_005219.2 [Protein:29725609]</PRODUCT>
  <OTHERGENE>CALM1 [GeneID:801]</OTHERGENE>
  <INTERACT>NP_008819.1 [Protein:5901912]</INTERACT>
</Rec>
...
```

This is passed to a final xtract command that prints the fields in the desired order:

```
xtract -pattern Rec -def "-" \
  -element GENEID SYMBOL PRODUCT INTERACT OTHERGENE COMPLEX SOURCE PMIDS DESCR
```

Assembly

Complete Genomes

What complete genomes are available for *Escherichia coli*?

```
esearch -db assembly -query \
  "Escherichia coli [ORGN] AND representative [PROP]" |
elink -target nuccore -name assembly_nuccore_refseq |
efetch -format docsum |
xtract -pattern DocumentSummary -element AccessionVersion Slen Title |
sed 's/,.*///' |
sort-table -k 2,2nr
```

This search finds genomic assemblies and sorts the results by sequence length, allowing complete genomes to be easily distinguished from smaller plasmids:

```
NC_002695.2      5498578      Escherichia coli 0157:H7 str. Sakai DNA
NC_000913.3      4641652      Escherichia coli str. K-12 substr. MG1655
NC_002128.1      92721       Escherichia coli 0157:H7 str. Sakai plasmid p0157
NC_002127.1      3306        Escherichia coli 0157:H7 str. Sakai plasmid pOSAK1
```

The sed command removes text (e.g., complete genome, complete sequence, primary assembly) after a comma.

A similar query for humans, additionally filtering out scaffolds, contigs, and plasmids:

```
esearch -db assembly -query "Homo sapiens [ORGN] AND representative [PROP]" |
elink -target nuccore -name assembly_nuccore_refseq |
efetch -format docsum |
xtract -pattern DocumentSummary -element AccessionVersion Slen Title |
sed 's/,.*///' |
grep -v scaffold | grep -v contig | grep -v plasmid | grep -v patch |
sort
```

returns the assembled chromosome and mitochondrial sequence records:

```
NC_000001.11      248956422     Homo sapiens chromosome 1
NC_000002.12      242193529     Homo sapiens chromosome 2
NC_000003.12      198295559     Homo sapiens chromosome 3
NC_000004.12      190214555     Homo sapiens chromosome 4
NC_000005.10      181538259     Homo sapiens chromosome 5
NC_000006.12      170805979     Homo sapiens chromosome 6
```

NC_000007.14	159345973	Homo sapiens chromosome 7
NC_000008.11	145138636	Homo sapiens chromosome 8
NC_000009.12	138394717	Homo sapiens chromosome 9
NC_000010.11	133797422	Homo sapiens chromosome 10
NC_000011.10	135086622	Homo sapiens chromosome 11
NC_000012.12	133275309	Homo sapiens chromosome 12
NC_000013.11	114364328	Homo sapiens chromosome 13
NC_000014.9	107043718	Homo sapiens chromosome 14
NC_000015.10	101991189	Homo sapiens chromosome 15
NC_000016.10	90338345	Homo sapiens chromosome 16
NC_000017.11	83257441	Homo sapiens chromosome 17
NC_000018.10	80373285	Homo sapiens chromosome 18
NC_000019.10	58617616	Homo sapiens chromosome 19
NC_000020.11	64444167	Homo sapiens chromosome 20
NC_000021.9	46709983	Homo sapiens chromosome 21
NC_000022.11	50818468	Homo sapiens chromosome 22
NC_000023.11	156040895	Homo sapiens chromosome X
NC_000024.10	57227415	Homo sapiens chromosome Y
NC_012920.1	16569	Homo sapiens mitochondrion

This process can be automated to loop through a list of specified organisms:

```
for org in \
  "Agrobacterium tumefaciens" \
  "Bacillus anthracis" \
  "Escherichia coli" \
  "Neisseria gonorrhoeae" \
  "Pseudomonas aeruginosa" \
  "Shigella flexneri" \
  "Streptococcus pneumoniae"
do
  esearch -db assembly -query "$org [ORGN]" |
  efilter -query "representative [PROP]" |
  elink -target nuccore -name assembly_nuccore_refseq |
  efetch -format docsum |
  xtract -pattern DocumentSummary -element AccessionVersion Slen Title |
  sed 's/,.*//' |
  grep -v -i -e scaffold -e contig -e plasmid -e sequence -e patch |
  sort-table -k 2,2nr
done
```

which generates:

NC_011985.1	4005130	Agrobacterium radiobacter K84 chromosome 1
NC_011983.1	2650913	Agrobacterium radiobacter K84 chromosome 2
NC_005945.1	5228663	Bacillus anthracis str. Sterne chromosome
NC_003997.3	5227293	Bacillus anthracis str. Ames chromosome
NC_002695.1	5498450	Escherichia coli O157:H7 str. Sakai chromosome
NC_018658.1	5273097	Escherichia coli O104:H4 str. 2011C-3493 ...
NC_011751.1	5202090	Escherichia coli UMN026 chromosome
NC_011750.1	5132068	Escherichia coli IAI39 chromosome
NC_017634.1	4747819	Escherichia coli O83:H1 str. NRG 857C chromosome
NC_000913.3	4641652	Escherichia coli str. K-12 substr. MG1655
NC_002946.2	2153922	Neisseria gonorrhoeae FA 1090 chromosome
NC_002516.2	6264404	Pseudomonas aeruginosa PA01 chromosome
NC_004337.2	4607202	Shigella flexneri 2a str. 301 chromosome
NC_003028.3	2160842	Streptococcus pneumoniae TIGR4 chromosome
NC_003098.1	2038615	Streptococcus pneumoniae R6 chromosome

SNP

SNP Data Table

How can you obtain a tab-delimited table of SNP attributes from a document summary?

```
efetch -db snp -id 11549407 -format docsum |
snp2tbl
```

The **snp2tbl** script extracts fields from HGVS data and prints the individual values for further processing:

rs11549407	NC_000011.10	5226773	G	A	Genomic	Substitution	HBB
rs11549407	NC_000011.10	5226773	G	C	Genomic	Substitution	HBB
rs11549407	NC_000011.10	5226773	G	T	Genomic	Substitution	HBB
rs11549407	NG_000007.3	70841	C	A	Genomic	Substitution	HBB
rs11549407	NG_000007.3	70841	C	G	Genomic	Substitution	HBB
rs11549407	NG_000007.3	70841	C	T	Genomic	Substitution	HBB
...							
rs11549407	NM_000518.5	167	C	A	Coding	Substitution	HBB
rs11549407	NM_000518.5	167	C	G	Coding	Substitution	HBB
rs11549407	NM_000518.5	167	C	T	Coding	Substitution	HBB
rs11549407	NP_000509.1	39	Q	*	Protein	Termination	HBB
rs11549407	NP_000509.1	39	Q	E	Protein	Missense	HBB
rs11549407	NP_000509.1	39	Q	K	Protein	Missense	HBB

Columns are SNP identifier, accession.version, offset from start of sequence, letters to delete, letters to insert, sequence class, variant type, and gene name. The internal steps (**snp2hgvs**, **hgvs2spdi**, and **spdi2tbl**) are described below.

Amino Acid Substitutions

What are the missense products of green-sensitive opsin?

```
esearch -db gene -query "OPN1MW [PREF] AND human [ORGN]" |
elink -target snp | efilter -class missense |
efetch -format docsum |
```

SNP document summaries contain HGVS data of the form:

```
NC_000023.11:g.154193517C>A, ... , NP_000504.1:p.Ala285Val
```

This can be parsed by an xtract **-hgvs** command within the **snp2hgvs** script:

```
snp2hgvs |
```

into a structured representation of nucleotide and amino acid replacements:

```
...
<HGVS>
  <Id>782327292</Id>
  <Gene>OPN1MW</Gene>
  <Variant>
    <Class>Genomic</Class>
    <Type>Substitution</Type>
    <Accession>NC_000023.11</Accession>
    <Position>154193516</Position>
    <Deleted>C</Deleted>
    <Inserted>A</Inserted>
    <Hgvs>NC_000023.11:g.154193517C>A</Hgvs>
  </Variant>
```

```

...
<Variant>
  <Class>Coding</Class>
  <Type>Substitution</Type>
  <Accession>NM_000513.2</Accession>
  <Offset>853</Offset>
  <Deleted>C</Deleted>
  <Inserted>T</Inserted>
  <Hgvs>NM_000513.2:c.854C>T</Hgvs>
</Variant>
<Variant>
  <Class>Protein</Class>
  <Type>Missense</Type>
  <Accession>NP_000504.1</Accession>
  <Position>284</Position>
  <Deleted>A</Deleted>
  <Inserted>D</Inserted>
  <Hgvs>NP_000504.1:p.Ala285Asp</Hgvs>
</Variant>
<Variant>
  <Class>Protein</Class>
  <Type>Missense</Type>
  <Accession>NP_000504.1</Accession>
  <Position>284</Position>
  <Deleted>A</Deleted>
  <Inserted>V</Inserted>
  <Hgvs>NP_000504.1:p.Ala285Val</Hgvs>
</Variant>
</HGVS>
...

```

where the original 1-based HGVS positions are converted to 0-based in the XML.

Passing those results through the **hgvs2spdi** script:

hgvs2spdi |

converts CDS-relative offsets to sequence-relative positions:

```

...
<SPDI>
  <Id>782327292</Id>
  <Gene>OPN1MW</Gene>
  ...
  <Variant>
    <Class>Coding</Class>
    <Type>Substitution</Type>
    <Accession>NM_000513.2</Accession>
    <Position>935</Position>
    <Offset>853</Offset>
    <Deleted>C</Deleted>
    <Inserted>A</Inserted>
    <Hgvs>NM_000513.2:c.854C>A</Hgvs>
    <Spdi>NM_000513.2:935:C:A</Spdi>
  </Variant>
  <Variant>
    <Class>Coding</Class>
    <Type>Substitution</Type>
    <Accession>NM_000513.2</Accession>
    <Position>935</Position>

```

```

    <Offset>853</Offset>
    <Deleted>C</Deleted>
    <Inserted>T</Inserted>
    <Hgvs>NM_000513.2:c.854C>T</Hgvs>
    <Spdi>NM_000513.2:935:C:T</Spdi>
  </Variant>
  ...
</SPDI>
  ...

```

Piping this through the **spdi2tbl** script:

```
spdi2tbl |
```

completes the final step of the **snp2tbl** process to generate a SNP Data Table. Filtering that through:

```
grep Protein | grep Missense | cut -f 1-5
```

results in a table of amino acid substitutions sorted by accession, position, and residue:

```

rs1238141906    NP_000504.1    40    E    K
rs1189783086    NP_000504.1    42    P    L
rs1257135801    NP_000504.1    45    H    Y
rs1284438666    NP_000504.1    63    V    I
rs1223726997    NP_000504.1    64    I    T
  ...

```

Those rows are processed in a while loop that caches the current sequence data:

```

while read rsid accn ofs del ins
do
  if [ "$accn" != "$last" ]
  then
    seq=$( efetch -db protein -id "$accn" -format gpc < /dev/null |
           xtract -pattern INSDSeq -lower INSDSeq_sequence )
    last="$accn"
  fi
  pos=$((ofs + 1))
  echo ">$rsid [$accn $ins@$pos]"
  echo "$seq" |
  transmute -replace -offset "$ofs" -delete "$del" -insert "$ins" -lower |
  fold -w 50
done

```

and uses transmute **-replace** to generate modified FASTA with substituted residues in upper case:

```

>rs1238141906 [NP_000504.1 K@41]
maqqwslqrlagrhpdqsyedstqssiftytnsnstrgpfKgpnghiapr
wvyhltsvwmifvviavsvftnglvlaatmkfkklrhplnwilvnlavadl
aetviastisvvnqvygyfvlghpmcvlegytvsclgitglwslaiiswe
  ...

```

SNP-Modified Product Pairs

How can you match codon modifications with amino acid substitutions?

```

efetch -db snp -id 11549407 -format docsum |
snp2tbl |
tbl2prod

```

For SNPs that have different substitutions at the same position, the **tbl2prod** script translates coding sequences (after nucleotide modification), and sorts them with protein sequences (after residue replacement), to produce adjacent matching CDS/protein pairs:

```
rs11549407    NM_000518.5:167:C:T    MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWT*R...
rs11549407    NP_000509.1:39:Q:*          MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWT*R...
rs11549407    NM_000518.5:167:C:G        MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWTER...
rs11549407    NP_000509.1:39:Q:E          MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWTER...
rs11549407    NM_000518.5:167:C:A        MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWTKR...
rs11549407    NP_000509.1:39:Q:K          MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWTKR...
rs11549407    NM_000518.5:167:C:+        MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWTQR...
rs11549407    NP_000509.1:39:Q:+        MVHLTPEEKSAVTALWGKVVNDEVGGEALGRLLVVYPWTQR...
```

The "+" sign indicates the unmodified "wild-type" nucleotide or amino acid.

Structure

Structural Similarity

What archaea structures are similar to snake venom phospholipase?

```
esearch -db structure -query "crotalus [ORGN] AND phospholipase A2" |
elink -related |
efilter -query "archaea [ORGN]" |
efetch -format docsum |
xtract -pattern DocumentSummary \
    -if PdbClass -equals Hydrolase \
    -element PdbDescr |
sort -f | uniq -i
```

Structure neighbors use geometric comparison to find proteins that are too divergent to be detected by sequence similarity with a BLAST search:

```
Crystal Structure Of Autoprocessed Form Of Tk-Subtilisin
Crystal Structure Of Ca2 Site Mutant Of Pro-S324a
Crystal Structure Of Ca3 Site Mutant Of Pro-S324a
...
```

Taxonomy

Taxonomic Lineage

What are the major taxonomic lineage nodes for humans?

```
efetch -db taxonomy -id 9606 -format xml |
xtract -pattern Taxon -first TaxId -tab "\n" -element ScientificName \
    -block "**/Taxon" \
    -if Rank -is-not "no rank" -and Rank -is-not clade \
    -tab "\n" -element Rank,ScientificName
```

This uses the double star / child construct to recursively explore the data hierarchy:

```
9606          Homo sapiens
superkingdom  Eukaryota
kingdom       Metazoa
phylum      Chordata
subphylum   Craniata
superclass    Sarcopterygii
```

```

class          Mammalia
superorder     Euarchontoglires
order          Primates
...

```

Taxonomy Search

Which organisms contain an annotated RefSeq genome MatK gene?

```

esearch -db nuccore -query "MatK [GENE] AND NC_0:NC_999999999 [PACC]" |
efetch -format docsum |
xtract -pattern DocumentSummary -element TaxId |
sort -n | uniq |
epost -db taxonomy |
efetch -format docsum |
xtract -pattern DocumentSummary -element ScientificName |
sort

```

The first query obtains taxonomy UIDs from nucleotide document summaries and uploads them for separate retrieval from the taxonomy database:

```

Acidosasa purpurea
Acorus americanus
...
Zingiber spectabile
Zygnema circumcarinatum

```

BioSample

BioSample document summaries:

```
esummary -db biosample -id SAMN34375013 |
```

use XML attributes to identify data element types:

```
<Attribute attribute_name="strain" harmonized_name="strain" ...>KF24</Attribute>
```

A two-stage xtract pipeline can produce a tab-delimited table, with one column for each selected field. Piping the data to the first command:

```

xtract -rec BioSampleInfo -pattern DocumentSummary \
-wrp Accession -element Accession \
-wrp Title -element DocumentSummary/Title \
-wrp Link -sep "|" -numeric Links/Link \
-group Attribute -if @harmonized_name \
-TAG -lower @harmonized_name -wrp "&TAG" -element Attribute |

```

generates an intermediate form, with XML object names taken from the "harmonized_name" attributes:

```

<BioSampleInfo>
  <Accession>SAMN34375013</Accession>
  <Title>Microbe sample from Bacillus subtilis</Title>
  <Link>960711</Link>
  <isolation_source>rhizosphere soil</isolation_source>
  <collection_date>missing</collection_date>
  <geo_loc_name>China: Kaifeng, Henan Province</geo_loc_name>
  <sample_type>bacterial isolate</sample_type>
  <lat_lon>34.14 N 114.05 E</lat_lon>
  <strain>KF24</strain>
</BioSampleInfo>

```

(A **bsmp2info** script containing this first xtract command is now included with EDirect.)

Desired fields can then be selected by name in the last line of the second xtract command, with a "-" placeholder printed for any missing values:

```
xtract -pattern BioSampleInfo -def "-" -first Accession Title Link \
      geo_loc_name isolation_source strain lat_lon
```

Using `-first` instead of `-element` eliminates possible redundant entries during the attribute name transition from "country" to "geo_loc_name".

SRA

Using RunInfo Format

SRA data can be retrieved in RunInfo format:

```
efetch -db sra -id SRR6314034 -format runinfo |
```

as comma-separated values, with the first line containing the field names:

```
Run, ReleaseDate, LoadDate, spots, bases, spots_with_mates, avgLength, ...
SRR6314034, 2017-11-21 23:27:11, 2017-11-21 23:25:38, 128, 539118, 0, ...
```

Piping to the **csv2xml** script, and using the `-header` flag:

```
csv2xml -set Set -rec Rec -header |
```

converts the data into XML:

```
<Set>
  <Rec>
    <Run>SRR6314034</Run>
    <ReleaseDate>2017-11-21 23:27:11</ReleaseDate>
    <LoadDate>2017-11-21 23:25:38</LoadDate>
    <spots>128</spots>
    <bases>539118</bases>
    <spots_with_mates>0</spots_with_mates>
    <avgLength>4211</avgLength>
    <size_MB>0</size_MB>
    <AssemblyName></AssemblyName>
    <download_path>...</download_path>
    <Experiment>SRX3413965</Experiment>
    <LibraryName>child</LibraryName>
    <LibraryStrategy>AMPLICON</LibraryStrategy>
    <LibrarySelection>PCR</LibrarySelection>
    <LibrarySource>GENOMIC</LibrarySource>
    <LibraryLayout>SINGLE</LibraryLayout>
    <InsertSize>0</InsertSize>
    <InsertDev>0</InsertDev>
    <Platform>PACBIO_SMRT</Platform>
    <Model>PacBio RS II</Model>
    <SRASstudy>SRP125431</SRASstudy>
    <BioProject>PRJNA418990</BioProject>
    <Study_Pubmed_id></Study_Pubmed_id>
    <ProjectID>418990</ProjectID>
    <Sample>SRS2707133</Sample>
    <BioSample>SAMN08040264</BioSample>
    <SampleType>simple</SampleType>
    <TaxID>9606</TaxID>
```

```

<ScientificName>Homo sapiens</ScientificName>
<SampleName>BBS9_del_mother</SampleName>
<g1k_pop_code></g1k_pop_code>
<source></source>
<g1k_analysis_group></g1k_analysis_group>
<Subject_ID></Subject_ID>
<Sex>female</Sex>
<Disease></Disease>
<Tumor>no</Tumor>
<Affection_Status></Affection_Status>
<Analyte_Type></Analyte_Type>
<Histological_Type></Histological_Type>
<Body_Site></Body_Site>
<CenterName>ICAHN SCHOOL OF MEDICINE AT MOUNT SINAI</CenterName>
<Submission>SRA633054</Submission>
<dbgap_study_accession></dbgap_study_accession>
<Consent>public</Consent>
<RunHash>19AC4EB8A65D733274756464DCCF65EA</RunHash>
<ReadHash>AC8F39C51F95D9CD1BD0CBFBB669AD1E</ReadHash>
</Rec>
</Set>

```

This can then be piped through xtract:

```
xtract -pattern Rec -def "-" -element Run Experiment BioProject BioSample
```

to retrieve the desired values by field name:

```
SRR6314034    SRX3413965    PRJNA418990    SAMN08040264
```

Installation of EDirect on the Cloud

To install the EDirect software, open a terminal window and execute the following command:

```
sh -c "$(curl -fsSL https://ftp.ncbi.nlm.nih.gov/entrez/entrezdirect/install-edirect.sh)"
```

At the end of installation, answer "y" to have the script run the PATH update command to edit your configuration file, so that EDirect programs can be run in subsequent terminal sessions.

One installation is complete, run:

```
export PATH=${PATH}:${HOME}/edirect
```

to set the PATH for the current terminal session.

Downloading BLAST Software

Obtain the Magic-BLAST software if it is not already installed:

```
download-ncbi-software magic-blast
```

Preparing Chromosome Files

Retrieve the sequence of human chromosome 7 with:

```
efetch -db nuccore -id NC_000007 -format fasta -immediate > NC_000007.fsa
```

Download protein-coding genes on human chromosome 7 in document summary format, and extract the gene ranges into a tab-delimited table:

```
esearch -db gene -query "Homo sapiens [ORGN] AND 7 [CHR]" |
efilter -status alive -type coding | efetch -format docsum |
gene2range "7" > NC_000007.gen
```

SRA Gene Analysis

Run a **magicblast** search of SRR6314034 against chromosome 7:

```
magicblast -sra SRR6314034 -subject NC_000007.fsa -outfmt asn |
asn2xml > SRR6314034.xml
```

Passing the alignment details through the **blst2tkns** script:

```
cat SRR6314034.xml |
blst2tkns |
```

tokenizes the alignment parts:

```
index      1
score      6268
start      33088037
stop       33167397
strand     plus
match      12
mismatch   1
genomic-ins 1
...
```

Piping these to the **split-at-intron** script:

```
split-at-intron |
```

detects large genomic insertions:

```
6268    1    plus    33088037..33091003, 33163798..33167397
4910    2    plus    33089462..33091003, 33163799..33167397
1814    3    plus    33164965..33167397
...
```

These are filtered by minimum score with:

```
filter-columns '$1 > 1000' |
```

The **fuse-ranges** script then merges overlapping alignments:

```
fuse-ranges |
```

into a minimal set of extended ranges:

```
plus      33088037      33091003      2967
plus      33163798      33167398      3601
plus      33272248      33272278      31
plus      33394238      33394551      314
minus     33088037      33091004      2968
minus     33163798      33167398      3601
minus     33255886      33256012      127
minus     33394326      33394551      226
```

Running each range through the **find-in-gene** script:

```
while read std min max len
do
```

```
cat NC_000007.gen |
find-in-gene "$std" "$min" "$max"
done |
sort -f | uniq -i
```

returns the names of gene(s) that overlap the aligned segments:

```
BBS9
```

PubChem

PubChem in Entrez

Entrez can search on the complete synonym of a compound:

```
esearch -db pccompound -query "catechol [CSYN]" |
efetch -format uid
```

to return a small number of closely matching compound identifiers (CIDs):

```
73160
9064
289
```

Entrez document summaries for a compound:

```
efetch -db pccompound -id 289 -format docsum |
xtract -pattern DocumentSummary -element MolecularFormula IsomericSmiles
```

contains general descriptive information fields for the compound:

```
C6H6O2      C1=CC=C(C(=C1)O)O
```

Power User Gateway (PUG) REST Query Form

PubChem also supports a RESTful service for more advanced queries. Nquire provides a -pugrest URL shortcut. The base form of a search request is is:

```
nquire -pugrest [compound|substance|assay] [input] [operation] [output]
```

Searches can use the name of a compound:

```
nquire -pugrest compound name catechol cids TXT
```

to obtain the best matching compound identifier:

```
289
```

The CID can be used as the input key to obtain a title and description:

```
nquire -pugrest compound cid 289 description XML
```

or to retrieve a much more detailed record:

```
nquire -pugrest compound cid 289 record XML |
```

that includes the canonical or isomeric SMILES codes:

```
xtract -pattern PC-InfoData \
-if PC-Urn_label -equals SMILES -and PC-Urn_name -equals Isomeric \
-element PC-InfoData_value_sval
```

PubChem Chemical Identifiers

Certain identifier types require POST arguments to encode special symbols:

```
nquire -pugrest compound smiles description XML \
  -smiles "C1=CC=C(C(=C1)O)O" |
xtract -pattern InformationList -element Title Description
```

This returns the chemical name and description:

```
Catechol      Catechol is a benzenediol comprising...
```

Other identifier key types that are encoded in separate arguments are shown below:

```
nquire -pugrest compound inchi synonyms TXT \
  -inchi "1S/C6H6O2/c7-5-3-1-2-4-6(5)8/h1-4,7-8H"

nquire -pugrest compound inchikey cids JSON \
  -inchikey "YCIMNLLNPGFGHC-UHFFFAOYSA-N"

nquire -pugrest compound/fastsubstructure/smarts/cids/XML \
  -smarts "[#7]-[#6]-1=[#6]-[#6](C#C)=[#6](-[#6]-[#8])-[#6]=[#6]-1"
```

(Nquire -inchi will supply the expected "InChI=" prefix if it is missing in the argument string.)

PUG-REST Asynchronous Queries

Some PUG-REST queries are computationally intensive and run asynchronously:

```
nquire -pugrest compound/superstructure/cid/2244/XML |
```

The returned <ListKey> token is piped to an nquire -pugwait command, which polls the server until the results are available:

```
nquire -pugwait
```

Identifiers are then downloaded and placed directly in an ENTREZ_DIRECT message:

```
<ENTREZ_DIRECT>
  <Db>pccompound</Db>
  <Count>3750</Count>
  <Id>87</Id>
  <Id>175</Id>
  <Id>176</Id>
  ...
  <Id>162400221</Id>
  <Id>162416056</Id>
  <Id>162417911</Id>
</ENTREZ_DIRECT>
```

Support for this form was added when EDirect was redesigned in 2020.

Bibliometrics

Reverse Chronological Order

Repackaging an entire document summary and creating a set of keys for sorting:

```
esearch -db pubmed -query "tn3 transposition immunity" |
efetch -format docsum |
```

```
xtract -rec Rec -pattern DocumentSummary -INDX "+" \
-group DocumentSummary -pkg SortKeys \
-unit DocumentSummary -wrp INDX -element "&INDX" \
-unit PubDate -wrp YEAR -year PubDate \
-unit PubDate -wrp DATE -date "*" \
-unit Title -wrp TITL -lower Title \
-unit Author -position first -wrp FAUT -lower Name \
-unit Author -position last -wrp LAUT -lower Name \
-unit Authors -wrp ANUM -num Author/Name \
-unit DocumentSummary -wrp SIZE -len "*" \
-group DocumentSummary -pkg DS -element "*" |
```

produces an intermediate structure with separate containers for the sort keys and the original docsum:

```
...
<Rec>
  <SortKeys>
    <INDX>5</INDX>
    <YEAR>1989</YEAR>
    <DATE>1989/04</DATE>
    <TITL>nucleotide sequences required for tn3 transposition immunity.</TITL>
    <FAUT>kans ja</FAUT>
    <LAUT>casadaban mj</LAUT>
    <ANUM>2</ANUM>
    <SIZE>1695</SIZE>
  </SortKeys>
  <DS>
    <DocumentSummary>
      <Id>2539356</Id>
      <PubDate>1989 Apr</PubDate>
      <Source>J Bacteriol</Source>
      ...
    </DocumentSummary>
  </DS>
</Rec>
...
```

This allows sorting of records by a new tag derived from `-year`:

```
xtract -rec Rec -pattern Rec -sort-rev SortKeys/YEAR |
xtract -set DocumentSummarySet -pattern Rec \
-group DS/DocumentSummary -element "*" |
transmute -format -combine |
xtract -pattern DocumentSummary -element Id PubDate
```

to present publication records with the most recent shown first:

```
36257990    2022 Oct 18
28096365    2017 Jan 31
22624153    2012 Feb
21729108    2011 Sep
8595595     1996 Jan
```

Count Unique Journals

Some bibliometric analyses require fetching huge numbers of PubMed records. One such example is counting the number of unique journals publishing papers indexed under a given category.

A common function consolidates code to execute a query and fetch the records from the EDirect local PubMed archive. The `xtract -histogram` shortcut acts as an `-element` command followed by a built-in `sort-uniq-count`.

(This can save significant time when the number of results is in the millions.) The line count gives the number of journals publishing at least one paper that satisfies the query:

```
CountUniqueJournals() {
    qry="$1"

    # count number of unique journals publishing papers that match a query
    phrase-search -db pubmed -query "$qry" | fetch-pubmed |
    xtract -pattern PubMedArticle -histogram Journal/ISOAbbreviation |
    wc -l | tr -d ' '
}
```

Looping through a series of years, and calling this function with and without a filter of interest in the query:

```
for year in {2016..2022}
do

    # want to find percent of journals still using unstructured date
    filt="medline date [PROP]"
    base="journal article [PTYP] AND $year [YEAR]"

    # subset of journals publishing a paper with filter condition
    subs=$( CountUniqueJournals "$base AND $filt" )

    # number of unique journals in selected publication types
    totl=$( CountUniqueJournals "$base" )

    frac="-"
    if [ "$totl" -gt 0 ]
    then
        # calculate (integer) percentage
        frac=$((subs * 100 / totl))
    fi

    # print year, journal counts, and percent using filter condition
    printf "$year\t$subs\t$totl\t$frac\n"

done | align-columns -h 2 -g 4 -a r
```

returns the year, the numbers of unique journals with the filter and in total for that year, and the ratio of the two as a percentage. In this example the last column shows a steady decrease in the percentage of journals providing an unstructured publication date:

2016	1933	10362	18
2017	1153	10473	11
2018	1071	10321	10
2019	980	10027	9
2020	919	10579	8
2021	950	10947	8
2022	835	10600	7